# Natural Language Processing

Course Notes

## Contents


**1 The Birth of Human Language** **5**
- 1.1 Human Language vs. Animal Communication . . . 5
  - 1.1.1 Hint 1: talking about what is not here and now . . . 5
  - 1.1.2 Hint 2: the bees, an exception . . . 6
- 1.2 When Did Language Appear? . . . 7
  - 1.2.1 The timeline of humankind . . . 7
  - 1.2.2 What recent research says . . . 8
  - 1.2.3 Did the Neanderthals speak? . . . 8
- 1.3 Why Did Language Evolve? . . . 9
  - 1.3.1 The obstetrical dilemma . . . 9
  - 1.3.2 Other key drivers . . . 10
  - 1.3.3 Three theories of how language emerged . . . 10
- 1.4 The Role of Gossip . . . 11
  - 1.4.1 Functions of early gossip . . . 12

**2 What Is Natural Language Processing?** **13**
- 2.1 About the Course . . . 13
- 2.2 Definition of NLP . . . 14
- 2.3 A Short History of NLP . . . 15
  - 2.3.1 Symbolic NLP (1950s – early 1990s) . . . 15
  - 2.3.2 Statistical NLP (1990s – 2010s) . . . 15
  - 2.3.3 Neural NLP (2010s) . . . 16
  - 2.3.4 Transformers (“it seems a long time ago...”) . . . 16
  - 2.3.5 Generative AI and LLMs (present) . . . 16
- 2.4 Why Traditional NLP Still Matters . . . 16
- 2.5 Case Study: ChatBDI . . . 17
  - 2.5.1 The idea: think BDI, talk LLM . . . 17
  - 2.5.2 Scenarios . . . 18
  - 2.5.3 Architecture (kept simple) . . . 18
  - 2.5.4 Experiments: it is a hard problem . . . 19

**3 Levels of Linguistic Analysis** **20**
3.1 The Three Levels . . . 20
3.2 Analogy with Programming Languages . . . 21
3.3 A Map of the Course . . . 22
3.3.1 Syntax . . . 22
3.3.2 Semantics . . . 23
3.3.3 Transformers and LLMs . . . 23
3.3.4 Features for representing a text . . . 23
3.3.5 Pragmatics . . . 24

**4 Terminology** **24**
4.1 Morphemes, Words and Compounds . . . 25
4.1.1 Morphemes . . . 25
4.1.2 Derivational and inflectional morphemes . . . 26
4.1.3 Words . . . 26
4.1.4 Compounds . . . 27
4.2 Parts of Speech: a Grammar Recap . . . 27
4.2.1 Overview . . . 28
4.2.2 Nouns . . . 29
4.2.3 Pronouns . . . 29
4.2.4 Verbs . . . 30
4.2.5 Adjectives . . . 32
4.2.6 Adverbs . . . 33
4.2.7 Determiners and articles . . . 33
4.2.8 Prepositions . . . 34
4.2.9 Conjunctions . . . 36
4.2.10 Interjections, numerals and particles . . . 36
4.2.11 Worked example: analisi grammaticale in English . . . 37
4.2.12 From words to sentences: analisi logica . . . 37
4.3 Corpora, Genre and Collocations . . . 38
4.3.1 Corpus . . . 38
4.3.2 Genre . . . 39
4.3.3 Collocations . . . 39
4.4 Precision and Recall . . . 40
4.4.1 The same thing with a confusion matrix . . . 41
4.4.2 Four worked examples . . . 41
4.4.3 Another example: five algorithms . . . 43
4.4.4 Exercises . . . 44

**5 Regular Expressions** **45**
5.1 The Syntax of Regular Expressions . . . 46
5.1.1 Disjunction: square brackets and the pipe . . . 46
5.1.2 Negation inside brackets . . . 46
5.1.3 Repetition: `?`, `*`, `+` and the wildcard `.` . . . 47
5.1.4 Anchors: `^` and `$` . . . 47

5.1.5 Summary of the operators . . . 47
5.2 Worked Example: Finding "the" . . . 48
5.3 ELIZA: Conversation by Substitution . . . 49
5.3.1 A tiny ELIZA in Python . . . 49
5.3.2 Exercises . . . 50

**6 Syntax at the Word Level** **51**
6.1 Word Segmentation . . . 51
6.1.1 Why it is not trivial . . . 51
6.1.2 The MaxMatch algorithm . . . 52
6.1.3 NLTK . . . 52
6.2 Stop Words and TF-IDF . . . 53
6.2.1 Stop words . . . 53
6.2.2 Frequent in the document or frequent in the language? . . . 53
6.2.3 Term frequency and inverse document frequency . . . 54
6.2.4 Worked example on $C_2$ . . . 55
6.3 Stemming and Lemmatisation . . . 56
6.3.1 The goal: one form for many . . . 56
6.3.2 Stemming vs. lemmatisation . . . 57
6.3.3 The Porter stemmer . . . 57
6.3.4 The seven steps . . . 58
6.3.5 Examples and problems . . . 59
6.3.6 Two texts to test a stemmer . . . 60
6.4 Minimum Edit Distance . . . 61
6.4.1 How similar are two strings? . . . 61
6.4.2 Definition . . . 62
6.4.3 Edit distance as search . . . 63
6.4.4 The algorithm . . . 63
6.4.5 The table and the backtrace . . . 64
6.4.6 Exercises . . . 65

**7 Syntax at the Sentence Level** **66**
7.1 Sentence Breaking . . . 67
7.2 Part-of-Speech Tagging . . . 68
7.2.1 The Penn Treebank tagset . . . 68
7.3 Grammars and Parsing . . . 69
7.3.1 Chomsky and generative grammar . . . 70
7.3.2 Grammars and the Chomsky hierarchy . . . 70
7.3.3 A grammar for the Wumpus world . . . 71
7.3.4 Definite Clause Grammars . . . 73
7.4 Statistical POS Tagging: Hidden Markov Models . . . 74
7.4.1 The model . . . 74
7.4.2 The weather example . . . 75
7.4.3 The Viterbi algorithm . . . 75
7.4.4 From weather to POS tags . . . 77

7.4.5 Maximum Entropy Markov Models . . . 77
7.4.6 Exercises . . . 78

**8 Distributional Semantics** **79**
8.1 The Distributional Hypothesis . . . 79
8.1.1 Meaning and distribution . . . 79
8.1.2 What is the meaning of "bardiwac"? . . . 80
8.2 The Co-occurrence Matrix . . . 80
8.2.1 A thought experiment: deciphering hieroglyphs . . . 80
8.2.2 English as seen by the computer . . . 81
8.3 Geometric Interpretation . . . 82
8.3.1 Words as points . . . 82
8.3.2 Direction, not location . . . 83

**9 Frame and Model-Theoretic Semantics** **84**
9.1 Frame Semantics . . . 84
9.1.1 Word meaning revisited . . . 85
9.1.2 Two properties of frames . . . 85
9.1.3 From syntax to semantics . . . 86
9.1.4 Semantic role labelling . . . 86
9.2 Three Types of Semantics . . . 86
9.3 Model-Theoretic Semantics in Practice: Semantic Parsing . . . 88
9.3.1 Executable semantic parsing . . . 88
9.3.2 A brief history of semantic parsing . . . 89
9.3.3 Compositional semantics . . . 89
9.3.4 Language variation . . . 90

**10 Lexical Semantics, WSD and NER** **90**
10.1 The Lexicon and Lexical Relations . . . 91
10.1.1 Lexicon and lexical units . . . 91
10.1.2 Synonymy . . . 92
10.1.3 Antonymy . . . 92
10.1.4 Hyponymy and hypernymy . . . 92
10.1.5 Meronymy and holonymy . . . 93
10.1.6 Homonymy . . . 93
10.2 Thesauri and WordNet . . . 94
10.2.1 Structured lexicons and thesauri . . . 94
10.2.2 WordNet . . . 94
10.3 Word Sense Disambiguation . . . 96
10.3.1 The Lesk algorithm . . . 96
10.3.2 Extensions and limitations . . . 97
10.4 Named Entity Recognition . . . 98

# 1 The Birth of Human Language

[Slides · Intro: 4–21]

Before teaching computers to process language, it is worth asking what language *is* and where it comes from. This section is not technical: it frames the object of study of the whole course. The questions it raises (what makes human language special? why did it appear?) come back later, for example when we see that language is used not only to describe the world but also to *act* on other people (pragmatics, Section 3).

This section covers:

1. what distinguishes human language from animal communication (Section 1.1);
2. when language appeared, on the timeline of human evolution (Section 1.2);
3. the evolutionary pressures that may have pushed our species towards language, and the main theories about how it emerged (Section 1.3);
4. the "gossip" hypothesis: language as a tool for managing social relationships (Section 1.4).

## 1.1 Human Language vs. Animal Communication

[Slides · Intro: 5–8]

Many animals communicate: birds sing to mark territory, vervet monkeys have different alarm calls for eagles, leopards and snakes, dogs growl and wag their tails. So what is special about *human* language? The course poses this as an open question and gives two hints.

### 1.1.1 Hint 1: talking about what is not here and now

[Slides · Intro: 5–7]

The first hint is given only through two pictures: a portrait of the Latin poet **Horace** and the poster of the film ***Dead Poets Society***. What links them is the phrase *carpe diem* ("seize the day"): it comes from one of Horace's odes, and it is the motto that the teacher in the film gives his students.

The most likely reading of the hint is this: *carpe diem* is a sentence about *time*. It urges you to act now because the future is uncertain and the past is gone. To say it at all, a language must be able to talk about things that are not in front of the speaker: the future, the past, possibilities, abstract ideas. Animal signals almost never do this: an alarm call means "danger *here*, *now*".

**Definition — Displacement**

A communication system has **displacement** if its messages can refer to things that are *removed in time or space* from the moment and place of communication:

past and future events, distant places, hypothetical or imaginary entities, abstract concepts.

Displacement is one of the "design features" of language listed by the linguist Charles Hockett (1960). Others worth knowing, because they will come back in this course, are:

- **productivity**: a finite set of words and rules generates an unbounded number of new sentences, never heard before, that are still understood;
- **duality of patterning**: meaningless units (sounds, letters) combine into meaningful units (morphemes, words; see Section 4.1), which combine into sentences;
- **arbitrariness**: there is no natural link between a word and what it means (*dog*, *cane*, *Hund* all name the same animal);
- **cultural transmission**: a language is learnt from the community, not inherited genetically.

### 1.1.2 Hint 2: the bees, an exception

[Slides · Intro: 8]

The second hint says that the language of bees is an *exception* to the most evident difference between human language and animal communication: bees convey a specific piece of information that no other animal conveys in the wild.

That piece of information is *the location of a distant food source*. When a forager bee comes back to the hive, it performs the **waggle dance** (decoded by Karl von Frisch, Nobel Prize 1973):

- the **angle** $\alpha$ of the straight "waggle run" with respect to the vertical of the honeycomb equals the angle between the direction of the food and the direction of the sun;
- the **duration** of the waggle run encodes the **distance** to the food (longer run, farther food).

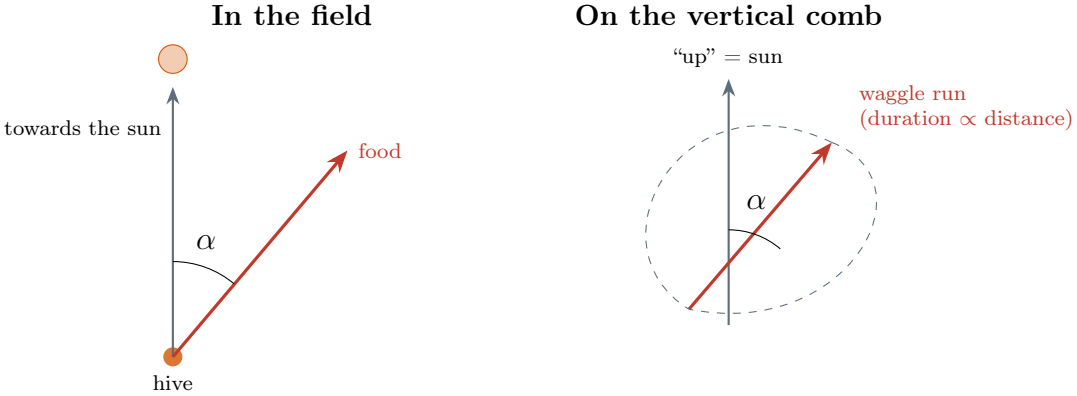


Figure 1: The waggle dance. The same angle $\alpha$ appears twice: in the field it separates the sun from the food, on the comb it separates "up" from the waggle run. The dashed loops are the return paths the bee walks between two waggle runs.

So bees can talk about something *far away*: their communication has a (limited) form of spatial displacement. It is still very far from human language: it covers only one topic (food and nest sites), it is not productive (bees cannot combine dances into new messages about new topics) and it is innate, not learnt.

**Intuition — the answer in one line**

Human language is (among other things) *displaced*: it lets us talk about the past, the future, the absent and the imaginary. Animal communication is almost always about the here and now; the bees' dance is the classic exception, because it describes a place the bees are not currently in.

## 1.2 When Did Language Appear?

[Slides · Intro: 9–12]

### 1.2.1 The timeline of humankind

[Slides · Intro: 9–10]

Table 1 places our species on the timeline of human evolution. It is adapted from Yuval Noah Harari's *Sapiens: A Brief History of Humankind* (2014). The abbreviations are MYA = million years ago, YA = years ago.

| When | What happened |
|---|---|
| 6 MYA | Last common "grandmother" of humans and chimpanzees |
| 2.5 MYA | Evolution of the genus *Homo* in Africa; first stone tools |
| 2 MYA | **Humans spread from Africa to Eurasia**; evolution of different human species (*Homo rudolfensis* and *Homo ergaster* in Africa, *Homo erectus* in East Asia) |
| 500,000 YA | Neanderthals evolve in Europe and the Middle East |
| 200,000 YA | *Homo sapiens* evolves in East Africa |
| 70,000 YA | **Sapiens spreads out of Africa** |
| 70,000–30,000 YA | **Cognitive revolution**: invention of boats, arrows, needles; beginnings of art, religion, commerce, social stratification, **and language** |
| 45,000 YA | Sapiens settles Australia |
| 30,000 YA | Extinction of Neanderthals |
| 16,000 YA | Sapiens settles America |
| 13,000 YA | Extinction of *Homo floresiensis*: **Homo sapiens is the only surviving human species** |
| 12,000 YA | Agricultural revolution, permanent settlements |

Table 1: Milestones of human evolution (adapted from Harari). Note how recent everything after the cognitive revolution is compared with the 6 million years of the whole story.

The course also shows a map of the movement of *Homo sapiens* out of Africa: from East Africa into the Middle East, then along the coasts of southern Asia to Australia (about 50,000 years ago), into Europe and northern Asia, and finally across the Bering region into America (about 15,000 years ago), while Neanderthals already lived in Europe and *Homo erectus* in Asia.

### 1.2.2 What recent research says

[Slides · Intro: 11]

Harari places the birth of language in the cognitive revolution, about 70,000–30,000 years ago. A 2025 study reported by MIT News ("When did human language emerge?", 14 March 2025) argues for an earlier date:

- **135,000 years ago**: genomic and geographic data suggest that the biological and cognitive *capacity* for language was already fundamentally present. The argument is that the earliest split between human populations happened about 135,000 years ago, and since *all* human populations have language, the capacity must predate the split.
- **100,000 years ago**: archaeological records show an explosion of symbolic behaviours, like art and complex tools, indicating active social communication. Language was being *used* socially.
- **Gradual origins**: scientists widely agree that language developed from a complex mix of hand gestures, facial expressions and vocalisations over time, not in a single step.

**Watch out — capacity vs. use**

The two dates answer different questions. "The capacity for language existed 135,000 years ago" is about biology (the brain and vocal tract could support language). "Symbolic behaviour exploded 100,000 years ago" is about culture (people were actually communicating with symbols). Do not mix them up when asked "when did language appear?".

### 1.2.3 Did the Neanderthals speak?

[Slides · Intro: 12]

The course leaves this as a question for discussion. There is no definitive answer, but these are the main pieces of evidence usually brought up:

- **for**: Neanderthals had the same variant as ours of the *FOXP2* gene, involved in speech and language; a Neanderthal hyoid bone (the bone that supports the

tongue) found at Kebara, Israel, is almost identical to ours; they made complex tools, used fire, buried their dead and probably made simple symbolic objects;
- **against / uncertain**: their symbolic record is much poorer than that of *Sapiens* of the same period, and soft tissue (the vocal tract, the brain itself) does not fossilise, so the anatomy of speech can only be inferred.

Many researchers today think Neanderthals had *some* form of language, though possibly less complex than ours. Note that if language capacity predates 135,000 years ago in *Sapiens*, the question of whether it also predates the split between *Sapiens* and Neanderthals (several hundred thousand years ago) remains open.

## 1.3 Why Did Language Evolve?

[Slides · Intro: 13–18]

Knowing *when* language appeared does not tell us *why*. This part of the course follows the linguist Sverker Johansson (*The Dawn of Language: How We Came to Talk*), for whom one major driver was the **obstetrical dilemma**.

### 1.3.1 The obstetrical dilemma

[Slides · Intro: 13–16]

Two evolutionary trends pulled the human body in opposite directions:

- **Upright walking (bipedalism)** restructured the human pelvis into a basket-like shape that supports the internal organs and makes walking efficient, but *narrows the birth canal.*
- **Large brains**: hominid brain size increased dramatically over millions of years, so the baby's head became large compared with the mother's pelvis.

**Consequence**: labour and delivery are difficult and dangerous in the human species (not only in humans, but especially in humans).

Evolution's "compromise" was to have babies born **immature**: their brain keeps growing a lot after birth. Human newborns cannot even cling to their mother, as the babies of all other higher primates do, and they need the protection of their family or social group for a very long time.

**Intuition — how this leads to language**

The course asks: how might these facts relate to the birth of language? Follow the chain:

1. A difficult birth means a mother needs help: **midwives** and social support. Giving birth becomes a social, cooperative event.
2. A baby that cannot cling must be carried or **put down**. A mother who puts the baby down to gather food can still soothe and monitor it from a distance with her **voice** ("motherese").

3. A long childhood is a long **window for learning**: there is time to absorb something as complex as a language from the community.

Table 2 summarises the argument.

| **Evolutionary pressure** | **Physical consequence** | **Behavioural and language outcome** |
|---|---|---|
| Bipedalism and big brains | Narrow birth canal; difficult, dangerous labour | Need for **midwives and social support**, forcing cooperative social structures |
| Neurological immaturity | Babies are born completely helpless and cannot cling | Mothers must **"put the baby down"**, using vocalisations to soothe and monitor them from afar |
| Extended childhood | Years of dependency requiring group protection | Long childhoods provide a **massive window for learning** complex language from the community |

Table 2: From the obstetrical dilemma to language.

### 1.3.2 Other key drivers

[Slides · Intro: 17]

Besides the obstetrical dilemma, three more drivers are usually cited:

- **Survival and cooperation**: early humans needed to coordinate group hunts, plan defences against predators and share crucial resources.
- **Social bonding**: communication grew from basic physical grooming into storytelling around fires, building community trust and deep empathy (this is developed in Section 1.4).
- **Abstract thought**: language enabled humans to refer to things not immediately present, like the past, the future or imaginary cultural concepts. This is displacement again (Section 1.1).

### 1.3.3 Three theories of how language emerged

[Slides · Intro: 18]

There are three main families of theories. They differ on *speed* (gradual or sudden) and on the *starting medium* (voice or gestures).

| | Continuity theory | Discontinuity theory | Gesture-first theory |
|---|---|---|---|
| Core idea | Language evolved **gradually** over millions of years out of animal communication systems | Language appeared **suddenly** as a unique, exclusively human genetic mutation | Language originated from **physical gestures** and manual signs before shifting to speech |
| Key mechanism | Natural selection slowly shaped the vocal tract and brain tissue to handle complex communication | A sudden cognitive "leap" or single genetic mutation re-wired the brain for universal grammar | Bipedalism freed the hands, allowing early humans to use detailed hand and facial signs to communicate |
| Primary driver | The gradual need for social bonding and survival coordination in expanding groups | An internal computational upgrade in the brain, prioritising thought organisation over communication | The mechanical advantage of visual "miming" before vocalisation evolved to take over |
| Famous proponents | Charles Darwin | Noam Chomsky | Michael Corballis |

Table 3: The three main theories on the emergence of language.

**Intuition — telling them apart**

- **Darwin**: language is like any other trait, shaped slowly by natural selection; "more of" what animals already have.
- **Chomsky**: language is primarily a tool for *thinking*, and its core (the ability to combine units recursively, "universal grammar") appeared in one jump; communication came second.
- **Corballis**: language started in the *hands*; the voice took over later because it works in the dark, at a distance and with the hands busy.

Chomsky's name will come back in this course for a very different reason: his work on formal grammars underlies the syntactic parsing we will study later.

## 1.4 The Role of Gossip

[Slides · Intro: 19–21]

One of the most popular explanations of *why* language evolved is surprisingly down-to-earth: **gossip**. The idea is due to the anthropologist Robin Dunbar (*Grooming, Gossip and the Evolution of Language*, 1996), and Harari's *Sapiens* makes it widely known.

**Key result — Gossip as vocal grooming**

Gossip acted as a *vocal substitute for physical grooming* in primates, allowing early humans to bond and manage social relationships in groups too large for direct touch alone.

The argument has three steps:

1. **Primate roots.** Primates maintain social bonds and group cohesion through physical grooming (picking ticks and fleas from each other).
2. **The group size limit.** Grooming is one-to-one and takes time. As early human groups grew larger, physical grooming became too time-consuming to bond with every member.
3. **Vocal grooming.** Language evolved as a form of "vocal grooming" that lets one person bond with *several* others at the same time, instead of one at a time.

**Intuition — why the numbers matter**

If grooming one partner takes a fixed amount of time per day, the time a primate must spend grooming grows with the size of its group, and there are only so many hours in a day. Dunbar estimated that the natural human group size ("Dunbar's number", about 150 people) would require spending around 40% of the day grooming. Talking to three or four listeners at once removes that bottleneck.

### 1.4.1 Functions of early gossip

[Slides · Intro: 21]

Gossip was not idle chatter. It did three useful jobs for the group:

- **Information sharing**: it transmitted crucial *reputational data*: who could be trusted, and who was a free rider (someone who takes advantage of the group without contributing).
- **Social cohesion**: it strengthened group identity, enforced social norms and managed complex social networks without written records.
- **Conflict reduction**: it reduced friction inside the group by aligning moral judgements and expectations.

**Why this matters for NLP**

Two ideas from this section shape how we look at language in NLP:

- language is mostly about *people* and their relationships, not just a code for describing facts. A lot of real text (social media, chats, reviews) is exactly "gossip" in this sense, and analysing it means dealing with opinions, emotions and social intentions;
- language is used to *do* things to others (to bond, to warn, to persuade), which is the subject of **pragmatics** (Section 3).

**Cheat sheet — the birth of human language**

- Key difference from animal communication: **displacement** (talking about the past, the future, the absent, the imaginary). The bees' waggle dance is the classic exception: it describes the direction and distance of distant food.
- Harari: language arises with the **cognitive revolution**, 70,000–30,000 years

ago. Recent studies: the *capacity* was present by **135,000** years ago, symbolic *use* exploded around **100,000** years ago; origins were gradual (gestures, faces, voice).
- **Obstetrical dilemma** (Johansson): bipedalism + big brains ⇒ difficult births and immature babies ⇒ need for social support, vocal soothing at a distance and a long learning window.
- Other drivers: survival and cooperation, social bonding, abstract thought.
- Theories: **continuity** (gradual; Darwin), **discontinuity** (sudden mutation; Chomsky), **gesture-first** (hands before voice; Corballis).
- **Gossip** (Dunbar): language as vocal grooming for groups too large for physical grooming; it shares reputational data, builds cohesion and reduces conflict.

# 2 What Is Natural Language Processing?

[Slides · Intro: 1–3; Terminology introduction: 1–22]

This section introduces the field: what the course covers, what NLP is, how it evolved from hand-written rules to large language models, why the "old" methods are still needed, and a real research example showing how hard the problems are even today.

This section covers:

1. how the course is organised (Section 2.1);
2. the definition of NLP and its two directions, understanding and generation (Section 2.2);
3. the five eras of NLP (Section 2.3);
4. when traditional NLP is still required (Section 2.4);
5. a case study: ChatBDI, letting software agents talk in natural language (Section 2.5).

## 2.1 About the Course

[Slides · Intro: 1–3; Terminology introduction: 2]

The course (taught by Viviana Mascardi, University of Genoa) loosely follows the textbook *Speech and Language Processing* (3rd edition draft) by Dan Jurafsky and James H. Martin, freely available at `https://web.stanford.edu/~jurafsky/slp3/`. Some slides integrate those associated with the book. There is no single reference book, however: the slides, plus the pointers to material on the web and on the course's AulaWeb page, are enough to understand the subject, even without attending the lessons.

- A large share of the lessons covers **"Annotating Linguistic Structure"** (Volume III of the book), extended with lessons on the birth of human language

(Section 1), semantics and pragmatics.
- The remaining lessons cover **Large Language Models** (Volume I).
- Topics of Volume I already covered in other Machine Learning courses (logistic regression, neural networks, RNNs, ...) are *not* repeated.

**Key result — Exam**

The exam has two parts: a **written exam** and an **individual project**. Students can propose the project themselves, so they are free to experiment in a practical way with the topics they liked most.

The (preliminary) programme for 2026–2027 is, in order: introduction and regular expressions; syntax at the word level (segmentation, stop words, TF-IDF, stemming, lemmatisation); minimum edit distance, the sentence level and rule-based POS tagging; grammars and statistical (Hidden Markov Model) POS tagging; semantics (distributional, frame, model-theoretic and lexical semantics, a comparison among types of semantics, ontologies and the Semantic Web); practical labs on NLP libraries, text summarisation, chatbots and agentic AI; pragmatics; a wrap-up on classical NLP; and finally BERT, GPT and LLMs. Section 3.3 shows how these topics fit together.

## 2.2 Definition of NLP

[Slides · Terminology introduction: 3]

**Definition — Natural Language Processing (NLP)**

**Natural language processing** is a subfield of linguistics, computer science and artificial intelligence concerned with the interactions between computers and human language. In particular, it studies how to program computers to
- process and analyse large amounts of natural language data;
- **understand** text: **Natural Language Understanding (NLU)**;
- **produce** understandable text in human languages from some underlying non-linguistic representation of information: **Natural Language Generation (NLG)**.

A "natural" language is one that evolved naturally among humans (English, Italian, Chinese, sign languages), as opposed to *formal* languages designed on purpose (Python, logic, arithmetic). Natural languages are ambiguous, redundant, constantly changing and full of exceptions; formal languages are designed to avoid all of that. This is exactly why processing them with a computer is hard.

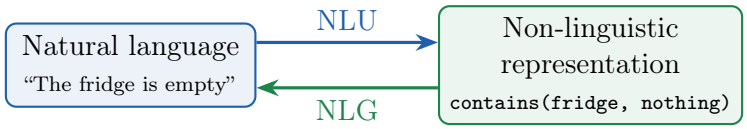



Figure 2: NLU and NLG go in opposite directions between text and an internal representation (here a logical fact a program can use).

## 2.3 A Short History of NLP

[Slides · Terminology introduction: 4–8]

NLP has gone through five main eras. Each changed *where the knowledge about language comes from*: first from people writing rules, then from statistics over corpora, then from neural networks learning their own representations.

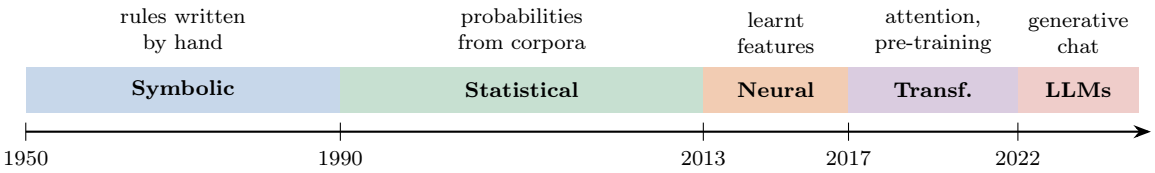



Figure 3: The eras of NLP. Boundaries are approximate and the eras overlap: the neural era is usually dated from word embeddings (around 2013), the transformer era from the 2017 transformer paper, the LLM era from the release of ChatGPT (2022).

### 2.3.1 Symbolic NLP (1950s – early 1990s)

[Slides · Terminology introduction: 4]

Programs based on **hand-written rules** and knowledge bases. Classic systems:

- **ELIZA** (1966, Joseph Weizenbaum, MIT): a chatbot imitating a psychotherapist, using simple pattern matching and substitution ("I am sad" → "Why are you sad?"). It understood nothing, yet many users attributed real understanding to it.
- **SHRDLU** (1968–1970, Terry Winograd, MIT): a program that understood commands in English about a small "blocks world" ("put the red pyramid on the blue block") and could answer questions about it.
- **PARRY** (1972, Kenneth Colby, Stanford): a chatbot simulating a person with paranoid schizophrenia; famously, it was made to "talk" with ELIZA.
- **Ontologies** (e.g. MARGIE, 1973, Roger Schank): representing the meaning of sentences with structured conceptual knowledge.

Links: https://en.wikipedia.org/wiki/ELIZA, https://en.wikipedia.org/wiki/SHRDLU, https://en.wikipedia.org/wiki/PARRY, https://www.ijcai.org/Proceedings/73.

### 2.3.2 Statistical NLP (1990s – 2010s)

[Slides · Terminology introduction: 5]

- **Large corpora** became available, also multilingual (e.g. parliament proceedings translated into many languages).
- Statistical models make **soft, probabilistic decisions** by attaching real-valued weights to each input feature, instead of applying hard rules.

- Example: **part-of-speech tagging with Hidden Markov Models** (covered later in the course).

### 2.3.3 Neural NLP (2010s)

[Slides · Terminology introduction: 6]

- **End-to-end learning** of a higher-level task (e.g. question answering), instead of relying on a pipeline of separate intermediate tasks (tokenise, tag, parse, . . . ).
- No need to **engineer features** by hand: the network learns its own representation of the language.
- A new paradigm, distinct from statistical NLP (example: **neural machine translation**).
- Example: **word embeddings**, vectors representing words so that words with similar meaning are close in space.

### 2.3.4 Transformers ("it seems a long time ago. . . ")

[Slides · Terminology introduction: 7]

**BERT** (Bidirectional Encoder Representations from Transformers, Google) is a model pre-trained on huge amounts of text and then fine-tuned for specific tasks. It is built on the **transformer** architecture, introduced in the paper *Attention Is All You Need* (Vaswani et al., NeurIPS 2017).

**Watch out — dates and the link on the slide**

The slide says "BERT, Google (2019)" and links to a NeurIPS 2017 paper. That link is the *transformer* paper (*Attention Is All You Need*), not the BERT paper. BERT (Devlin et al.) was released in October 2018 and published at NAACL in 2019, so both 2018 and 2019 are found in the literature.

### 2.3.5 Generative AI and LLMs (present)

[Slides · Terminology introduction: 8]

- **GPT**: Generative Pre-trained Transformer. **ChatGPT** was released in November 2022.
- Other large language models and tools: Llama, Gemini, . . . ; and Ollama, a tool to run open models locally.

## 2.4 Why Traditional NLP Still Matters

[Slides · Terminology introduction: 9–10]

If LLMs can do almost everything, why study tokenisers, taggers and grammars? Because traditional NLP methods and approaches are still required in three situations.

**Key result — When traditional NLP is required**

1. **Not enough data**: when the amount of training data is insufficient to apply machine learning or deep learning methods successfully, or to train LLMs (e.g. low-resource languages, very specialised domains).
2. **Pre- and post-processing**: when NLP pipelines need preprocessing (e.g. tokenisation) or postprocessing and transformation of their output (e.g. extracting knowledge from syntactic parses). This includes *refining and validating the results* of ML, DL and LLM models.
3. **LLMs cannot be used**, for technical reasons (cost, latency, hardware), legal reasons (privacy, data that cannot leave the organisation) or ethical reasons (lack of transparency, bias).

**Intuition**

Even an LLM starts with a tokeniser, a very traditional NLP component. And when an LLM produces an answer that a program must use (a database query, a logical fact), checking that the answer is well formed and consistent is again a job for traditional methods. The case study below shows exactly this.

The course will therefore discuss (some of) the common problems of NLP following a *standard linguistic analysis approach* (Section 3), with examples of (some of) the common techniques and their common implementations.

## 2.5 Case Study: ChatBDI

[Slides · Terminology introduction: 11–22]

The course shows a real example from the lecturer's recent research, at the intersection of multi-agent systems (S&DAI) and NLP. It is useful early on because it shows, on a concrete problem, almost every topic of the course: syntax, semantics, pragmatics, edit distance and LLMs.

### 2.5.1 The idea: think BDI, talk LLM

[Slides · Terminology introduction: 12–13]

A **BDI agent** (Belief–Desire–Intention) is a software agent that reasons symbolically: it has *beliefs* about the world (facts such as `has(owner, beer)`), *desires* (goals) and *intentions* (plans it has committed to). BDI agents in a multi-agent system (MAS) talk to each other with an **Agent Communication Language (ACL)**: each message has a **performative** (the type of communicative act, e.g. `tell` to inform, `askOne`/`askAll` to ask a question, `achieve` to ask for a goal to be pursued) and a **content**, a logical term.

ChatBDI keeps the BDI agent as the "brain" and uses an LLM only as its **language sensor–actuator**: the LLM translates a human's natural-language sentence into an ACL message the agent understands (and the agent's answers back into natural

language). So humans can chat with agents that were never designed to understand natural language.

### 2.5.2 Scenarios

[Slides · Terminology introduction: 14–15]

- **Closed MAS**: a known set of agents, e.g. the classic domestic-robot example with a *robot*, an *owner* and a *supermarket* (the robot brings beer from the fridge to the owner and orders more from the supermarket). A user can ask "Which are the available agents?", ask the robot which plans it has, or ask it to describe a plan, and receives a natural-language answer.
- **Open MAS**: agents can join and leave, e.g. an auction of flight tickets where agents (Bob, Giacomo, Maria, . . . ) post bids such as "I have a bid for the flight from Athens to Paris on 18/12/2015 for 50 euros" and the winner is announced in natural language.

### 2.5.3 Architecture (kept simple)

[Slides · Terminology introduction: 16–17]

A human writes a sentence addressed to an agent, say "@Bob, . . . ". Before that, ChatBDI has asked Bob for its **lexicon**: the list of terms (predicates) Bob understands.

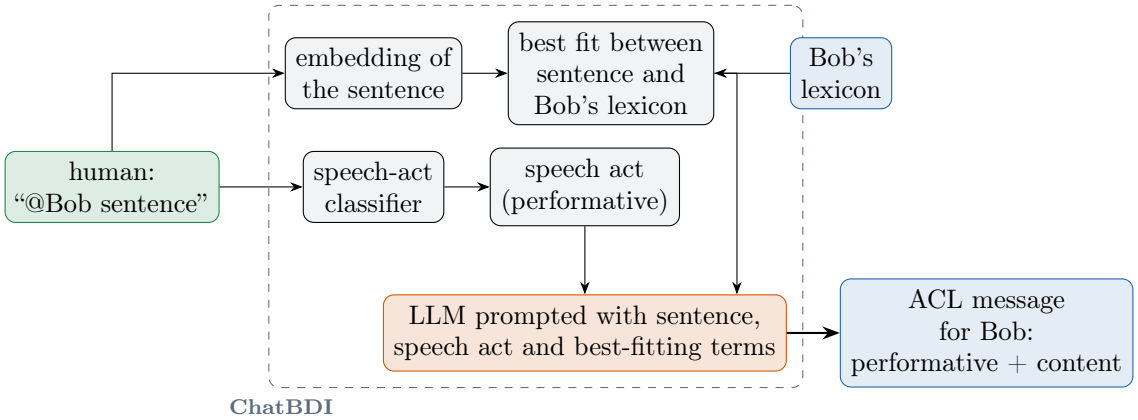



Figure 4: ChatBDI translating a human sentence into an ACL message. The performative comes from a classifier, the content from an LLM guided by the terms of the recipient's lexicon that best fit the sentence.

1. A **speech-act classifier** decides the performative: is the human informing (`tell`), asking (`askOne`, `askAll`), requesting an action (`achieve`)?
2. The sentence is turned into an **embedding** (a vector) and compared with Bob's lexicon to find the terms that **best fit** it.
3. An **LLM is prompted** with the sentence, the speech act and the best-fitting terms, and produces the content of the message in ACL.

### 2.5.4 Experiments: it is a hard problem

[Slides · Terminology introduction: 18–22]

Many LLMs were compared on the two sub-tasks: getting the **performative** right and getting the **message content** right (the latter scored lower overall). To measure how close a generated term is to the expected one, the experiments use the **Levenshtein distance**: the minimum number of character insertions, deletions and substitutions needed to turn one string into the other (the *minimum edit distance*, a topic of the course). Three examples show what can go wrong:

**Worked example — a partially correct translation**

Sentence: "Employer 7 attend JS course till 1/11/2025".

- Performative: `tell`, expected `tell`: correct.
- Term: `training_attended(7, JS, 1/11/2025)`; expected `training_attended(7, js, "1/11/2025")`.
- Levenshtein distance = 4: `J`→`j` and `S`→`s` (2 substitutions), plus the two quotation marks around the date (2 insertions).

Almost right, but for a symbolic agent `JS` and `js` are different constants and `1/11/2025` (an arithmetic expression!) is not the string `"1/11/2025"`: the agent's plan will not match. Generating the content took about 15 seconds.

**Worked example — right form, wrong meaning**

Sentence: "beans are very good for you, they contain 24 grams of protein per 100 grams".

- Performative: `tell`, correct.
- Term: `calories(beans, unit(24, gr), 100)`; expected `protein(beans, unit(100, gr), 24)`. Levenshtein distance = 13.

The term is syntactically perfect, but it says something different: the predicate is wrong (*calories* instead of *protein*) and the two quantities are swapped. This is a *semantic* error.

**Worked example — the wrong speech act**

Sentence: "Tell me the pressure of both front tires".

- Performative: `tell`; expected `askAll`: **incorrect**.
- Term: `tire_pressure(_, 28)`; expected `tire_pressure(front, _)`. Levenshtein distance = 7.

The sentence starts with the verb "tell", but the human is not *informing* the agent of anything: they are *asking* for information, about all ("both") front tyres, hence `askAll`. The surface form of a sentence and the act it performs can differ: this is a *pragmatic* error.

**Why this matters for NLP — the three errors**

The three examples are errors at the three levels of linguistic analysis of Section 3.1: wrong *form* of a constant (syntax), wrong *meaning* of the fact (semantics), wrong *act* performed by the sentence (pragmatics). They also show why traditional NLP is still needed next to LLMs: to check and repair the LLM's output (Section 2.4).

**Cheat sheet — what is NLP**

- NLP: computers processing human language; **NLU** (text → representation) and **NLG** (representation → text).
- Eras: **symbolic** (1950s–early 1990s; ELIZA, SHRDLU, PARRY, MARGIE), **statistical** (1990s–2010s; corpora, probabilities, HMM POS tagging), **neural** (end-to-end, learnt features, word embeddings), **transformers** (BERT), **generative LLMs** (GPT, ChatGPT 2022).
- Traditional NLP is still needed when data are scarce, for pre/post-processing and validation of model output, and when LLMs cannot be used (technical, legal, ethical reasons).
- ChatBDI: BDI agents "think", an LLM "talks"; a sentence becomes performative + content; errors appear at the syntactic, semantic and pragmatic level; Levenshtein distance measures how far the generated term is from the expected one.

# 3 Levels of Linguistic Analysis

[Slides · Terminology introduction: 23–33]

The course organises NLP problems following a standard linguistic analysis approach. This short section introduces the three levels of that analysis, explains them with an analogy from programming, and uses them as a map of the whole course.

This section covers:

1. the three levels: syntax, semantics, pragmatics (Section 3.1);
2. the analogy with programming languages (Section 3.2);
3. the map of the course topics, level by level (Section 3.3).

## 3.1 The Three Levels

[Slides · Terminology introduction: 23–24]

**Watch out — exam**

The lecturer is explicit: when the exam asks for "the three levels of linguistic analysis", it refers to **these three**: syntax, semantics and pragmatics.

**Definition — The three levels of linguistic analysis**

Given a natural-language utterance, we can ask three questions, each building on the previous one:

- **Syntax**: *what is grammatical?* The form and structure of the utterance: which sequences of words are well formed, and how words group into phrases and sentences.
- **Semantics**: *what does it mean?* The literal meaning of the words and of the way they are combined.
- **Pragmatics**: *what does it do?* The purpose of the utterance in its context: what the speaker wants to achieve by saying it.

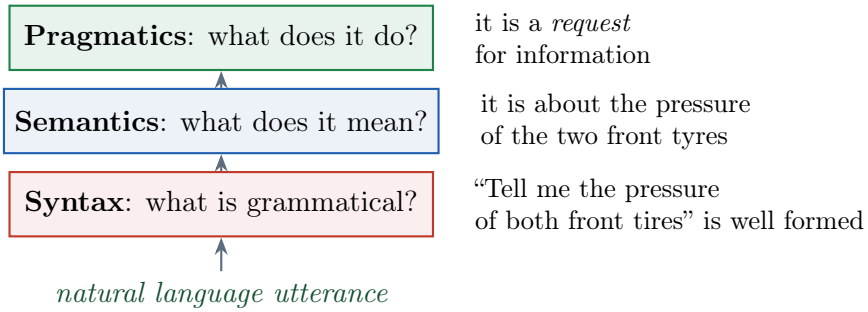



Figure 5: The three levels, from the bottom up, applied to a sentence from the ChatBDI case study (Section 2.5).

**Worked example — the same idea with everyday sentences**

- "Colorless green ideas sleep furiously" (Chomsky): syntactically correct, semantically nonsense.
- "Me want cookie": syntactically wrong, but the meaning is perfectly clear.
- "Can you pass the salt?": syntax and semantics describe a question about your *ability*; pragmatically it is a *request*, and answering "yes" without passing the salt misses the point.

## 3.2 Analogy with Programming Languages

[Slides · Terminology introduction: 25–26]

The three levels have a natural counterpart in programs:

| Level | A program is correct at this level if... |
|---|---|
| Syntax | there are no compiler errors |
| Semantics | there are no implementation bugs |
| Pragmatics | it implements the right algorithm (for the purpose) |

Three examples show that the levels are independent of each other:

- **Different syntax, same semantics** (both mean 5):

$$2+3 \quad \Longleftrightarrow \quad 3+2$$

- **Same syntax, different semantics** (1 and 1.5): the expression `3 / 2` is `1` in Python 2.7 (integer division between integers) but `1.5` in Python 3 (true division):

`3 / 2` (Python 2.7) $\quad \not\Longleftrightarrow \quad$ `3 / 2` (Python 3)

- **Good semantics, bad pragmatics**: a correct, bug-free implementation of a deep neural network for estimating the probability of a coin flip. It works, but it is absurd for the purpose: counting heads and dividing by the number of flips does the job.

**Intuition**

Syntax is about the *form*, semantics about the *content*, pragmatics about the *use*. Each level can be right while the one above is wrong: a sentence (or a program) can be well formed and meaningless, or meaningful and pointless.

## 3.3 A Map of the Course

[Slides · Terminology introduction: 27–33]

The topics of the course can be placed on the three levels. This list is a roadmap: each item will get its own section later in these notes.

### 3.3.1 Syntax

[Slides · Terminology introduction: 27–29]

The course starts with **regular expressions**, the basic tool for finding patterns in text (Section 5). Then the word level (Section 6) and the sentence level (Section 7):

| **Word level** | **What it is about** |
|---|---|
| Word segmentation | splitting a text into words (tokens) |
| Identification of stop words | finding very frequent words with little content (*the*, *of*, *and*) that are often filtered out |
| Stemming and lemmatisation | reducing *running*, *ran*, *runs* to a common form (a crude stem or the dictionary lemma *run*) |
| Minimum edit distance | how many edits turn one string into another (the Levenshtein distance used in Section 2.5) |
| **Sentence level** | |
| Sentence breaking | splitting a text into sentences (harder than it looks: think of "Dr. Smith" or "3.14") |
| Part-of-speech tagging | labelling each word as noun, verb, adjective, … |
| Parsing | building the grammatical structure (tree) of a sentence |

### 3.3.2 Semantics

[Slides · Terminology introduction: 30]

Covered in Sections 8 (distributional), 9 (frame and model-theoretic) and 10 (lexical semantics, WSD, NER).

- **Distributional semantics**: the meaning of a word is learnt from the contexts it appears in ("you shall know a word by the company it keeps").
- **Lexical semantics**: the meaning of words and the relations among them (synonyms, antonyms, hypernyms such as *animal* for *dog*).
- **Ontologies**: formal, structured representations of the concepts of a domain and their relations.
- **Word sense disambiguation**: choosing the right meaning of an ambiguous word in context (*bank* of a river or a bank that lends money).
- **Named entity recognition (NER)**: finding names of people, places, organisations, dates, ... in text.

### 3.3.3 Transformers and LLMs

[Slides · Terminology introduction: 31]

**BERT** and **GPT**, introduced in Section 2.3.

### 3.3.4 Features for representing a text

[Slides · Terminology introduction: 32]

Many NLP tasks (classification, sentiment analysis, authorship detection, ...) need to turn a text into numbers. The most common non-trivial features are:

- **Bag of Words**: a text is represented by the counts of its words, ignoring their order.
- **N-grams**, at word and character level: counts of sequences of $n$ consecutive words ("natural language", $n = 2$) or characters ("nat", $n = 3$), which keep some local order.
- **TF-IDF** (term frequency – inverse document frequency): word counts reweighted so that words frequent in *this* document but rare in the collection count more.
- **Flesch–Kincaid readability**: how hard a text is to read, from the average sentence length and the average number of syllables per word. The grade-level version is
$$\text{FKGL} = 0.39\,\frac{\text{words}}{\text{sentences}} + 11.8\,\frac{\text{syllables}}{\text{words}} - 15.59,$$
roughly the US school grade needed to understand the text.
- **LIWC** (Linguistic Inquiry and Word Count): counts of words in psychologically meaningful categories (emotions, pronouns, cognitive processes, ...), from a hand-built dictionary.

- **Emotion words**, based on models of basic emotions: **Ekman**'s six (anger, disgust, fear, happiness, sadness, surprise) or **Plutchik**'s eight (joy, trust, fear, surprise, sadness, disgust, anger, anticipation).
- ...

### 3.3.5 Pragmatics

[Slides · Terminology introduction: 33]

"**How to Do Things with Words**" is the title of the book (1962) by the philosopher J. L. Austin that founded the study of *speech acts*. Austin noticed that some sentences do not describe the world but *perform* an action: "I name this ship...", "I promise...", "Shoot her!". Saying them, in the right context, *is* doing the act. Compare them with a statement like "France is hexagonal", which describes the world and can be (roughly) true or false.

**Why this matters for NLP**

The *performatives* of agent communication languages in the ChatBDI case study (`tell`, `askAll`, `achieve`, ...; Section 2.5) come directly from speech-act theory: every message is classified by the act it performs.

**Cheat sheet — levels of linguistic analysis**

- The three levels (exam!): **syntax** (what is grammatical?), **semantics** (what does it mean?), **pragmatics** (what does it do?).
- Programming analogy: no compiler errors / no implementation bugs / the right algorithm. $2 + 3$ vs. $3 + 2$: different syntax, same semantics; `3/2` in Python 2.7 vs. 3: same syntax, different semantics; a neural network for a coin flip: good semantics, bad pragmatics.
- Syntax: regular expressions; word level (segmentation, stop words, stemming/lemmatisation, edit distance); sentence level (sentence breaking, POS tagging, parsing).
- Semantics: distributional and lexical semantics, ontologies, WSD, NER.
- Text features: BoW, n-grams, TF-IDF, Flesch–Kincaid, LIWC, emotion words.
- Pragmatics: Austin, *How to Do Things with Words*, speech acts.

# 4 Terminology

[Slides · Terminology introduction: 34–51; Semantics (long version): 44–50]

Every field has its vocabulary, and NLP borrows a lot of it from linguistics (morpheme, corpus, collocation) and from information retrieval (precision, recall). These terms are used throughout the course without being redefined, so it pays to fix them now.

This section covers:

1. the units words are made of: morphemes, words and compounds (Section 4.1);
2. a recap of the parts of speech (noun, verb, adjective, ...) with their Italian names and Penn tags (Section 4.2);
3. collections of texts and what we observe in them: corpora, genre, collocations (Section 4.3);
4. how to measure the quality of a system that retrieves documents: precision and recall (Section 4.4).

## 4.1 Morphemes, Words and Compounds

[Slides · Terminology introduction: 35–38, 42]

### 4.1.1 Morphemes

[Slides · Terminology introduction: 35–36]

**Definition — Morpheme**

A **morpheme** is the smallest grammatical unit in a language; in other words, the smallest *meaningful* unit of a language. The field of linguistics that studies morphemes is called **morphology**.

A morpheme is not the same as a word. The principal difference is that a morpheme may or may not stand alone, whereas a word, by definition, is freestanding. In *dogs* there is one word but two morphemes: *dog* (the animal) and *-s* ("more than one"). Neither *d* nor *og* carries meaning, so they are not morphemes.

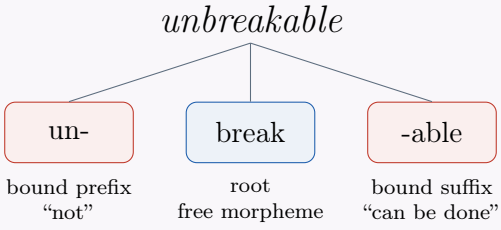
**Definition — Free and bound morphemes**

Every morpheme is either free or bound.

- A **free morpheme** can function independently as a word (e.g. *town*, *dog*).
- A **bound morpheme** appears only as part of a word, always together with a **root** and sometimes with other bound morphemes. For example, *un-* never appears alone. Most bound morphemes in English are **affixes**: **prefixes** (before the root, *un-*) and **suffixes** (after the root, *-ness*).

**Worked example — unbreakable**

*Unbreakable* is made of three morphemes:



**Watch out — *-able* is bound**

The slide calls *-able* "a free morpheme signifying *can be done*". As a suffix it is **bound**: it cannot stand alone with the meaning "that can be done" and needs a root (*break-able*, *read-able*). The English word *able* exists, but it is a separate free morpheme with a different use ("she is able to swim"). So *unbreakable* = bound prefix + free root + bound suffix.

### 4.1.2 Derivational and inflectional morphemes

[Slides · Terminology introduction: 37]

Bound morphemes are further divided according to *what they change*.

**Definition — Derivational and inflectional morphemes**

- A **derivational morpheme**, combined with a root, changes the **meaning** or the **part of speech** of the word: it creates a *new word*.
- An **inflectional morpheme** modifies a verb's tense, aspect, mood, person or number, or a noun's, pronoun's or adjective's number, gender or case, *without* changing the word's meaning or class (part of speech): it creates a different *form of the same word*.

| Root | Morpheme | Result | What changed |
|---|---|---|---|
| *Derivational* | | | |
| happy (adjective) | -ness | happiness (noun) | part of speech: adjective → noun |
| kind | un- | unkind | meaning is inverted |
| *Inflectional* | | | |
| dog | -s | dogs | number (singular → plural) |
| wait | -ed | waited | tense (present → past) |

Table 4: Derivation makes a new word, inflection a new form of the same word.

**Intuition — a dictionary test**

Ask: "would a dictionary give the result its own entry?" *Happiness* and *unkind* have their own entries (derivation), *dogs* and *waited* are found under *dog* and *wait* (inflection). This is exactly the difference between *lemmatisation* (going back to the dictionary form, removing inflection) and the analysis of word formation, which we will meet later in the course.

### 4.1.3 Words

[Slides · Terminology introduction: 38]

**Definition — Word**

A **word** is the smallest element that can be uttered in isolation with objective or practical meaning.

This contrasts with a morpheme, which is the smallest unit of meaning but does not necessarily stand on its own:

- a word may consist of a **single morpheme**: *oh!*, *rock*, *red*, *quick*, *run*, *expect*;
- or of **several**: *rock-s*, *red-ness*, *quick-ly*, *runn-ing*, *un-expect-ed*;
- and a morpheme may be unable to stand on its own as a word: in the examples, *-s*, *-ness*, *-ly*, *-ing*, *un-*, *-ed*.

### 4.1.4 Compounds

[Slides · Terminology introduction: 42]

**Definition — Compound**

A **compound** is a word composed of more than one *free* morpheme.

The last part is usually the **head** (it gives the category of the whole word), and the first part is the **modifier**. Compounds can be written as one word (*football*), with a hyphen (*blue-green*) or even as two words (*snow white*).

| Modifier | Head | Compound | Modifier | Head | Compound |
|---|---|---|---|---|---|
| noun | noun | football | noun | verb | browbeat |
| adjective | noun | blackboard | adjective | verb | highlight |
| verb | noun | breakwater | verb | verb | freeze-dry |
| preposition | noun | underworld | preposition | verb | undercut |
| noun | adjective | snow white | noun | preposition | love-in |
| adjective | adjective | blue-green | adverb | preposition | forthwith |
| verb | adjective | tumbledown | verb | preposition | takeout |
| preposition | adjective | over-ripe | preposition | preposition | without |

Table 5: English compounds by the category of their two parts.

**Watch out — compound vs. derived word**

*Blackboard* is a compound (*black* and *board* are both free). *Unkind* is *not* a compound: *un-* is bound, so it is a derived word. Also note that the meaning of a compound is often not the sum of its parts: a *blackboard* can be green.

## 4.2 Parts of Speech: a Grammar Recap

[Not in the slides: added material]

The course often talks about nouns, pronouns, adjectives, determiners… without defining them: they are assumed from school grammar. This subsection is a quick

recap to come back to whenever needed, with the link to the Italian grammar you already know and the Penn Treebank tags used in POS tagging (Section 7.2). Italian translations are always in *green*; most tables pair each English word with its Italian counterpart, one per row.

**Intuition — English and Italian names**

The English **parts of speech** (also *word classes* or *lexical categories*) are the Italian **parti del discorso**. Assigning one to each word of a sentence is what Italian school calls **analisi grammaticale**, and what NLP calls **POS tagging**. The sentence-level analysis (subject, object, ...) is the **analisi logica**, recalled at the end of this subsection.

### 4.2.1 Overview

[Not in the slides: added material]

| English | Italiano | Class | Penn tags | Examples |
|---|---|---|---|---|
| Noun | nome (sostantivo) | open | NN, NNS, NNP, NNPS | cat *gatto*, ideas *idee*, Rome *Roma* |
| Verb | verbo | open | VB, VBD, VBG, VBN, VBP, VBZ, MD | run *correre*, ran *corse*, is *è*, can *può* |
| Adjective | aggettivo (qualificativo) | open | JJ, JJR, JJS | red *rosso*, bigger *più grande*, best *il migliore* |
| Adverb | avverbio | open | RB, RBR, RBS, WRB | quickly *velocemente*, very *molto*, where *dove* |
| Pronoun | pronome | closed | PRP, PRP$, WP, WP$, EX | I *io*, me *me/mi*, mine *il mio*, who *chi* |
| Determiner | articolo, agg. determinativo | closed | DT, PDT, WDT | the *il/la*, a *un/una*, this *questo*, every *ogni* |
| Preposition | preposizione | closed | IN, TO | of *di*, to *a*, from *da*, in *in* |
| Conjunction | congiunzione | closed | CC, IN | and *e*, but *ma*, because *perché* |
| Interjection | interiezione | closed | UH | oh *oh*, wow *caspita*, ouch *ahi* |
| Numeral | aggettivo numerale | closed | CD | three *tre*, 1908 |
| Particle | (no equivalent) | closed | RP | give *up* *rinunciare*, turn *off* *spegnere* |

Table 6: Parts of speech at a glance. *Open* classes accept new words all the time (*to google* *googlare*, *selfie*); *closed* classes are small fixed lists (Section 7.3).

**Watch out — variable and invariable**

In Italian, five parts of speech are *variabili* (articolo, nome, aggettivo, pronome, verbo: they change with gender, number, person, tense) and four are *invariabili* (avverbio, preposizione, congiunzione, interiezione). English inflects much less: **articles and adjectives never change** (*the red car* *la macchina rossa*, *the red cars* *le macchine rosse*: in English only *car* changes), and nouns have number but **no grammatical gender**.

### 4.2.2 Nouns

[Not in the slides: added material]

A **noun** names a person, animal, thing, place or idea. **Italiano**: *nome* / *sostantivo*.

| Type | Italiano | Examples | Penn |
|---|---|---|---|
| common | nome comune | city *città*, dog *cane* | NN |
| proper | nome proprio | Genoa *Genova*, Mary *Maria* | NNP |
| plural | plurale | dogs *cani*, children *bambini*, the Alps *le Alpi* | NNS, NNPS |
| countable | numerabile | a chair *una sedia*, two chairs *due sedie* | NN, NNS |
| uncountable (mass) | non numerabile | water *acqua*, information *informazioni*, advice *consigli* | NN |
| concrete / abstract | concreto / astratto | table *tavolo* / happiness *felicità* | NN |
| compound | nome composto | football *calcio*, bus stop *fermata dell'autobus* | NN |
| possessive *'s* | complemento di specificazione | Mary's book *il libro di Maria* | NNP + POS |

**Differences with Italian**: no gender (*the table il tavolo*, *the chair la sedia*: the same *the*); uncountable nouns have no plural and no *a* even when Italian uses a plural (*some information delle informazioni*, *a piece of advice un consiglio*).

### 4.2.3 Pronouns

[Not in the slides: added material]

A **pronoun** stands for a noun (or a whole noun phrase). **Italiano**: *pronome*.

**Personal, possessive and reflexive pronouns.** One row per person:

| **Person** | **Subject**<br>*pers. soggetto* | **Object**<br>*pers. complemento* | **Possessive adj.**<br>*agg. possessivo* | **Possessive pron.**<br>*pron. possessivo* | **Reflexive**<br>*riflessivo* |
|---|---|---|---|---|---|
| 1s | I *io* | me *me, mi* | my *mio* | mine *il mio* | myself *mi, me stesso* |
| 2s | you *tu* | you *te, ti* | your *tuo* | yours *il tuo* | yourself *ti, te stesso* |
| 3s m. | he *lui* | him *lui, lo, gli* | his *suo* | his *il suo* | himself *si, sé stesso* |
| 3s f. | she *lei* | her *lei, la, le* | her *suo* | hers *il suo* | herself *si, sé stessa* |
| 3s thing | it *esso, –* | it *lo, la* | its *suo* | – | itself *si* |
| 1p | we *noi* | us *noi, ci* | our *nostro* | ours *il nostro* | ourselves *ci, noi stessi* |
| 2p | you *voi* | you *voi, vi* | your *vostro* | yours *il vostro* | yourselves *vi, voi stessi* |
| 3p | they *loro* | them *loro, li, le* | their *loro* | theirs *il loro* | themselves *si, sé stessi* |
| **Penn tag** | PRP | PRP | PRP$ | PRP | PRP |

**The other pronouns.**

| Type | English | Italiano | Penn |
|---|---|---|---|
| Demonstrative | this / these | *questo, questa / questi, queste* | DT |
| | that / those | *quello, quella / quelli, quelle* | DT |
| Relative | who (people) | *che, il quale* *the man who called l'uomo che ha chiamato* | WP |
| | which (things) | *che, il quale* *the book which I read il libro che ho letto* | WDT |
| | that (people, things) | *che* | WDT |
| | whom | *cui* *the girl to whom I wrote la ragazza a cui ho scritto* | WP |
| | whose | *il cui* *the man whose car... l'uomo la cui macchina...* | WP$ |
| Interrogative | who? | *chi?* | WP |
| | what? | *che cosa?* | WP |
| | which? | *quale?* | WDT |
| | whose? | *di chi?* | WP$ |
| Indefinite | someone / somebody | *qualcuno* | NN |
| | something | *qualcosa* | NN |
| | nobody / no one | *nessuno* | NN |
| | nothing | *niente, nulla* | NN |
| | everybody / everyone | *tutti, ognuno* | NN |
| | everything | *tutto* | NN |
| | anybody / anything | *chiunque / qualsiasi cosa* (in questions: *qualcuno / qualcosa*) | NN |

**Differences with Italian**:

- Italian is *pro-drop*: the subject pronoun can be omitted (*mangio*). English always needs it (*I eat*), even a dummy one: *it rains piove*, *there is a cat c'è un gatto* (*there* here is "existential", tag EX).
- The possessive adjective has *no article* in English: *my book il mio libro*; and it agrees with the *owner*: *her brother suo fratello* (of a woman), *his sister sua sorella* (of a man).
- Italian unstressed object pronouns (*mi, ti, lo, la, ci, vi, li, le, gli*) go before the verb, English ones after it: *lo vedo I see him.*

### 4.2.4 Verbs

[Not in the slides: added material]

A **verb** expresses an action, an event or a state. **Italiano**: *verbo.*

**The forms of an English verb.** English has only a few verb forms, and they are exactly what the Penn tags list:

| Tag | Form | Example | Italian equivalent |
|---|---|---|---|
| VB | base form | *to eat mangiare*, *I can eat posso mangiare* | infinito |
| VBP | present, not 3rd sing. | *I/we/they eat mangio / mangiamo / mangiano* | presente indicativo |
| VBZ | present, 3rd sing. | *she eats (lei) mangia* | presente, 3ª sing. |
| VBD | past tense | *she ate mangiò / mangiava / ha mangiato* | passato remoto / imperfetto / pass. prossimo |
| VBN | past participle | *she has eaten ha mangiato* | participio passato |
| VBG | gerund / present participle | *she is eating sta mangiando*, *eating is fun mangiare è divertente* | gerundio, infinito sostantivato |

**English tenses and Italian tempi.** Everything beyond those forms is built with auxiliaries (*be, have, do*) and modals:

| English tense | Example | Italiano | Tempo / modo |
|---|---|---|---|
| present simple | I eat | *mangio* | presente |
| present continuous | I am eating | *sto mangiando / mangio* | presente (stare + gerundio) |
| past simple | I ate | *mangiai / ho mangiato* | passato remoto / prossimo |
| past continuous | I was eating | *mangiavo / stavo mangiando* | imperfetto |
| present perfect | I have eaten | *ho mangiato* | passato prossimo |
| past perfect | I had eaten | *avevo mangiato* | trapassato prossimo |
| future | I will eat | *mangerò* | futuro semplice |
| future perfect | I will have eaten | *avrò mangiato* | futuro anteriore |
| conditional | I would eat | *mangerei* | condizionale presente |
| past conditional | I would have eaten | *avrei mangiato* | condizionale passato |
| imperative | eat! | *mangia!* | imperativo |
| passive | it is / was eaten | *è / fu mangiato* | forma passiva |

The Italian *congiuntivo* has (almost) no English form: *I think he is right penso che abbia ragione*, *if I were rich se fossi ricco*.

**Auxiliaries and modals.**

| English | Italiano | Example | Penn |
|---|---|---|---|
| be | *essere* (also *stare*) | *she is tired è stanca*, *she is eating sta mangiando* | VB… |
| have | *avere* | *she has eaten ha mangiato* | VB… |
| do | – (no equivalent) | *do you know? sai?*, *I don't know non so* | VB… |
| can | *potere, sapere* | *I can swim so nuotare*, *can I go? posso andare?* | MD |
| must | *dovere* | *you must go devi andare* | MD |
| should | *dovrei* (condizionale) | *you should rest dovresti riposare* | MD |
| may / might | *potrei, può darsi che* | *it might rain potrebbe piovere* | MD |
| will | (futuro) | *she will come verrà* | MD |
| would | (condizionale) | *she would come verrebbe* | MD |

Modals never change form (*she can*, not *she cans*) and are followed by the base form (VB). They correspond to the Italian *verbi servili* (*potere, dovere, volere*) or to a tense of the main verb.

**Other verb distinctions.**

- **Transitive** *transitivo*: *she reads a book lei legge un libro*; **intransitive** *intransitivo*: *she sleeps lei dorme*.
- **Phrasal verbs**: verb + particle (RP) with a new meaning: *give up rinunciare*, *look after badare a*, *turn off spegnere*, *find out scoprire*.

### 4.2.5 Adjectives

[Not in the slides: added material]

An **adjective** describes or qualifies a noun. **Italiano**: *aggettivo qualificativo*.

| **Degree** *(grado)* | **Short adjective** | **Long adjective** | **Penn** |
|---|---|---|---|
| positive *(positivo)* | big *grande* | expensive *costoso* | JJ |
| comparative of majority *(comp. di maggioranza)* | bigger than *più grande di* | more expensive than *più costoso di* | JJR / RBR + JJ |
| comparative of minority *(comp. di minoranza)* | less big than *meno grande di* | less expensive than *meno costoso di* | RBR + JJ |
| comparative of equality *(comp. di uguaglianza)* | as big as *grande come* | as expensive as *costoso come* | RB + JJ |
| relative superlative *(superlativo relativo)* | the biggest *il più grande* | the most expensive *il più costoso* | JJS / RBS + JJ |
| absolute superlative *(superlativo assoluto)* | very big *grandissimo* | very expensive *costosissimo* | RB + JJ |

Irregular: *good, better, best buono, migliore, il migliore*; *bad, worse, worst cattivo, peggiore, il peggiore*; *far, further, furthest lontano, più lontano, il più lontano*.

Position: **attributive** *a red car una macchina rossa* vs. **predicative** *the car is red la macchina è rossa* (*funzione attributiva / predicativa*).

**Differences with Italian**: English adjectives are invariable and come *before* the noun: *a red car, two red cars una macchina rossa, due macchine rosse*. What Italian calls *aggettivi determinativi* (possessivi, dimostrativi, indefiniti, interrogativi, numerali) are **not** adjectives in English grammar: *my mio* is PRP$, *this questo* and *every ogni* are DT, *three tre* is CD.

### 4.2.6 Adverbs

[Not in the slides: added material]

An **adverb** modifies a verb, an adjective, another adverb or a whole sentence. **Italiano**: *avverbio*. Manner adverbs are formed with *-ly*, the Italian *-mente*: *quick-ly veloce-mente*.

| **Type** | **English → Italiano** | **Penn** |
|---|---|---|
| manner *(di modo)* | quickly *velocemente*, slowly *lentamente*, well *bene*, badly *male* | RB |
| time *(di tempo)* | now *ora*, today *oggi*, yesterday *ieri*, tomorrow *domani*, soon *presto*, already *già*, still *ancora* | RB, NN |
| place *(di luogo)* | here *qui*, there *lì*, everywhere *ovunque*, inside *dentro*, outside *fuori* | RB |
| frequency *(di frequenza)* | always *sempre*, often *spesso*, sometimes *a volte*, never *mai* | RB |
| quantity *(di quantità)* | very *molto*, too *troppo*, quite *abbastanza*, a little *un po'*, almost *quasi* | RB |
| affirmation / negation | yes *sì*, not *non*, also *anche*, certainly *certamente* | RB, UH |
| interrogative *(interrogativo)* | when? *quando?*, where? *dove?*, why? *perché?*, how? *come?*, how much? *quanto?* | WRB |

Comparison works as for adjectives: *fast velocemente*, *faster più velocemente* (RBR), *fastest il più velocemente* (RBS).

### 4.2.7 Determiners and articles

[Not in the slides: added material]

A **determiner** comes before a noun and specifies *which* or *how many*. **Italiano**: *articolo* and most *aggettivi determinativi*.

| Type | English | Italiano | Penn |
|---|---|---|---|
| definite article *(determinativo)* | the | *il, lo, la, l', i, gli, le*: *the boy, the girl, the books* *il ragazzo, la ragazza, i libri* | DT |
| indefinite article *(indeterminativo)* | a / an | *un, uno, una, un'*: *a dog, an apple, an hour* *un cane, una mela, un'ora* | DT |
| partitive *(partitivo)* | some / – | *del, dello, della, dei, degli, delle*: *some bread* *del pane* | DT |
| demonstrative *(dimostrativo)* | this / these | *questo, questa / questi, queste* | DT |
| | that / those | *quel, quello, quella / quei, quegli, quelle* | DT |
| quantifier *(indefinito)* | every / each | *ogni / ciascuno*: *every day* *ogni giorno* | DT |
| | some / any | *qualche, alcuni / qualsiasi* | DT |
| | no | *nessuno*: *no money* *nessun soldo* | DT |
| | all / both | *tutti / entrambi* | DT, PDT |
| | many / much | *molti / molto*: *many books, much time* *molti libri, molto tempo* | JJ |
| | few / little | *pochi / poco* | JJ |
| interrogative *(interrogativo)* | which? / what? | *quale? / che?*: *which book?* *quale libro?* | WDT |
| | whose? | *di chi?*: *whose car?* *di chi è la macchina?* | WP$ |

The English articles never change: one *the* for all seven Italian forms, and the choice between *a* and *an* depends only on the *sound* that follows (*an hour*, *a university*). Predeterminers (PDT) come before another determiner: *all the cats* *tutti i gatti*, *both the girls* *entrambe le ragazze*.

### 4.2.8 Prepositions

[Not in the slides: added material]

A **preposition** links a noun phrase to the rest of the sentence, expressing place, time, means, ... **Italiano**: *preposizione*.

**The Italian preposizioni semplici and their English translations.** There is no one-to-one mapping: each Italian preposition has several English translations, depending on the meaning.

| Italiano | English | Examples |
|---|---|---|
| *di* | of | *a glass of wine un bicchiere di vino* |
| | from | *I am from Genoa sono di Genova* |
| | about | *let's talk about it parliamone (parliamo di questo)* |
| | than | *taller than me più alto di me* |
| | 's | *Mary's book il libro di Maria* |
| *a* | to | *I go to Rome vado a Roma*, *I give it to Bob lo do a Bob* |
| | at | *at home a casa*, *at 5 alle 5* |
| | in | *I live in Genoa abito a Genova* |
| *da* | from | *from Rome to Milan da Roma a Milano* |
| | by | *written by Dante scritto da Dante* |
| | for / since | *for two years da due anni*, *since 2020 dal 2020* |
| | at / to someone's | *at Mario's da Mario*, *to the doctor dal dottore* |
| *in* | in / into | *in Italy in Italia*, *into the room nella stanza* |
| | by | *by train in treno* |
| *con* | with | *with Bob con Bob*, *with a knife con un coltello* |
| *su* | on | *on the table sul tavolo* |
| | about | *a book about Rome un libro su Roma* |
| | out of | *one out of ten uno su dieci* |
| *per* | for | *for you per te*, *for two hours per due ore* |
| | through | *through the city per la città* |
| | to (purpose) | *I study to learn studio per imparare* |
| *tra / fra* | between | *between Rome and Milan tra Roma e Milano* |
| | among | *among friends tra amici* |
| | in (time) | *in two hours tra due ore* |

**Other common English prepositions.** *under sotto*, *over / above sopra*, *near vicino a*, *behind dietro*, *in front of davanti a*, *before prima di*, *after dopo*, *during durante*, *until fino a*, *without senza*, *against contro*, *towards verso*.

**Preposizioni articolate.** English has none: the preposition and the article stay two separate words (two tokens).

| | | | | |
|---|---|---|---|---|
| **Italiano** | *del* (di + il) | *al* (a + il) | *dal* (da + il) | *nel* (in + il) |
| **English** | of the / from the | to the / at the | from the / by the | in the |
| **Italiano** | *sul* (su + il) | *della, alla, dalla, nella, sulla* | *dei, ai, dai, nei, sui* | ... |
| **English** | on the | of/to/from/in/on the | of/to/from/in/on the | |

In Penn, all prepositions are IN except *to*, which always gets its own tag TO (also in *to run correre*).

### 4.2.9 Conjunctions

[Not in the slides: added material]

A **conjunction** connects words, phrases or clauses. **Italiano**: *congiunzione*.

| **Type** | **English** | **Italiano** | **Example** |
|---|---|---|---|
| coordinating | and | *e* | *bread and butter pane e burro* |
| *(coordinanti)*, | or | *o, oppure* | *tea or coffee? tè o caffè?* |
| CC | but | *ma, però* | *tired but happy stanco ma felice* |
| | nor | *né* | *neither Bob nor Mary né Bob né Maria* |
| | so | *quindi, perciò* | *it rained, so I stayed pioveva, quindi sono rimasto* |
| subordinating | because | *perché, poiché* | *I stayed because it rained sono rimasto perché pioveva* |
| *(subordinanti)*, | although | *sebbene, anche se* | *although it was late sebbene fosse tardi* |
| IN | if | *se* | *if you want se vuoi* |
| | when | *quando* | *when I arrived quando sono arrivato* |
| | while | *mentre* | *while she slept mentre dormiva* |
| | that | *che* | *I know that you are right so che hai ragione* |
| | until | *finché* | *wait until I call aspetta finché non chiamo* |
| | so that | *affinché* | *so that you understand affinché tu capisca* |
| | as / since | *siccome* | *as it was late, ... siccome era tardi, ...* |

Penn uses IN for both prepositions and subordinating conjunctions: *before lunch prima di pranzo* and *before I left prima che partissi* get the same tag.

### 4.2.10 Interjections, numerals and particles

[Not in the slides: added material]

| Type | English → Italiano | Penn |
|---|---|---|
| interjection *(interiezione)* | oh *oh*, wow *caspita*, ouch *ahi*, hey *ehi*, yes *sì*, no *no*, hello *ciao* | UH |
| cardinal *(numerale cardinale)* | one *uno*, three *tre*, twenty *venti*, 1908, 3.14 *3,14* | CD |
| ordinal *(numerale ordinale)* | first *primo*, second *secondo*, third *terzo* | JJ |
| particle (phrasal verbs) | give up *rinunciare*, turn off *spegnere*, go away *andare via* | RP |

Particles look like prepositions but belong to the verb, changing its meaning. No Italian part of speech corresponds; the closest is an adverb (*andare via*, *tirare su*). Note the decimal separator: *3.14* in English, *3,14* in Italian, which matters for tokenisers (Section 7.1).

### 4.2.11 Worked example: analisi grammaticale in English

[Not in the slides: added material]

**Worked example — tagging a sentence**

*Wow, there are three very big dogs in your garden, but my sister did not see them because she was running.*
*Caspita, ci sono tre cani molto grandi nel tuo giardino, ma mia sorella non li ha visti perché stava correndo.*

| Word | it. | Tag | Parte del discorso | Word | it. | Tag | Parte del discorso |
|---|---|---|---|---|---|---|---|
| Wow | *caspita* | UH | interiezione | my | *mia* | PRP$ | agg. possessivo |
| there | *ci* | EX | (sogg. fittizio) | sister | *sorella* | NN | nome comune |
| are | *sono* | VBP | verbo (essere) | did | – | VBD | verbo ausiliare |
| three | *tre* | CD | agg. numerale | not | *non* | RB | avv. di negazione |
| very | *molto* | RB | avv. di quantità | see | *visti* | VB | verbo |
| big | *grandi* | JJ | agg. qualificativo | them | *li* | PRP | pron. pers. compl. |
| dogs | *cani* | NNS | nome comune, plur. | because | *perché* | IN | cong. subordinante |
| in | *nel* | IN | preposizione | she | – | PRP | pron. pers. sogg. |
| your | *tuo* | PRP$ | agg. possessivo | was | *stava* | VBD | verbo ausiliare |
| garden | *giardino* | NN | nome comune | running | *correndo* | VBG | verbo (gerundio) |
| but | *ma* | CC | cong. coordinante | | | | |

The "–" cells show two differences from Italian: the auxiliary *did* has no Italian counterpart (Italian negates with *non* alone), and the subject *she* is dropped in Italian (*stava correndo*). Note also *in your* → *nel tuo* (preposizione articolata). A statistical tagger (NLTK's default one) gets most of this right but tags *Wow* as NNP (a proper noun): a capitalised unknown word at the start of a sentence looks like a name, and interjections are rare in the newspaper text taggers are trained on.

### 4.2.12 From words to sentences: analisi logica

[Not in the slides: added material]

Parts of speech classify *words*. The **functions** words and phrases play in a sentence are the subject of *analisi logica*; in the parse trees of Section 7.3 they correspond to positions in the tree. Running example: *Cynthia sold the bike to Bob for $200 yesterday ieri Cynthia ha venduto la bici a Bob per 200 dollari*.

| **English** | **Italiano** | **In the parse tree / example** |
|---|---|---|
| Subject | soggetto | the $NP$ before the verb: *Cynthia Cynthia* |
| Predicate (verb) | predicato verbale | the verb of the $VP$: *sold ha venduto* |
| Predicate (*be* + noun/adj.) | predicato nominale | *she is happy lei è felice* (*happy* = nome del predicato) |
| Direct object | complemento oggetto | the $NP$ after the verb: *the bike la bici* |
| Indirect object | complemento di termine | *to Bob a Bob*; also without *to*: *gave Bob the bike diede la bici a Bob* |
| Adverbials | complementi indiretti (prezzo, tempo, luogo, . . . ) | $PP$s and adverbs: *for $200 per 200 dollari*, *yesterday ieri*, *in Genoa a Genova* |

**Why this matters for NLP**

These functions are what semantic role labelling turns into roles (Section 9.1): in the running example the subject is the SELLER, the direct object the GOODS, the indirect object the BUYER, the *for*-phrase (*complemento di prezzo*) the PRICE.

**Cheat sheet — parts of speech**

- Open classes: **noun** (NN. . . = nome), **verb** (VB. . . , MD = verbo), **adjective** (JJ. . . = agg. qualificativo), **adverb** (RB. . . = avverbio).
- Closed classes: **pronoun** (PRP, WP = pronome), **determiner** (DT = articolo, agg. determinativi), **preposition** (IN, TO = preposizione), **conjunction** (CC coordinating, IN subordinating = congiunzione), **interjection** (UH = interiezione); plus numerals (CD) and particles (RP).
- English vs. Italian: no gender, invariable articles and adjectives, adjectives before the noun, subject pronoun always required, no preposizioni articolate, prepositions do not map one-to-one, few verb forms (the rest via auxiliaries and modals), *do* in questions and negations.
- Analisi grammaticale = POS tagging; analisi logica = functions in the sentence (subject, predicate, objects, adverbials).

## 4.3 Corpora, Genre and Collocations

[Slides · Terminology introduction: 39–41]

### 4.3.1 Corpus

[Slides · Terminology introduction: 39]

**Definition — Corpus (plural: corpora)**

A **text corpus** is a large and structured set of texts (nowadays usually stored and processed electronically). Corpora are used for statistical analysis and hypothesis testing, for checking occurrences, and for validating linguistic rules within a specific language territory.

Example: the corpora by Mark Davies, originally at `https://corpus.byu.edu/` and now at `https://www.english-corpora.org/`: COCA (Corpus of Contemporary American English), NOW (News on the Web), and others. You can search them to see how often and in which contexts a word or phrase is used.

**Why this matters for NLP**

Corpora are the raw material of statistical and neural NLP (Section 2.3): probabilities, word embeddings and LLMs are all learnt from corpora. What the corpus contains (its size, languages, genres, time period) determines what the model learns.

### 4.3.2 Genre

[Slides · Terminology introduction: 40]

**Definition — Genre**

- **Genre**: a style or category of art, music or literature.
- **Genre of a corpus**: usually defined on the basis of *text-external* features, such as medium, function and format (e.g. newspaper articles, academic papers, fiction, spoken conversation transcripts, social media posts).

"Text-external" means that the genre is decided by *where the text comes from and what it is for*, not by counting words inside it. Texts of different genres differ a lot (sentence length, vocabulary, formality), so a model trained on newspapers often performs worse on tweets.

### 4.3.3 Collocations

[Slides · Terminology introduction: 41]

**Definition — Collocation**

A **collocation** is a sequence of words or terms that co-occur *more often than would be expected by chance*. Examples: "pay attention", "fast food", "make an effort", "powerful engine".

*Fast* and *quick* mean roughly the same thing, yet native speakers combine them with different nouns (example from the NOW corpus):

| Natural English | Unnatural English |
|---|---|
| the fast train | the ~~quick~~ train |
| fast food | ~~quick~~ food |
| a quick shower | a ~~fast~~ shower |
| a quick meal | a ~~fast~~ meal |

**Intuition — "more often than by chance"**

If words were combined at random, *fast food* and *quick food* would appear about as often as the frequencies of *fast* and *quick* predict. In a corpus, *fast food* appears far more often than that, and *quick food* hardly ever. That excess is what makes it a collocation. Nothing in grammar or meaning forbids "quick food": collocations are a matter of *usage*, which is why they are learnt from corpora.

## 4.4 Precision and Recall

[Slides · Terminology introduction: 43–51; Semantics (long version): 44–50]

Suppose a program searches a collection of documents and returns those it thinks are relevant to a query. How good is it? Two questions matter, and they pull in opposite directions:

- of what it *returned*, how much is actually relevant? (**precision**)
- of what is actually relevant, how much did it *find*? (**recall**)

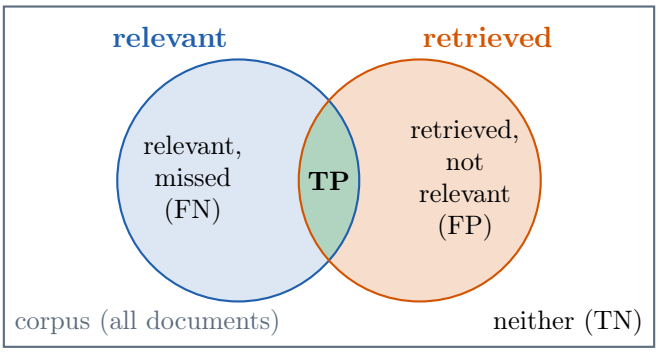



Figure 6: The green overlap (TP) holds the documents that are both relevant and retrieved. Precision looks at the orange circle (what fraction of it is green), recall looks at the blue circle (what fraction of it is green).

**Definition — Precision and recall**

$$\text{precision} = \frac{|\{\text{relevant documents}\} \cap \{\text{retrieved documents}\}|}{|\{\text{retrieved documents}\}|}$$

$$\text{recall} = \frac{|\{\text{relevant documents}\} \cap \{\text{retrieved documents}\}|}{|\{\text{relevant documents}\}|}$$

**Precision** (also called *positive predictive value*) is the fraction of retrieved doc-

uments that are relevant to the query. **Recall** (also called *sensitivity*) is the fraction of relevant documents that are successfully retrieved. Both are between 0 and 1; $|S|$ denotes the number of elements of the set $S$.

The numerator is the *same* in both formulas: the documents that are both relevant and retrieved. Only the denominator changes: precision divides by what the system *returned*, recall by what it *should have returned*.

### 4.4.1 The same thing with a confusion matrix

[Not in the slides: added material]

Each document falls in one of four cases:

| | **relevant** | **not relevant** |
|---|---|---|
| **retrieved** | true positive (TP) | false positive (FP) |
| **not retrieved** | false negative (FN) | true negative (TN) |

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}.$$

Neither formula uses TN: the (usually huge) number of irrelevant documents that were correctly left out does not count.

### 4.4.2 Four worked examples

[Slides · Terminology introduction: 44–51]

The setting is the same for all four examples. You implement an application that retrieves the documents containing one specific word ("cat") and its inflectional forms ("cats", see Section 4.1) in a corpus of **1000** documents. Exactly **24** documents contain "cat": these are the **relevant** documents.

**Worked example — 1: a typical system**

The application retrieves **36** documents; **16** of them contain "cat" and 20 do not. So $TP = 16$, $FP = 20$, $FN = 24 - 16 = 8$, $TN = 1000 - 16 - 20 - 8 = 956$.

$$\text{precision} = \frac{16}{36} = \frac{4}{9} \approx 0.444 \qquad \triangleleft\ TP/(TP + FP) = 16/(16 + 20)$$

$$\text{recall} = \frac{16}{24} = \frac{2}{3} \approx 0.667 \qquad \triangleleft\ TP/(TP + FN) = 16/(16 + 8)$$

Fewer than half of the results are right, and a third of the cat documents are missed.

**Worked example — 2: return everything**

The application retrieves **all 1000** documents; **24** of them contain "cat" and 976 do not. So $TP = 24$, $FP = 976$, $FN = 0$, $TN = 0$.

$$\text{precision} = \frac{24}{1000} = 0.024$$
$$\text{recall} = \frac{24}{24} = 1$$

Perfect recall, for free: a system that returns everything never misses anything. Its precision is terrible.

**Watch out — a typo in the slides**

Example 2 on the slides says the retrieved documents "out of which 24 contain the word cat, and 1986 do not contain it". Since only 1000 documents are retrieved, the correct number is $1000 - 24 = 976$. The precision and recall values on the slides are correct.

**Worked example — 3: play it safe**

The application retrieves only **2** documents, and both contain "cat". So $TP = 2$, $FP = 0$, $FN = 22$, $TN = 976$.

$$\text{precision} = \frac{2}{2} = 1$$
$$\text{recall} = \frac{2}{24} = \frac{1}{12} \approx 0.083$$

Perfect precision, almost for free: return only the one or two results you are completely sure about. Its recall is terrible.

**Worked example — 4: the perfect system**

The application retrieves **24** documents, all containing "cat". So $TP = 24$, $FP = 0$, $FN = 0$, $TN = 976$.

$$\text{precision} = \frac{24}{24} = 1, \qquad \text{recall} = \frac{24}{24} = 1.$$

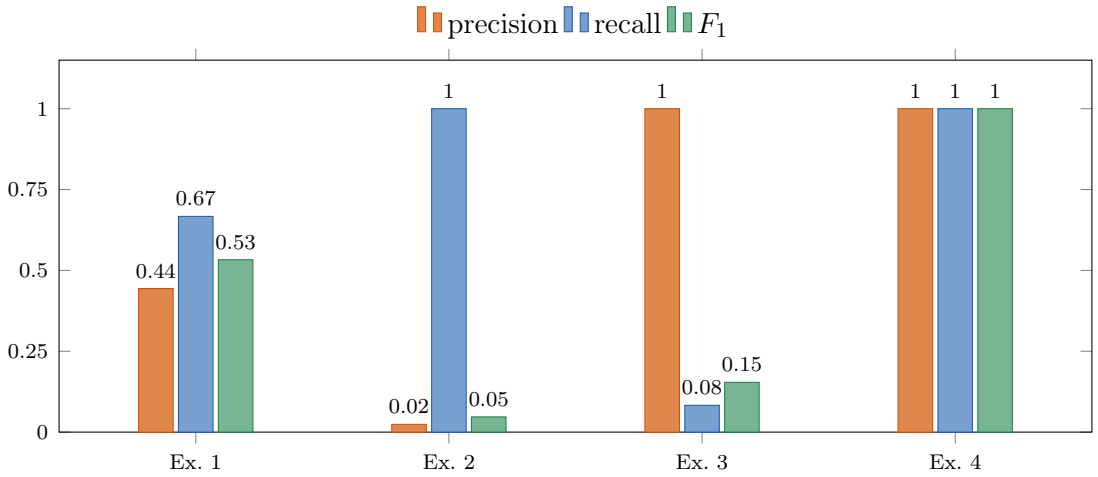



Figure 7: Precision, recall and $F_1$ for the four examples. Examples 2 and 3 each score perfectly on one measure with a trivial strategy; only $F_1$ exposes them.

**Intuition — the trade-off**

Precision and recall must always be read *together*. Returning more documents tends to raise recall and lower precision (Example 2 is the extreme); returning fewer tends to raise precision and lower recall (Example 3). A single number that is high only when *both* are high is the $F_1$ score below.

**Definition — $F_1$ score (not on the slides)**

The $F_1$ **score** is the harmonic mean of precision $P$ and recall $R$:

$$F_1 = \frac{2PR}{P+R} = \frac{2\,TP}{2\,TP + FP + FN}.$$

The harmonic mean is dominated by the smaller of the two values, so a system cannot get a good $F_1$ by maximising only one of them.

For the four examples:

$$\begin{aligned}
\text{Ex. 1:}\quad F_1 &= \frac{2 \cdot 16}{2 \cdot 16 + 20 + 8} = \frac{32}{60} \approx 0.533 \\
\text{Ex. 2:}\quad F_1 &= \frac{2 \cdot 24}{2 \cdot 24 + 976 + 0} = \frac{48}{1024} \approx 0.047 \\
\text{Ex. 3:}\quad F_1 &= \frac{2 \cdot 2}{2 \cdot 2 + 0 + 22} = \frac{4}{26} \approx 0.154 \\
\text{Ex. 4:}\quad F_1 &= \frac{2 \cdot 24}{2 \cdot 24 + 0 + 0} = 1
\end{aligned}$$

### 4.4.3 Another example: five algorithms

[Slides · Semantics (long version): 44–50]

The long version of the semantics slides repeats the definitions with a second example.

A query asks for the documents about topic $T$ in a corpus $C$. The correct answer is known: $D_1, D_2, D_4, D_5, D_6, D_7, D_8$ deal with $T$ (7 relevant documents) and no other document does.

| **Algorithm** | **Returns** | $\lvert\text{rel} \cap \text{ret}\rvert$ | **Precision** | **Recall** |
|---|---|---|---|---|
| A1 | $D_1, D_2, D_3, D_4$ | 3 | $3/4 = 75\%$ | $3/7 \approx 43\%$ |
| A2 | $D_1, \ldots, D_8$ | 7 | $7/8 = 87.5\%$ | $7/7 = 100\%$ |
| A3 | $D_1$ only | 1 | $1/1 = 100\%$ | $1/7 \approx 14\%$ |
| A4 | all and only the relevant | 7 | $7/7 = 100\%$ | $7/7 = 100\%$ |
| A5 | 1000 docs, incl. the 7 relevant | 7 | $7/1000 = 0.7\%$ | $7/7 = 100\%$ |

The same lessons as before: A3 buys perfect precision by returning almost nothing, A5 buys perfect recall by returning almost everything; only A4 is perfect on both. (On the slide for A2 the formulas are labelled "precision(A1)" and "recall(A1)": a copy-paste slip, the values are those of A2.)

### 4.4.4 Exercises

[Not in the slides: added material]

**Exercise 1 — morphemes**

Split each word into morphemes and classify each morpheme as free or bound, and each bound morpheme as derivational or inflectional: *unhappiness*, *walked*, *teachers*, *rewritten*. Which of *bookshelf*, *bookish*, *books* is a compound?

**Solution**

- *un-happi-ness*: *un-* bound, derivational (negates the meaning); *happy* free root; *-ness* bound, derivational (adjective → noun).
- *walk-ed*: *walk* free root; *-ed* bound, inflectional (past tense).
- *teach-er-s*: *teach* free root; *-er* bound, derivational (verb → noun, "one who teaches"); *-s* bound, inflectional (plural).
- *re-writ-ten*: *re-* bound, derivational ("again"); *write* free root; *-en* bound, inflectional (past participle).

Only *bookshelf* is a compound (*book* + *shelf*, both free). *Bookish* is derived (*-ish* is bound), *books* is inflected.

*Lesson*: derivational morphemes sit closer to the root than inflectional ones (*teach-er-s*, not *teach-s-er*).

**Exercise 2 — precision and recall**

A corpus has 500 documents, 40 of which are relevant to a query. A search engine returns 50 documents, 30 of which are relevant. Compute TP, FP, FN, TN, precision, recall and $F_1$. What happens to precision and recall if the engine also returns 50 more documents, none of them relevant?

**Solution**

$TP = 30$, $FP = 50-30 = 20$, $FN = 40-30 = 10$, $TN = 500-30-20-10 = 440$.

$$\text{precision} = \frac{30}{50} = 0.6, \qquad \text{recall} = \frac{30}{40} = 0.75,$$
$$F_1 = \frac{2 \cdot 30}{2 \cdot 30 + 20 + 10} = \frac{60}{90} \approx 0.667.$$

With 50 more irrelevant documents: $TP = 30$, $FP = 70$, $FN = 10$, so precision $= 30/100 = 0.3$ (halved) while recall stays $30/40 = 0.75$, and $F_1 = 60/(60 + 70 + 10) = 60/140 \approx 0.429$.
*Lesson*: adding wrong results never changes recall but always lowers precision.

**Cheat sheet — terminology**

- **Morpheme**: smallest meaningful unit; **free** (stands alone: *dog*) or **bound** (*un-*, *-ness*, *-s*).
- Bound morphemes: **derivational** (new word: meaning or part of speech changes, *happy* → *happiness*) or **inflectional** (new form of the same word: tense, number, …; *dog* → *dogs*).
- **Word**: smallest element uttered in isolation with meaning; one or more morphemes. **Compound**: word made of $\geq 2$ *free* morphemes.
- **Corpus**: large structured set of texts. **Genre of a corpus**: from text-external features (medium, function, format). **Collocation**: words co-occurring more often than by chance (*fast food*, not *quick food*).
- precision $= TP/(TP+FP)$: of what I returned, how much is right; recall $= TP/(TP+FN)$: of what is right, how much I returned; $F_1 = 2PR/(P+R)$. Returning everything gives recall 1; returning very few sure hits gives precision 1.

# 5 Regular Expressions

[Slides · Syntax: RE & word: 1–15]

Regular expressions are the first tool of the syntactic level (Section 3.3). They are the simplest way to describe a *set of strings* by a pattern, and they are everywhere in NLP: tokenisers, sentence splitters, data cleaning, rule-based chatbots, information extraction. Formally they describe the *regular languages*, the bottom level of the Chomsky hierarchy that we will meet in Section 7.3.

This section covers:

1. the syntax of regular expressions: disjunction, negation, repetition, anchors (Section 5.1);
2. a worked example that shows the two kinds of errors a pattern can make (Section 5.2);
3. ELIZA: a whole "conversation" built from cascades of substitutions (Section 5.3).

In these notes a pattern is written in colour, e.g. `[wW]oodchuck`, and the text it matches is underlined.

## 5.1 The Syntax of Regular Expressions

[Slides · Syntax: RE & word: 3–8, 10]

**Definition — Regular expression**

A **regular expression** (RE, regex) is a formal language for specifying text strings. A regex is a *pattern*; searching a text with it returns the substrings that the pattern *matches*.

The motivating question: how can we search for any of *woodchuck*, *woodchucks*, *Woodchuck*, *Woodchucks* with a single query? A plain string search for "woodchuck" misses the capitalised forms; we need a way to say "either `w` or `W`" and "an optional `s`".

### 5.1.1 Disjunction: square brackets and the pipe

[Slides · Syntax: RE & word: 4, 6]

Letters inside square brackets mean **one character among these**:

| Pattern | Matches | Example |
|---|---|---|
| `[wW]oodchuck` | *Woodchuck* or *woodchuck* | |
| `[1234567890]` | any digit | |
| `[A-Z]` | an upper-case letter (a **range**) | Drenched Blossoms |
| `[a-z]` | a lower-case letter | my beans were impatient |
| `[0-9]` | a single digit | Chapter 1: Down the Rabbit Hole |

Brackets choose among single *characters*. To choose among longer strings use the **pipe** `|`:

| Pattern | Matches |
|---|---|
| `groundhog\|woodchuck` | *groundhog* or *woodchuck* (they are the same animal!) |
| `yours\|mine` | *yours* or *mine* |
| `a\|b\|c` | the same as `[abc]` |
| `[gG]roundhog\|[Ww]oodchuck` | both animals, capitalised or not |

### 5.1.2 Negation inside brackets

[Slides · Syntax: RE & word: 5]

A caret `^` right after the opening bracket negates the set: `[^Ss]` means "any character that is *neither* `S` nor `s`".

| Pattern | Matches | Example |
|---|---|---|
| `[^A-Z]` | not an upper-case letter | Oyfn pripetchik |
| `[^Ss]` | neither `S` nor `s` | I have no exquisite reason |
| `[^e^]` | neither `e` nor `^` | Look here |
| `a^b` | the literal string `a^b` | Look up a^b now |

**Watch out — the caret has three meanings**

`^` means *negation* only when it is the *first* character inside `[]`. In `[e^]` it is an ordinary character, in `a^b` it is an ordinary character, and at the start of a pattern (`^[A-Z]`) it is an *anchor* (see below).

### 5.1.3 Repetition: `?`, `*`, `+` and the wildcard `.`

[Slides · Syntax: RE & word: 7]

These operators apply to the character (or group) right before them:

| Pattern | Meaning | Matches |
|---|---|---|
| `colou?r` | previous character optional (0 or 1) | *color*, *colour* |
| `oo*h!` | 0 or more of the previous character | *oh!*, *ooh!*, *oooh!*, ... |
| `o+h!` | 1 or more of the previous character | *oh!*, *ooh!*, *oooh!*, ... |
| `baa+` | `ba` followed by one or more `a` | *baa*, *baaa*, *baaaa*, ... |
| `beg.n` | `.` is any single character | *begin*, *begun*, *beg3n* |

The star `*` and the plus `+` are called the **Kleene star** and **Kleene plus**, after the mathematician Stephen C. Kleene who introduced regular expressions. Note that `oo*h!` and `o+h!` describe exactly the same set of strings: $x^+$ is just $xx^*$.

### 5.1.4 Anchors: `^` and `$`

[Slides · Syntax: RE & word: 8]

Anchors do not match a character but a *position*: `^` the start of the line, `$` the end.

| Pattern | Matches | Example |
|---|---|---|
| `^[A-Z]` | an upper-case letter at the start of a line | Palo Alto |
| `^[^A-Za-z]` | a non-letter at the start of a line | 1 "Hello" |
| `\.$` | a full stop at the end of a line | The end. |
| `.$` | any character at the end of a line | The end? The end! |

In `\.$` the backslash **escapes** the dot: `\.` is a literal full stop, while `.` alone is the wildcard.

### 5.1.5 Summary of the operators

[Slides · Syntax: RE & word: 10]

| Operator | Meaning | Example | Example meaning |
|---|---|---|---|
| `+` | one or more | `a+` | 1 or more `a` characters |
| `*` | zero or more | `a*` | 0 or more `a` characters |
| `?` | optional | `a?` | 0 or 1 `a` characters |
| `[]` | choose 1 | `[abc]` | `a` or `b` or `c` |
| `[-]` | range | `[a-z]` | any character between `a` and `z` |
| `[^]` | not | `[^a]` | any character that is not `a` |
| `()` | grouping | `([a-z])+` | one or more characters between `a` and `z` |
| `(\|)` | or | `(ey\|ax)` | the string `ey` or the string `ax` |
| `ab` | follow | `ab` | `a` followed by `b` |
| `^` | start | `^a` | `a` at the start of the string/line |
| `$` | end | `a$` | `a` at the end of the string/line |
| `\s` | whitespace | `\sa` | a whitespace followed by `a` |
| `.` | any character | `a.b` | `a`, any character, `b` |

**Watch out — a typo in the summary slide**

The slide gives `(a-z)+` as the grouping example. Inside round brackets the hyphen is *not* a range: `(a-z)+` matches the literal three-character string `a-z` (repeated). The range needs square brackets: `([a-z])+` or simply `[a-z]+`.

Tools to experiment: `https://regex101.com/` shows step by step what a pattern matches and explains each part of it.

## 5.2 Worked Example: Finding "the"

[Slides · Syntax: RE & word: 9]

**Task**: find all instances of the word "the" in a text. Writing the right pattern takes a few attempts, and each attempt fixes one kind of error. We test them on

*The other day the theology student said: the end.*

**Worked example — three attempts**

1. `the`: matches *the* three times, but also the *the* inside *other* and *theology*, and it **misses** the capitalised *The* at the start.
2. `[tT]he`: now *The* is found, but it still **incorrectly returns** the *the* inside *other* and *theology*.
3. `[^a-zA-Z][tT]he[^a-zA-Z]`: "the" must be surrounded by non-letters, so *other* and *theology* are excluded.

Two kinds of errors appeared, and they are exactly the two denominators of precision and recall (Section 4.4):

- **false positives**: matching strings we should not have matched (*other*, *theology*). Reducing them increases **precision**;
- **false negatives**: not matching strings we should have matched (*The*). Reducing them increases **recall**.

Fixing one kind of error often introduces the other, so building a pattern is a continuous trade-off between the two.

**Watch out — attempt 3 is still not perfect**

`[^a-zA-Z][tT]he[^a-zA-Z]` requires a character *before* and *after* the word, so it misses "The" at the very beginning of a line (there is no character before it) and a "the" at the very end. It also includes the surrounding spaces in the match. Two common fixes:

- allow the start/end of the line as alternatives: `(^|[^a-zA-Z])[tT]he([^a-zA-Z]|$)`;
- use the *word boundary* anchor `\b` (available in Python and most regex engines): `\b[tT]he\b` matches *The*, *the*, *the* and nothing else in the test sentence.

## 5.3 ELIZA: Conversation by Substitution

[Slides · Syntax: RE & word: 11–15]

ELIZA (Section 2.3) is the classic proof of how far regular expressions alone can go. It understands nothing: it applies a **cascade** of regex substitutions to the user's sentence and returns the result. A conversation with an online version (https://www.masswerk.at/elizabot/) looks like this:

```
ELIZA: How do you do.  Please tell me your problem.
YOU:   I had an unpleasant discussion with a friend of mine
ELIZA: Can you elaborate on that ?
YOU:   She told me that I am stupid
ELIZA: Is it because you are stupid that you came to me ?
YOU:   yes, it is exactly because I am stupid...
ELIZA: I see.
```

*"Is it because you are stupid that you came to me?"* is just the user's *"I am stupid"* with *I am* rewritten to *you are* and pasted into a template.

### 5.3.1 A tiny ELIZA in Python

[Slides · Syntax: RE & word: 14–15]

Three substitutions applied in cascade with Python's `re.sub`, which takes a pattern, a replacement and a string:

```
import re
str = "I feel sad, my heart is broken"
str = re.sub("I", "you", str)        # you feel sad, my heart is broken
str = re.sub("my", "your", str)      # you feel sad, your heart is broken
str = re.sub(r"you (feel|are) (sad|sorry|desperate)(.*)",
             r"Why do you say that you \1 \2\3?", str)
print(str)   # Why do you say that you feel sad, your heart is broken?
```

- Steps 1 and 2 swap the point of view (*I* → *you*, *my* → *your*).
- Step 3 uses **capture groups**: each pair of round brackets in the pattern remembers what it matched, and `\1`, `\2`, `\3` in the replacement paste it back. Here `\1` = *feel*, `\2` = *sad*, `\3` = *, your heart is broken.*
- The `r"..."` prefix makes a *raw string*, so that Python does not interpret the backslashes itself.

**Watch out — order and context matter**

`re.sub("I", "you", str)` replaces *every* capital `I`, also inside words ("*It*" becomes "*yout*"), and the order of the cascade matters: if we swapped *you* → *I* as well, a later rule could undo an earlier one. Real ELIZA implementations use word boundaries (`\bI\b`) and swap pronouns in a single pass.

### 5.3.2 Exercises

[Not in the slides: added material]

**Exercise 1 — prices**

Write a regex matching prices in dollars such as *$5* and *$19.99* (an optional decimal part with exactly two digits). What does it return on "not $3.5"?

**Solution**

`\$[0-9]+(\.[0-9][0-9])?`: an escaped dollar sign (`$` alone is the end anchor), one or more digits, then an optional group made of a literal dot and two digits. On "$3.5" the optional group cannot match (only one digit after the dot), so the regex returns just *$3*: a false positive of a wrong length. *Lesson*: a pattern can match only part of what you intended; add `\b` or a negative condition after the price if partial matches matter.

**Exercise 2 — reading patterns**

Describe in words what each pattern matches: (a) `^[A-Z][a-z]*\.$`; (b) `[0-9]+(st|nd|rd|th)`; (c) `\bcolou?rs?\b`.

**Solution**

(a) A whole line made of one capitalised word followed by a full stop ("Stop." but not "Stop now."). (b) An ordinal written with digits: *1st*, *22nd*, *3rd*, *100th* (and also the wrong *1th*: a regex checks form, not meaning). (c) The whole words *color*, *colour*, *colors*, *colours*, but not *discolor*.

**Cheat sheet — regular expressions**

- `[abc]` one of; `[a-z]` range; `[^a]` not (caret first in brackets); `x|y` one of two strings; `()` grouping and capture.
- `?` 0 or 1; `*` 0 or more (Kleene star); `+` 1 or more (Kleene plus); `.` any

character; `\.` a literal dot.
- Anchors `^` (start), `$` (end), `\b` (word boundary).
- Writing a pattern trades off **false positives** (lower precision) and **false negatives** (lower recall).
- ELIZA = a cascade of regex substitutions with capture groups (`\1`, `\2`, ...).

# 6 Syntax at the Word Level

[Slides · Syntax: RE & word: 16–82]

Before a sentence can be tagged or parsed, the text must be cut into words, and the words must often be cleaned and normalised. This section covers the word-level tasks of the syntactic level (Section 3.3):

1. **word segmentation**: where does a word start and end? (Section 6.1);
2. **stop words** and **TF-IDF**: which words carry little information, and how to measure how important a word is for a document (Section 6.2);
3. **stemming and lemmatisation**: reducing *organizes*, *organizing* to a common form, with the Porter stemmer in detail (Section 6.3);
4. **minimum edit distance**: how similar are two strings, computed with dynamic programming (Section 6.4).

## 6.1 Word Segmentation

[Slides · Syntax: RE & word: 16–20]

**Definition — Word segmentation**

**Word segmentation** (also *tokenisation*) is the problem of dividing a string of written language into its component words.

### 6.1.1 Why it is not trivial

[Slides · Syntax: RE & word: 17]

In English and in many other languages written with the Latin alphabet, the **space** is a good approximation of a word divider (word delimiter). It is only an approximation, because languages vary in how they treat collocations and compounds (Section 4.1): many English compound nouns are written in several ways, e.g. *multi-agent* = *multi-agent* = *multi agent*, and speakers differ on whether they think of them as one noun or as a noun phrase. Punctuation adds more cases (*don't*, *U.S.A.*, *3.14*, *e-mail*).

Many scripts have no word space at all, and there word segmentation is a hard problem:

- **Chinese** and **Japanese**: sentences are delimited, words are not;
- **Thai** and **Lao**: phrases and sentences are delimited, words are not;

- **Vietnamese**: syllables are delimited, words are not.

### 6.1.2 The MaxMatch algorithm

[Slides · Syntax: RE & word: 18]

A simple baseline for Chinese is the greedy **maximum matching** algorithm. It needs a **dictionary** (word list) of the language.

**Definition — MaxMatch**

1. Put a pointer at the beginning of the string.
2. Choose the **longest** dictionary word that matches the input starting at the pointer, and move the pointer to the end of that word.
3. If no word matches, move the pointer one character forward instead (creating a one-character "word").
4. Repeat from the new pointer position until the end of the string.

**Worked example — MaxMatch on English without spaces**

Input: `wecanonlyseeashortdistanceahead` ("we can only see a short distance ahead").

| Remaining input | Longest word | Why |
|---|---|---|
| `wecanonly...` | *we* | |
| `canonlysee...` | *canon* | longer than *can* |
| `lyseea...` | *l* | no word starts here: one character |
| `yseea...` | *y* | same |
| `seeashort...` | *see* | |
| `ashort...` | *ash* | longer than *a* and *as* |
| `ort...` | *o*, *r*, *t* | three single characters |
| `distanceahead` | *distance*, *ahead* | |

Output: *we canon l y see ash o r t distance ahead.*

**Intuition — why greedy fails here and works for Chinese**

Greedy choices are never undone: once *canon* is taken, the correct *can only* is impossible. In English, words are long and many short words hide inside longer ones, so this happens constantly. Chinese words are mostly one or two characters long, so the longest match is usually right, and MaxMatch is a surprisingly strong baseline there. Better segmenters score *whole* segmentations (e.g. with word probabilities) instead of committing greedily.

### 6.1.3 NLTK

[Slides · Syntax: RE & word: 19–20]

The course uses **NLTK** (Natural Language Toolkit, `https://www.nltk.org/`) to show some problems and some of their possible solutions. NLTK is a Python platform

for working with human language data: it provides interfaces to over 50 corpora and lexical resources such as WordNet (Section 10.2), and libraries for classification, tokenisation, stemming, tagging, parsing and semantic reasoning. The course also uses less known, but elegant, tools.

```python
import nltk
# nltk.download()   # only once, to download corpora and models
from nltk.tokenize import word_tokenize
print(word_tokenize("Don't segment U.S.A. badly, Mr. Smith!"))
# ['Do', "n't", 'segment', 'U.S.A.', 'badly', ',', 'Mr.', 'Smith', '!']
```

Note how the tokeniser splits *don't* into *Do* + *n't* (two morphemes, Section 4.1) but keeps *U.S.A.* and *Mr.* together: the full stop is not always a word boundary.

## 6.2 Stop Words and TF-IDF

[Slides · Syntax: RE & word: 21–47]

### 6.2.1 Stop words

[Slides · Syntax: RE & word: 21–23]

**Definition — Stop words**

**Stop words** are words that are filtered out before or after processing natural language text. The term usually refers to the *most common* words of a language (*the*, *of*, *and*, *a*, *to*, ...), but there is no single universal list used by all NLP tools, and some tools do not use one at all.

- **How a stop list is built**: sort the terms by *collection frequency* (the total number of times each term appears in the document collection), take the most frequent ones, and often filter them by hand according to their meaning in the domain of the documents. The words of the stop list are then discarded during indexing. Example lists: `https://www.ranks.nl/stopwords`.
- **When not to remove them**: some tools avoid removing stop words to support *phrase search* (exact, not keyword-based): "to be or not to be" consists almost only of stop words.

A word cloud of a novel (the slides show one for a Thomas Hardy novel) is dominated by *the*, *and*, *of*, *to*, *a*, *was*, *her*: they are frequent in *any* English text and say nothing about this one.

### 6.2.2 Frequent in the document or frequent in the language?

[Slides · Syntax: RE & word: 24–29]

**Intuition — the key question**

Is a word that appears often in a document always a stop word? No. A word can be frequent in a document $d$ for two very different reasons:

- it is related to the **topic** of $d$ ("cat" in a document about cats);
- it is frequent in the **language** ("the" in every document).

To tell them apart, we must look not only at $d$ but at a whole **corpus**: a topical word is frequent in *some* documents, a stop word in *all* of them.

The running example uses two short documents:

**Worked example — the corpus**

**Doc1** (54 words): *The cat (Felis catus) is a small carnivorous mammal. It is the only domesticated species in the family Felidae and often referred to as the domestic cat to distinguish it from wild members of the family. The cat is either a house cat or a farm cat, which are pets, or a feral cat.*

**Doc2** (42 words): *A car (or automobile) is a wheeled motor vehicle used for transportation. Cars became widely available in the early 20th century. One of the first cars accessible to the masses was the 1908 Model T. Cars were rapidly adopted in the US.*

| | **"the"** | **"cat"** | **"car(s)"** |
|---|---|---|---|
| Doc1 | 6 | 6 | 0 |
| Doc2 | 5 | 0 | 4 |
| corpus $C_2 = \{\text{Doc1}, \text{Doc2}\}$ | 11 | 6 | 4 |

Corpus $C_1$ contains only Doc1; corpus $C_2$ contains Doc1 and Doc2.

**Watch out — a counting slip in the slides**

The slides count **4** occurrences of "the" in Doc2 (and 10 in $C_2$). Doc2 actually contains it **5** times: *the early*, *the first*, *the masses*, *the 1908*, *the US*. This changes tf("the", Doc2) from 4/42 to 5/42, but not the final TF-IDF value, which is 0 either way (see below).

In $C_1$ alone, "the" and "cat" are equally frequent (6 times each): with a single document there is no way to tell a topical word from a stop word. In $C_2$ the difference appears: "the" is in *both* documents, "cat" only in one.

### 6.2.3 Term frequency and inverse document frequency

[Slides · Syntax: RE & word: 30–34]

**Definition — TF-IDF**

**Term frequency**: the number of times term $t$ occurs in document $d$, usually normalised by the length of the document:

$$\mathrm{tf}(t, d) = \frac{\text{number of occurrences of } t \text{ in } d}{\text{length}(d)}.$$

**Inverse document frequency**: the logarithm of the inverse fraction of documents of the corpus $C$ that contain $t$:

$$\mathrm{idf}(t, C) = \log \frac{|C|}{|\{d \in C : t \in d\}|},$$

where $|C|$ is the number of documents in the corpus.
**TF-IDF**: the product of the two,

$$\text{tf-idf}(t, d, C) = \mathrm{tf}(t, d) \cdot \mathrm{idf}(t, C).$$

- **tf** is high when $t$ is frequent *in this document*.
- **idf** is high when $t$ is *rare in the corpus*. If $t$ occurs in every document, the fraction is $|C|/|C| = 1$ and idf $= \log 1 = 0$: the word cannot distinguish documents at all, exactly like a stop word.
- The **logarithm** dampens the effect: a word in 1 document out of 1000 is more informative than one in 10 out of 1000, but not 10 times more.
- If $t$ appears in *no* document the denominator is 0. A common fix is to add 1 to the denominator; it makes sense when the corpus is large enough, since it slightly distorts the values.

**Watch out — base of the logarithm**

The slides' numbers use the base-10 logarithm ($\log 2 \approx 0.301$). Other sources use the natural logarithm ($\ln 2 \approx 0.693$). The base only rescales all idf values by the same factor, so the *ranking* of words does not change; just be consistent.

### 6.2.4 Worked example on $C_2$

[Slides · Syntax: RE & word: 35–46]

**Worked example — TF-IDF of "the", "cat", "car"**

First the idf values ($|C_2| = 2$):

$$\begin{aligned}
\mathrm{idf}(\text{the}, C_2) &= \log \tfrac{2}{2} = \log 1 = 0 && \triangleleft \text{ "the" is in both documents} \\
\mathrm{idf}(\text{cat}, C_2) &= \log \tfrac{2}{1} = \log 2 \approx 0.301 && \triangleleft \text{ only in Doc1} \\
\mathrm{idf}(\text{car}, C_2) &= \log \tfrac{2}{1} = \log 2 \approx 0.301 && \triangleleft \text{ only in Doc2}
\end{aligned}$$

Then the products:

$$\begin{aligned}
&\text{tf-idf}(\text{the}, \text{Doc1}, C_2) = \tfrac{6}{54} \cdot 0 = 0 \\
&\text{tf-idf}(\text{the}, \text{Doc2}, C_2) = \tfrac{5}{42} \cdot 0 = 0 \\
&\text{tf-idf}(\text{cat}, \text{Doc1}, C_2) = \tfrac{6}{54} \cdot 0.301 \approx 0.111 \cdot 0.301 \approx 0.033 \\
&\text{tf-idf}(\text{cat}, \text{Doc2}, C_2) = 0 \cdot 0.301 = 0 \qquad \triangleleft\ \text{tf} = 0\text{, idf irrelevant} \\
&\text{tf-idf}(\text{car}, \text{Doc1}, C_2) = 0 \cdot 0.301 = 0 \\
&\text{tf-idf}(\text{car}, \text{Doc2}, C_2) = \tfrac{4}{42} \cdot 0.301 \approx 0.095 \cdot 0.301 \approx 0.029
\end{aligned}$$

| tf-idf | "the" | "cat" | "car" |
|---|---|---|---|
| Doc1 | 0 | **0.033** | 0 |
| Doc2 | 0 | 0 | **0.029** |

Table 7: TF-IDF in $C_2$. Each document's only non-zero word is its topic.

"The" is as frequent as "cat" in Doc1, but its TF-IDF is 0 because it appears everywhere; "cat" and "car" come out as the words that characterise their documents. This is how TF-IDF finds *keywords* and, symmetrically, *stop words* (terms with idf close to 0) automatically. Online analysers such as `https://www.online-utility.org/text/analyzer.jsp` compute word frequencies of a text you paste.

**Why this matters for NLP**

TF-IDF is one of the text features of Section 3.3: representing each document by the TF-IDF values of its words gives a vector that search engines use to rank documents for a query and classifiers use to decide the topic of a text.

## 6.3 Stemming and Lemmatisation

[Slides · Syntax: RE & word: 48–65]

### 6.3.1 The goal: one form for many

[Slides · Syntax: RE & word: 48]

Documents use different forms of a word: *organize*, *organizes*, *organizing* (inflectional morphemes, Section 4.1); and there are families of derivationally related words with similar meanings: *democratic*, *democratization* (derivational morphemes). For a search, it is often useful that a query for one of these words also returns documents containing another word of the same set.

The goal of both stemming and lemmatisation is to **reduce inflectional forms, and sometimes derivationally related forms, of a word to a common base**

**form**:

$$\begin{aligned} \text{am, are, is} &\rightarrow \text{be} \\ \text{car, cars, car's, cars'} &\rightarrow \text{car} \\ \text{the boy's cars are different colors} &\rightarrow \text{the boy car be differ color} \end{aligned}$$

### 6.3.2 Stemming vs. lemmatisation

[Slides · Syntax: RE & word: 49]

**Definition — Stemming and lemmatisation**

- **Stemming** is a *crude heuristic* process that chops off the ends of words, hoping to get it right most of the time. It often removes derivational affixes too. The result, the *stem*, need not be a real word.
- **Lemmatisation** does things properly, with a *vocabulary* and a *morphological analysis* of words. It normally removes inflectional endings only and returns the base or dictionary form of the word, the **lemma**.

Stemming most commonly collapses derivationally related words; lemmatisation commonly collapses only the different inflectional forms of a lemma.

| Word | Porter stem | Lemma |
|---|---|---|
| are | ar | be |
| theses | these | thesis |
| universal | univers | universal |
| university | univers | university |
| generalizations | gener | generalization |

Table 8: A stemmer chops, a lemmatiser looks words up. Stems are not words, and unrelated words can end up with the same stem.

### 6.3.3 The Porter stemmer

[Slides · Syntax: RE & word: 50–52]

The **Porter stemmer** (Martin Porter, 1980) is the classic stemming algorithm for English. It is a simple **rule-based** and **heuristic** method: a cascade of suffix-rewriting rules such as ATIONAL → ATE (*relational* → *relate*), applied in sets, in order. The slides present it in seven steps.

#### Consonants, vowels and the measure.

- A **consonant** is a letter other than `a`, `e`, `i`, `o`, `u`, and other than `y` when preceded by a consonant. A **vowel** is any other letter. (So the `y` of *toy* is a consonant, the `y` of *happy* is a vowel.)

- Write $C$ for a sequence of one or more consonants and $V$ for a sequence of one or more vowels. Then *every* word has the form

$$[C]\,(VC)^m\,[V]$$

where the square brackets mean "optional". The number $m$ is the **measure** of the word (or stem): roughly, its number of syllables.

**Worked example — measures**

| Word | Letters as C/V | Form | $m$ |
|---|---|---|---|
| tree | tr·ee | $C\,V$ | 0 |
| trouble | tr·ou·bl·e | $C\,(VC)\,V$ | 1 |
| troubles | tr·ou·bl·e·s | $C\,(VC)(VC)$ | 2 |
| oaten | oa·t·e·n | $(VC)(VC)$ | 2 |

The slides write *troubles* as "C V CVC": grouping it as $C\,(VC)(VC)$ gives $m = 2$.

**Rule format.** Each rule has the form (*condition*) $S_1 \to S_2$ : if the word ends with suffix $S_1$ and the *stem* (what is left after removing $S_1$) satisfies the condition, $S_1$ is replaced by $S_2$. The conditions are:

| Condition | Meaning |
|---|---|
| $m > k$, $m = k$ | the measure of the stem |
| `*S` | the stem ends with the letter `S` (similarly `*L`, `*Z`) |
| `*v*` | the stem contains a vowel |
| `*d` | the stem ends with a double consonant (e.g. `-tt`, `-ss`) |
| `*o` | the stem ends with *cvc*, where the second *c* is not `w`, `x`, `y` |

The measure condition is what makes the stemmer safe on short words: ($m$>0) NESS $\to \varepsilon$ turns *goodness* into *good*, but it would not touch a hypothetical word whose stem would be empty.

### 6.3.4 The seven steps

[Slides · Syntax: RE & word: 53–58]

In the rules, $\varepsilon$ is the empty string (the suffix is deleted).

| Step | Rules | Examples |
|---|---|---|
| 1 (plurals) | SSES→SS; IES→I; SS→SS; S→ $\varepsilon$ | caresses→caress, ponies→poni, ties→ti, caress→caress, cats→cat |
| 2a (*-ed*, *-ing*) | ($m$>0) EED→EE; (`*v*`) ED→ $\varepsilon$; (`*v*`) ING→ $\varepsilon$ | agreed→agree, feed→feed; plastered→plaster, bled→bled; motoring→motor, sing→sing |
| 2b (clean-up, only if the 2nd or 3rd rule of 2a fired) | AT→ATE; BL→BLE; (`*d` and not `*L`, `*S`, `*Z`) → single letter; ($m$=1 and `*o`) → E | conflat(ed)→conflate, troubl(ing)→trouble, hopp(ing)→hop, tann(ed)→tan, fall(ing)→fall, fil(ing)→file, fail→fail |
| 3 ($y$) | (`*v*`) Y→I | happy→happi, sky→sky |
| 4 (derivational I) | ($m$>0) ATIONAL→ATE; IZATION→IZE; BILITI→BLE; ... | relational→relate, generalization→generalize, sensibiliti→sensible |
| 5 (derivational II) | ($m$>0) ICATE→IC; FUL→ $\varepsilon$; NESS→ $\varepsilon$; ... | triplicate→triplic, hopeful→hope, goodness→good |
| 6 (derivational III) | ($m$>1) ANCE, ENT, IVE, ER, AL, ANT, ... → $\varepsilon$ | allowance→allow, dependent→depend, effective→effect |
| 7a (final $e$) | ($m$>1) E→ $\varepsilon$; ($m$=1 and not `*o`) E→ $\varepsilon$ | probate→probat, cease→ceas |
| 7b (double $l$) | ($m$>1 and `*d` and `*L`) → single letter | controll→control, roll→roll |

Table 9: The Porter stemmer as presented in the slides (in Porter's paper the same rules are numbered 1a, 1b, 1c, 2, 3, 4, 5a, 5b).

**Watch out — corrections to the Porter slides**

- **Consonant definition**: the slide reads "a letter other than A, E, I, O, U, and Y preceded by consonant". Porter's definition is "other than A, E, I, O, U, *and other than* Y preceded by a consonant": a Y after a consonant is a vowel (*happy*), otherwise a consonant (*toy*).
- **Step 2a**: the slide writes the condition of EED as $m > 1$, then uses *agreed* → *agree* as an example, but *agr* has $m = 1$. Porter's rule is ($m > 0$) EED → EE, as in the table above.
- **Step 7a**: the slide's second rule is "($m$=1 and not `*o`) NESS → $\varepsilon$" with *goodness* as the example (copied from step 5). In Porter's algorithm this rule removes a final E: *cease* → *ceas*.

### 6.3.5 Examples and problems

[Slides · Syntax: RE & word: 59–62]

**Worked example — running the steps**

- *controlling* $\xrightarrow{\text{2a: }-\text{ING}}$ *controll* $\xrightarrow{\text{7b}}$ *control*.
- *singing* $\xrightarrow{\text{2a: }-\text{ING}}$ *sing* (no further rule applies).
- *computers* $\xrightarrow{\text{1: }-\text{S}}$ *computer* $\xrightarrow{\text{6: }-\text{ER}}$ *comput*.
- *generalizations* $\xrightarrow{1}$ *generalization* $\xrightarrow{\text{4: IZATION}\to\text{IZE}}$ *generalize* $\xrightarrow{\text{5: ALIZE}\to\text{AL}}$ *general* $\xrightarrow{\text{6: }-\text{AL}}$ *gener*.

**Watch out — the examples slide**

On the slide, *singing* ends with "Step 6, Rule 4 → compute", a line copied from the *computers* example: *singing* simply becomes *sing*. The slide also stops *computers* at *compute* and *generalizations* at *general*. Running the complete algorithm (e.g. NLTK's implementation) gives *comput* and *gener*: stems are not required to be words, and Porter's algorithm removes ER and AL in its last derivational step.

**Problems.** Being heuristic, the stemmer makes two kinds of errors:

- **over-stemming**: unrelated words get the same stem. *elephants* → *elephant* (step 1) → *eleph* (step 6 removes ANT as if it were a suffix); *university* and *universal* both become *univers*;
- **under-stemming**: forms of the same word get different stems. *thesis* → *thesi* but *theses* → *these*.

The slides also list *doing* → *doe* as a problem; the standard algorithm actually returns *do* for *doing* (the E of step 2b is added only when $m = 1$, and *do* has $m = 0$).

References: Porter, M. F. (1980), *An algorithm for suffix stripping*, Program 14(3):130–137; the Porter stemmer home page https://tartarus.org/martin/PorterStemmer/; Jurafsky and Martin, Section 3.4. Online demos: https://9ol.es/porter_js_demo.html; stemmers for other languages: http://text-processing.com/demo/stem/.

### 6.3.6 Two texts to test a stemmer

[Slides · Syntax: RE & word: 63–65]

The slides propose two short texts full of traps. Here is what a real stemmer does with them.

**Worked example — Italian: a Snowball stemmer**

*Sono stato in uno stato il cui statuto prevede che la biscia non possa entrare in una bisca. Una messe di polemiche è piovuta sullo statuto, per le limitazioni messe alla biscia, che è andata a messa insieme al console per consolarsi.*

| Words | Meanings | Stem |
|---|---|---|
| stato, stato, statuto | "been", "state", "statute" | *stat* |
| biscia, bisca | "grass snake", "gambling den" | *bisc* |
| messe, messe, messa | "harvest", "put" (participle), "mass" | *mess* |
| console, consolarsi | "consul", "to console oneself" | *consol* |

Every group is over-stemmed: completely unrelated words collapse. Even a lemmatiser needs context here: the two *stato* have different lemmas (*essere*, the verb; *stato*, the noun), and only the part of speech tells them apart (Section 7.2).

**Worked example — English: Miss Marple**

*Once Miss Marple had a cold and she went to the doctor: "I go!", she said, and she read the doctor's doctoral thesis. "These are good theses!", and – although she missed her train – Miss Marple understood that University has a universal mission.*

Porter gives: *Miss*, *missed* → *miss* (a title and a verb merged); *doctor*, *doctoral* → *doctor*; *University*, *universal* → *univers*; *theses* → *these*, the same as the pronoun *These*, while *thesis* → *thesi*; *has* → *ha*; *are* → *ar*. A lemmatiser instead returns *thesis* for *theses*, *be* for *are*, *have* for *has*, and keeps *university* and *universal* apart.

**Intuition — which one to use**

Stemming is fast, needs no dictionary and increases *recall* in search (more forms of a word are found), at the price of *precision* (unrelated words are merged). Lemmatisation is slower and needs a vocabulary and often the part of speech, but its output is made of real words. Choose according to which kind of error hurts your application more.

## 6.4 Minimum Edit Distance

[Slides · Syntax: RE & word: 66–82]

### 6.4.1 How similar are two strings?

[Slides · Syntax: RE & word: 66–68, 72–73]

Many tasks need a measure of how close two strings are:

- **Spell correction**: the user typed *graffe*. Which is closest: *graf*, *graft*, *grail* or *giraffe*?
- **Computational biology**: align two sequences of nucleotides (the bases Adenine, Guanine, Thymine/Uracil and Cytosine of DNA/RNA), giving each letter a partner letter or a gap -:

```
AGGCTATCACCTGACCTCCAGGCCGATGCCC
TAGCTATCACGACCGCGGTCGATTTGCCCGAC
```
aligned:
```
-AGGCTATCACCTGACCTCCAGGCCGA-TGCCC--
TAG-CTATCAC-GACCGC-GGTCGATTTGCCCGAC
```

Alignments are used in molecular phylogenetics (evolutionary relationships from hereditary differences in DNA), in comparing the genomes of ill and healthy individuals, and to understand the function of unknown proteins.

- **Evaluating machine translation and speech recognition**: compare the system's output (hypothesis H) with a reference (R), counting substitutions (S), insertions (I) and deletions (D):

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| R: | Spokesman | confirms | * | senior | government | adviser | was | shot | * |
| H: | Spokesman | said | the | senior | * | adviser | was | shot | dead |
| | | S | I | | D | | | | I |

- **Named entity extraction and coreference**: deciding that *IBM Inc.* and *IBM*, or *Stanford President John Hennessy* and *Stanford University President John Hennessy*, refer to the same entity.

### 6.4.2 Definition

[Slides · Syntax: RE & word: 69–71]

**Definition — Minimum edit distance**

The **minimum edit distance** between two strings is the minimum number of editing operations

- **insertion** of a character,
- **deletion** of a character,
- **substitution** of a character with another,

needed to transform one string into the other. If operations have *costs*, it is the minimum total cost. With cost 1 for insertions and deletions and **cost 2 for substitutions** it is called the **Levenshtein distance** (the version used in these notes unless stated otherwise).

An **alignment** shows the operations explicitly. For *intention* and *execution* (* = gap):

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| I | N | T | E | * | N | T | I | O | N |
| \| | \| | \| | \| | \| | \| | \| | \| | \| | \| |
| * | E | X | E | C | U | T | I | O | N |
| d | s | s | | i | s | | | | |

One deletion (d), three substitutions (s) and one insertion (i):

- if every operation costs 1, the distance is $1 + 3 + 1 = 5$;
- with Levenshtein costs (substitution = 2), it is $1 + 3 \cdot 2 + 1 = 8$.

**Worked example — the spelling question**

Levenshtein distances from *graffe*: *giraffe* $= 1$ (insert `i`), *graf* $= 2$ (delete `f` and `e`), *graft* $= 3$ (delete `f`, substitute `e`→`t`), *grail* $= 5$. So *giraffe* is the best correction. (With all costs equal to 1: $1, 2, 2, 3$.)

### 6.4.3 Edit distance as search

[Slides · Syntax: RE & word: 74–78]

Finding the minimum edit distance is a search for a path (a sequence of edits) from the start string to the final string:

- **initial state**: the word we are transforming;
- **operators**: insert, delete, substitute;
- **goal state**: the word we are trying to reach;
- **path cost**: what we minimise, the number (or cost) of edits.

The space of all edit sequences is huge, so we cannot explore it naively. But many distinct paths end up in the *same* state (the same prefix of $X$ turned into the same prefix of $Y$), and for each such state we only need to remember the *cheapest* way to reach it. That is exactly the idea of dynamic programming.

**Definition — The subproblems**

For strings $X$ of length $n$ and $Y$ of length $m$, let

$$D(i, j) = \text{edit distance between } X[1..i] \text{ and } Y[1..j],$$

i.e. between the first $i$ characters of $X$ and the first $j$ characters of $Y$. The edit distance between $X$ and $Y$ is $D(n, m)$.

**Dynamic programming** computes $D(n, m)$ *bottom-up* in a table, solving the problem by combining solutions to subproblems: first $D(i, j)$ for small $i, j$, then larger ones from the smaller values already computed.

### 6.4.4 The algorithm

[Slides · Syntax: RE & word: 79–80]

**Key result — Levenshtein distance by dynamic programming**

**Initialisation** (distance from the empty string):

$$D(i, 0) = i, \qquad D(0, j) = j.$$

**Recurrence**: for each $i = 1, \ldots, n$ and each $j = 1, \ldots, m$,

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 & \text{deletion of } X[i] \\ D(i,j-1) + 1 & \text{insertion of } Y[j] \\ D(i-1,j-1) + \begin{cases} 2 & \text{if } X[i] \neq Y[j] \\ 0 & \text{if } X[i] = Y[j] \end{cases} & \text{substitution or copy} \end{cases}$$

**Termination**: $D(n,m)$ is the distance.

**Intuition — reading the recurrence**

To turn $X[1..i]$ into $Y[1..j]$, look at the *last* column of an optimal alignment. Either the last character of $X$ was deleted (and the rest is an optimal alignment of $X[1..i-1]$ with $Y[1..j]$), or the last character of $Y$ was inserted, or the two last characters are aligned with each other (free if equal, a substitution otherwise). The minimum over the three cases is the answer.

The general version (Jurafsky and Martin) uses arbitrary costs `del-cost`, `ins-cost`, `sub-cost`, which can depend on the letters, to model that some letters are more likely to be inserted or confused than others; copying a letter costs `sub-cost`$(x, x) = 0$.

**Watch out — indices on the slide**

The slide defines $X$ of length $n$ and $Y$ of length $m$, but the recurrence loops over $i = 1 \ldots M$, $j = 1 \ldots N$ and terminates with $D(N, M)$. The letters are swapped: $i$ indexes $X$ and runs to $n$, $j$ indexes $Y$ and runs to $m$, and the result is $D(n, m)$. The bottom-up loop also covers $1 \leq i \leq n$, $1 \leq j \leq m$ (the slide writes strict inequalities).

### 6.4.5 The table and the backtrace

[Slides · Syntax: RE & word: 81–82]

**Worked example — intention → execution**

Rows: the source *intention*; columns: the target *execution*; `#` is the empty prefix. Each cell is computed from its left, upper and upper-left neighbours.

| | # | e | x | e | c | u | t | i | o | n |
|---|---|---|---|---|---|---|---|---|---|---|
| # | **0** | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| **i** | **1** | 2 | 3 | 4 | 5 | 6 | 7 | 6 | 7 | 8 |
| **n** | 2 | **3** | 4 | 5 | 6 | 7 | 8 | 7 | 8 | 7 |
| **t** | 3 | 4 | **5** | 6 | 7 | 8 | 7 | 8 | 9 | 8 |
| **e** | 4 | 3 | 4 | **5** | **6** | 7 | 8 | 9 | 10 | 9 |
| **n** | 5 | 4 | 5 | 6 | 7 | **8** | 9 | 10 | 11 | 10 |
| **t** | 6 | 5 | 6 | 7 | 8 | 9 | **8** | 9 | 10 | 11 |
| **i** | 7 | 6 | 7 | 8 | 9 | 10 | 9 | **8** | 9 | 10 |
| **o** | 8 | 7 | 8 | 9 | 10 | 11 | 10 | 9 | **8** | 9 |
| **n** | 9 | 8 | 9 | 10 | 11 | 12 | 11 | 10 | 9 | **8** |

Table 10: Levenshtein table. The bottom-right cell is the distance (8). Shaded cells: one minimum-cost path found by the backtrace.
Some cells, step by step:

$$D(1,1) = \min\big(D(0,1)+1,\ D(1,0)+1,\ D(0,0)+2\big) = \min(2,2,2) = 2$$
$$D(4,1) = \min\big(D(3,1)+1,\ D(4,0)+1,\ D(3,0)+0\big) = \min(5,5,3) = 3$$
$$D(9,9) = \min\big(D(8,9)+1,\ D(9,8)+1,\ D(8,8)+0\big) = \min(10,10,8) = 8$$

In $D(1,1)$ the letters differ (`i`, `e`), so the diagonal costs 2; in $D(4,1)$ and $D(9,9)$ they are equal, so the diagonal is a free copy.

**Backtrace.** The table gives the distance, not the alignment. To get the alignment, when filling each cell we remember *which* neighbour(s) gave the minimum (an arrow $\leftarrow$, $\uparrow$ or $\nwarrow$). After the table is full, we start from the bottom-right cell and follow the arrows back to $(0,0)$: $\uparrow$ is a deletion, $\leftarrow$ an insertion, $\nwarrow$ a substitution or a copy. The shaded path, read from the top-left, is: delete `i`; `n`→`e`; `t`→`x`; copy `e`; insert `c`; `n`→`u`; copy `t`, `i`, `o`, `n`. That is exactly the alignment shown above, with cost $1+2+2+0+1+2 = 8$. When several neighbours tie, there are several optimal alignments.

**Intuition — cost of the algorithm**

The table has $(n+1)(m+1)$ cells, each computed in constant time from three neighbours, so the algorithm takes $O(nm)$ time and space; the backtrace adds $O(n+m)$. The naive search over all edit sequences is exponential.

### 6.4.6 Exercises

[Not in the slides: added material]

**Exercise 1 — small tables**

Compute the Levenshtein table for *cat* → *cut* and for *cat* → *act*. Give the distance and an optimal alignment for each.

**Solution**

*cat* → *cut* (rows c, a, t; columns c, u, t):

| | # | *c* | *u* | *t* |
|---|---|---|---|---|
| # | 0 | 1 | 2 | 3 |
| *c* | 1 | 0 | 1 | 2 |
| *a* | 2 | 1 | 2 | 3 |
| *t* | 3 | 2 | 3 | 2 |

Distance 2: copy `c`, substitute `a`→`u` (cost 2), copy `t`. Deleting `a` and inserting `u` also costs 2: there are two optimal alignments.
*cat* → *act* (columns a, c, t):

| | # | *a* | *c* | *t* |
|---|---|---|---|---|
| # | 0 | 1 | 2 | 3 |
| *c* | 1 | 2 | 1 | 2 |
| *a* | 2 | 1 | 2 | 3 |
| *t* | 3 | 2 | 3 | 2 |

Distance 2, e.g. delete `c`, copy `a`, insert `c`, copy `t`.
*Lesson*: swapping two adjacent letters costs 2. The Damerau–Levenshtein variant adds *transposition* as a fourth operation, because swaps are a very common typing error.

**Cheat sheet — syntax at the word level**

- Word segmentation: spaces are only an approximation; Chinese, Japanese, Thai have no word spaces. MaxMatch = greedy longest dictionary word (good for Chinese, bad for English).
- Stop words: very frequent, little content; no universal list. A word frequent in a document may be topical: compare with the corpus.
- $\mathrm{tf}(t,d) = \#t/\mathrm{len}(d)$, $\mathrm{idf}(t,C) = \log\big(|C|/|\{d : t \in d\}|\big)$, tf-idf = tf · idf; a word in every document has idf = 0.
- Stemming = heuristic suffix chopping (Porter: measure $m$ in $[C](VC)^m[V]$, conditional rules in 7 steps; over- and under-stemming). Lemmatisation = dictionary + morphology, returns the lemma.
- Minimum edit distance: insertions, deletions, substitutions; Levenshtein: substitution costs 2. Dynamic programming: $D(i,0) = i$, $D(0,j) = j$, $D(i,j)$ = minimum over the upper neighbour +1 (deletion), the left neighbour +1 (insertion), the diagonal neighbour +2 (substitution) or +0 (copy); the backtrace gives the alignment; time and space $O(nm)$.

# 7 Syntax at the Sentence Level

[Slides · Syntax: sentence, rules: 1–17; Syntax: sentence, HMM: 1–20]

Once a text is split into words, the next questions concern whole sentences: where does a sentence end, what grammatical category does each word have in context, and how do the words group into a structure? This section covers the sentence-level tasks of the syntactic level (Section 3.3):

1. **sentence breaking** (Section 7.1);
2. **part-of-speech tagging** and the Penn Treebank tagset (Section 7.2);
3. **grammars and parsing**: Chomsky's generative grammar, the Chomsky hierarchy, a small grammar for English, Definite Clause Grammars (Section 7.3);
4. **statistical POS tagging** with Hidden Markov Models and the Viterbi algorithm (Section 7.4).

The first three are *rule-based* (deductive) approaches, the last one is *statistical* (inductive): two eras of NLP (Section 2.3) side by side.

## 7.1 Sentence Breaking

[Slides · Syntax: sentence, rules: 2–4]

**Definition — Sentence boundary disambiguation**

**Sentence boundary disambiguation** (SBD), also called *sentence breaking* or *sentence boundary detection*, is the problem of deciding where sentences begin and end.

Many NLP tools need their input divided into sentences, but punctuation is ambiguous:

- a **period** may mark an abbreviation (*Mr.*, *etc.*), a decimal point (*3.14*), an ellipsis (. . . ) or be part of an e-mail address, rather than the end of a sentence. About **47%** of the periods in the Wall Street Journal corpus mark abbreviations;
- **question and exclamation marks** appear in embedded quotations, emoticons, computer code and slang.

Languages like Japanese and Chinese have unambiguous markers for the end of a sentence.

**Key result — The standard rule-based approach**

To decide whether a token ends a sentence:
(a) if it is a period, it ends a sentence;
(b) but if the preceding token is in a hand-compiled list of **abbreviations**, it does not;
(c) and if the next token is **capitalised**, then it does end a sentence.
This simple strategy gets about **95%** of sentences right.

**Worked example — where the rules break**

*Mr. Smith paid 3.14 dollars. He left at 5 p.m. Then he slept.*

- *Mr.*: abbreviation, rule (b): not a boundary. Correct.
- *3.14*: the period is inside a token, not a token of its own. Correct.
- *p.m. Then*: *p.m.* is an abbreviation (b says "no boundary"), but the next word is capitalised (c says "boundary"). Here the sentence *does* end: the period of an abbreviation can also be the final period.

NLTK's pre-trained sentence splitter returns *["Mr. Smith paid 3.14 dollars.", "He left at 5 p.m. Then he slept."]*: it gets the first two cases right and misses exactly the third. The remaining 5% of errors are cases like this one, which need more context (or a statistical model) to resolve.

## 7.2 Part-of-Speech Tagging

[Slides · Syntax: sentence, rules: 5–6]

**Definition — Part-of-speech tagging**

**Part-of-speech tagging** (POS tagging, POST), also called *grammatical tagging* or *word-category disambiguation*, is the process of marking each word of a text with its part of speech, based on both its **definition** and its **context**, i.e. its relationship with adjacent and related words in a phrase, sentence or paragraph.

(If the categories themselves are fuzzy, see the recap with the Italian equivalents in Section 4.2.) The word "context" is the whole point: many words have several possible tags, and only the sentence tells which one is right.

**Worked example — the same word, different tags**

- *I book/VB a flight* vs. *I read a book/NN.*
- *Time/NN flies/VBZ like/IN an arrow* (time passes quickly) vs. *Time/VB flies/NNS like/IN an arrow* (measure the speed of flies as you would an arrow) vs. *Time/NN flies/NNS like/VBP an arrow* ("time flies", a kind of fly, are fond of an arrow).

### 7.2.1 The Penn Treebank tagset

[Slides · Syntax: sentence, rules: 6]

The most widely used set of tags for English is the **Penn Treebank** tagset, with 36 word tags:

| | Tag | Description | | Tag | Description |
|---|---|---|---|---|---|
| 1 | CC | coordinating conjunction | 19 | PRP$ | possessive pronoun |
| 2 | CD | cardinal number | 20 | RB | adverb |
| 3 | DT | determiner | 21 | RBR | adverb, comparative |
| 4 | EX | existential *there* | 22 | RBS | adverb, superlative |
| 5 | FW | foreign word | 23 | RP | particle |
| 6 | IN | preposition / subord. conj. | 24 | SYM | symbol |
| 7 | JJ | adjective | 25 | TO | *to* |
| 8 | JJR | adjective, comparative | 26 | UH | interjection |
| 9 | JJS | adjective, superlative | 27 | VB | verb, base form |
| 10 | LS | list item marker | 28 | VBD | verb, past tense |
| 11 | MD | modal | 29 | VBG | verb, gerund / present participle |
| 12 | NN | noun, singular or mass | 30 | VBN | verb, past participle |
| 13 | NNS | noun, plural | 31 | VBP | verb, non-3rd person sing. present |
| 14 | NNP | proper noun, singular | 32 | VBZ | verb, 3rd person sing. present |
| 15 | NNPS | proper noun, plural | 33 | WDT | *wh*-determiner |
| 16 | PDT | predeterminer | 34 | WP | *wh*-pronoun |
| 17 | POS | possessive ending | 35 | WP$ | possessive *wh*-pronoun |
| 18 | PRP | personal pronoun | 36 | WRB | *wh*-adverb |

Table 11: The Penn Treebank POS tagset.

**Intuition — reading the tags**

The tags encode morphology (Section 4.1): NN/NNS differ by the plural inflection, VB/VBD/VBG/VBN/VBP/VBZ by the verb's inflection, JJ/JJR/JJS by comparison. So a tagger is also, partly, a morphological analyser, and the tag is what a lemmatiser needs to choose the right lemma (Section 6.3).

**Worked example — a tagged sentence**

*The/DT boy/NN 's/POS cars/NNS are/VBP different/JJ colors/NNS ./.*

Online taggers to try: `http://parts-of-speech.info/` and Stanford CoreNLP, `https://corenlp.run/`.

There are two families of approaches, both covered in this course:

- **rule-based** (deductive): a grammar and a parser decide the tags as a by-product of the structure of the sentence (Section 7.3);
- **statistical** (inductive): a model learnt from a tagged corpus picks the most probable sequence of tags (Section 7.4).

## 7.3 Grammars and Parsing

[Slides · Syntax: sentence, rules: 7–17]

**Definition — Parsing**

In computational linguistics, **parsing** is the formal analysis by a computer of a sentence (or other string of words) into its **constituents**, resulting in a **parse tree** that shows their syntactic relations to each other. The tree may also contain semantic and other information.

Parsing is one of the possible ways to address the POS problem: once the sentence has a tree, every word has a category (the node right above it).

### 7.3.1 Chomsky and generative grammar

[Slides · Syntax: sentence, rules: 8–9]

In *Syntactic Structures* (1957), Noam Chomsky proposed the theory of **generative grammar**: a finite set of rules that recursively "specify" or "generate" all the well-formed expressions of a natural language (and only them). It is considered one of the most significant contributions to theoretical linguistics of the 20th century.

Recursion is what links generative grammar to the question of Section 1.1. "Animal languages" are forms of animal communication that show similarities to human language: the inventory of signs may be large, the signs relatively arbitrary, and they seem to be produced with some volition. But researchers agree that they are not as complex or expressive as human language: they lack the *creation of new patterns of signs under varied circumstances*, i.e. the productivity that recursive rules provide.

### 7.3.2 Grammars and the Chomsky hierarchy

[Slides · Syntax: sentence, rules: 10–13]

**Definition — Grammar and formal language**

A **grammar** specifies the compositional structure of complex messages. A **formal language** is a set of strings of **terminal symbols** (the words). Each string of the language can be analysed (recognised) or generated by the grammar, which is a set of **rewrite rules**, e.g.

$$S \to NP\ VP$$
$$\mathit{Article} \to \textbf{the} \mid \textbf{a} \mid \textbf{an} \mid \ldots$$

Here $S$ is the **sentence** (start) symbol; $NP$ (noun phrase), $VP$ (verb phrase) and *Article* are **nonterminals**; **the**, **a**, **an** are terminals; the bar | separates alternatives.

Restricting the *form* of the rules gives classes of grammars of increasing power, the **Chomsky–Schützenberger hierarchy**:

| Grammar | Languages | Automaton | Rules |
|---|---|---|---|
| Type-0 | recursively enumerable | Turing machine | no restrictions |
| Type-1 | context-sensitive | linear-bounded automaton | $\alpha A \beta \to \alpha \gamma \beta$ |
| Type-2 | context-free | (non-det.) pushdown automaton | $A \to \gamma$ |
| Type-3 | regular | finite-state automaton | $A \to aB$, $A \to a$ |

Table 12: The Chomsky hierarchy. $A, B$: nonterminals; $a$: a terminal; $\alpha, \beta, \gamma$: strings of terminals and nonterminals.

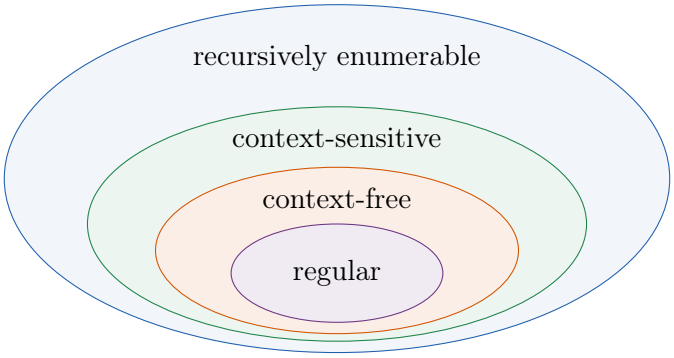



Figure 8: Each class contains the ones below it. Regular expressions (Section 5) describe exactly the regular languages.

In words:

- **Regular**: *nonterminal* $\to$ *terminal* [*nonterminal*], e.g. $S \to \mathbf{a}S$, $S \to \Lambda$ (the empty string): any number of **a**. Recognised with a fixed, finite memory.
- **Context-free**: *nonterminal* $\to$ anything, e.g. $S \to \mathbf{a}S\mathbf{b}$: generates $\mathbf{a}^n\mathbf{b}^n$, which needs to *count* the a's, impossible with finite memory (a stack suffices).
- **Context-sensitive**: more nonterminals on the left-hand side, e.g. $ASB \to AA\mathbf{a}BB$: $S$ can be rewritten only when surrounded by $A$ and $B$.
- **Recursively enumerable**: no constraints.

**Watch out — "natural languages are probably context-free"**

The slides say that natural languages are probably context-free, hence parsable in real time. This is a good working assumption: context-free grammars cover most syntactic constructions and are the basis of most parsers. Strictly speaking, however, a few constructions are known to go beyond context-free power, e.g. the cross-serial dependencies of Swiss German (Shieber, 1985). Linguists therefore say that natural languages are *mildly* context-sensitive.

### 7.3.3 A grammar for the Wumpus world

[Slides · Syntax: sentence, rules: 14–15]

The Wumpus world is a toy game from AI textbooks: an agent explores a cave with pits and a monster, the wumpus, and perceives stenches and breezes. A tiny fragment

of English for talking about it consists of a **lexicon** and a **grammar**.

**Lexicon**

| | | |
|---|---|---|
| *Noun* | → | stench \| breeze \| glitter \| nothing \| wumpus \| pit \| pits \| gold \| east \| ... |
| *Verb* | → | is \| see \| smell \| shoot \| feel \| stinks \| go \| grab \| carry \| kill \| turn \| ... |
| *Adjective* | → | right \| left \| east \| south \| back \| smelly \| ... |
| *Adverb* | → | here \| there \| nearby \| ahead \| right \| left \| east \| ... |
| *Pronoun* | → | me \| you \| I \| it \| ... |
| *Name* | → | John \| Mary \| Boston \| ... |
| *Article* | → | the \| a \| an \| ... |
| *Preposition* | → | to \| in \| on \| near \| ... |
| *RelPronoun* | → | that \| which \| ... |

The lexicon is divided into **closed classes** (articles, pronouns, prepositions: a small fixed set, new members are almost never added) and **open classes** (nouns, verbs, adjectives, adverbs: new words appear all the time).

**Grammar**

| | | | |
|---|---|---|---|
| $S$ | → | $NP\ VP$ | I + feel a breeze |
| *Subject* | → | *Pronoun* | I |
| | \| | *Noun* | pits |
| | \| | *Article Noun* | the + wumpus |
| | \| | *Article Adjective Noun* | the + smelly + wumpus |
| $NP$ | → | *Subject* | the wumpus |
| | \| | *Subject PP* | the wumpus + to the east |
| | \| | *Subject RelClause* | the wumpus + that is smelly |
| $VP$ | → | $Verb$ | stinks |
| | \| | $Verb\ NP$ | feel + a breeze |
| | \| | $Verb$ *Adjective* | is + smelly |
| | \| | $Verb\ PP$ | turn + to the east |
| | \| | $Verb$ *Adverb* | go + ahead |
| $PP$ | → | *Preposition* $NP$ | to + the east |
| *RelClause* | → | *RelPronoun* $VP$ | that + is smelly |

All rules have a single nonterminal on the left: this is a **context-free** grammar. Note the recursion: $NP \rightarrow$ *Subject* $PP$ and $PP \rightarrow$ *Preposition* $NP$, so noun phrases can nest without limit (*the wumpus to the east of the pit near the gold ...*).

**Worked example — parsing "I feel a breeze"**

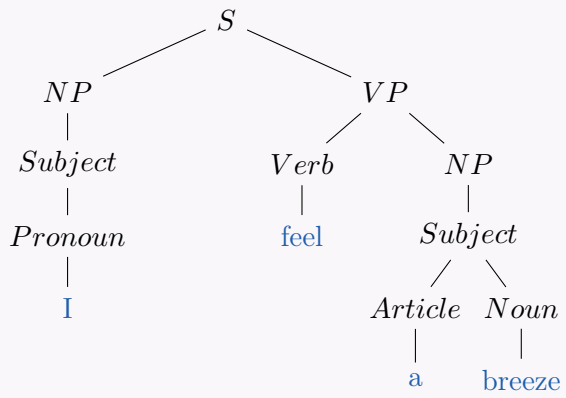



Figure 9: Parse tree. The node right above each word is its part of speech: *I*/Pronoun, *feel*/Verb, *a*/Article, *breeze*/Noun.
Derivation, always rewriting the leftmost nonterminal: $S \Rightarrow NP\ VP \Rightarrow Subject\ VP \Rightarrow Pronoun\ VP \Rightarrow$ I $VP \Rightarrow$ I $Verb\ NP \Rightarrow$ I feel $NP \Rightarrow$ I feel $Subject \Rightarrow$ I feel $Article\ Noun \Rightarrow$ I feel a breeze.

**Watch out — this grammar overgenerates**

The same rules also generate **I stinks*, **me feel a breeze* and **the wumpus feel a breeze*: nothing checks that subject and verb agree in person and number, or that a pronoun in subject position is *I* and not *me*. Adding a separate nonterminal for every combination (singular/plural, subject/object, ...) would make the grammar explode. Definite Clause Grammars solve this with *arguments*.

### 7.3.4 Definite Clause Grammars

[Slides · Syntax: sentence, rules: 16–17]

**Definition — Definite Clause Grammars (DCG)**

**DCGs** are a notation for grammars natively integrated in the Prolog programming language.

- They represent context-free grammars...
- ...but nonterminals can have **arguments** (compare with *attribute grammars*), which can express context-dependent constraints: this makes the formalism extremely powerful.
- Being Prolog, the same grammar works both as a **recogniser** (is this sentence grammatical?) and as a **generator** (list all grammatical sentences), and arbitrary reasoning can be added on top of it ("but, to your relief, we will not!").

**Worked example — agreement with an argument**

```
s           --> np(Num), vp(Num).          % subject and verb share Num
np(Num)     --> article, noun(Num).
vp(Num)     --> verb(Num).
article     --> [the].
noun(sing) --> [wumpus].     noun(plur) --> [wumpuses].
verb(sing) --> [stinks].     verb(plur) --> [stink].
```

The variable `Num` is the same in `np(Num)` and `vp(Num)`, so the query `phrase(s, [the, wumpus, stinks])` succeeds while `phrase(s, [the, wumpus, stink])` fails. The query `phrase(s, X)` enumerates all (two) sentences of the language. DCGs can be run online in SWISH, https://swish.swi-prolog.org/.

**Intuition — why arguments add power**

Without arguments, enforcing agreement needs one copy of each rule per combination of features ($NP_{sing}$, $NP_{plur}$, ...). With arguments, one rule says "whatever the number is, it must be the same here and there". Arguments can carry any Prolog term, including counters, so DCGs can describe even languages that are not context-free, such as $\mathbf{a}^n\mathbf{b}^n\mathbf{c}^n$.

## 7.4 Statistical POS Tagging: Hidden Markov Models

[Slides · Syntax: sentence, HMM: 1–20]

The rule-based approach of Section 7.3 needs a linguist to write the grammar. The statistical approach instead *learns* from a corpus already tagged by hand how likely each tag is after another tag, and how likely each word is for each tag, then picks the most probable tag sequence for a new sentence. The classic model is the Hidden Markov Model. The slides (from a Microsoft course on edX) introduce it with a non-linguistic example, which we follow; then we map it to POS tagging.

### 7.4.1 The model

[Slides · Syntax: sentence, HMM: 3]

**Definition — Hidden Markov Model (HMM)**

An **HMM** models a process that moves between **states**:

- the states are **hidden**: we cannot observe them;
- at each step the current state emits an **output** (observation), which we do see.

Our goal: from the sequence of outputs, find the most likely **sequence of states**. Formally an HMM has

- a set of states $Q$ and of possible observations $O$;
- **start probabilities** $\pi(s)$: probability that the first state is $s$;

- **transition probabilities** $a(s' \to s) = P(s_t = s \mid s_{t-1} = s')$;
- **emission probabilities** $b_s(o) = P(o_t = o \mid s_t = s)$.

Two assumptions make it "Markov" and keep it tractable: the next state depends only on the current state (not on the whole history), and each output depends only on the state that emits it.

### 7.4.2 The weather example

[Slides · Syntax: sentence, HMM: 4–5]

We are interested in the weather of a small town during the week of 5 October 1900. No weather records were kept, but we have the diary of a teenager who wrote down the one snack he had each day: hot chocolate, soda or ice cream. For our purposes the weather is *cold*, *normal* or *hot*. We are given:

| **Start / transition** $P$(today \| yesterday) | | | | **Emission** $P$(snack \| weather) | | | |
|---|---|---|---|---|---|---|---|
| from ↓ to → | Cold | Normal | Hot | | hot choc. | soda | ice cream |
| Start | .1 | .8 | .1 | Cold | .7 | .3 | 0 |
| Cold | .7 | .1 | .2 | Normal | .1 | .7 | .2 |
| Normal | .3 | .4 | .3 | Hot | 0 | .6 | .4 |
| Hot | .2 | .4 | .4 | | | | |

Each row sums to 1. For example $P(\text{hot} \mid \text{cold}) = .2$ and $P(\text{soda} \mid \text{hot}) = .6$. The diary for the 7 days is

soda, soda, hot chocolate, ice cream, soda, soda, ice cream.

**Question**: what was the most likely weather sequence for the week?

**Intuition — why not try every sequence?**

With 3 states and 7 days there are $3^7 = 2187$ possible weather sequences; with the 45 tags of a POS tagset and a 20-word sentence there are $45^{20}$, far too many. The Viterbi algorithm finds the best one in time proportional to (days) × (states)$^2$, using the same trick as the edit distance (Section 6.4): for every state at every step, keep only the best way to get there.

### 7.4.3 The Viterbi algorithm

[Slides · Syntax: sentence, HMM: 6–16]

**Key result — Viterbi**

Let $v_t(s)$ be the probability of the *best* state sequence that ends in state $s$ on day $t$ and emits the first $t$ observations.

$$\begin{aligned} \text{day } 1: \quad & v_1(s) = \pi(s) \cdot b_s(o_1) && \triangleleft \text{ p(transition) } \times \text{ p(output)} \\ \text{day } t > 1: \quad & v_t(s) = \max_{s'} \big[\, v_{t-1}(s') \cdot a(s' \to s) \,\big] \cdot b_s(o_t) \end{aligned}$$

For each cell we also store a **backpointer** to the $s'$ that achieved the maximum. At the end, pick the state with the largest $v_T(s)$ and follow the backpointers back to day 1.

**Worked example — the first two days**

**Day 1** (soda):

$$v_1(\text{Cold}) = .1 \cdot .3 = .03, \quad v_1(\text{Normal}) = .8 \cdot .7 = .56, \quad v_1(\text{Hot}) = .1 \cdot .6 = .06.$$

**Day 2** (soda), cell Cold: which yesterday is the best way to arrive at Cold?

$$\begin{aligned} v_2(\text{Cold}) &= \max\big( \underbrace{.03 \cdot .7}_{\text{from Cold}}, \ \underbrace{.56 \cdot .3}_{\text{from Normal}}, \ \underbrace{.06 \cdot .2}_{\text{from Hot}} \big) \cdot .3 \\ &= \max(.021,\ .168,\ .012) \cdot .3 = .168 \cdot .3 = .0504 \qquad \triangleleft \text{ backpointer: Normal} \end{aligned}$$

Similarly $v_2(\text{Normal}) = \max(.03 \cdot .1,\ .56 \cdot .4,\ .06 \cdot .4) \cdot .7 = .224 \cdot .7 = .1568$ and $v_2(\text{Hot}) = \max(.03 \cdot .2,\ .56 \cdot .3,\ .06 \cdot .4) \cdot .6 = .168 \cdot .6 = .1008$, both with backpointer Normal.

Continuing the same way for all seven days fills the **trellis**:

| | Day 1<br>soda | Day 2<br>soda | Day 3<br>hot choc. | Day 4<br>ice cream | Day 5<br>soda | Day 6<br>soda | Day 7<br>ice cream |
|---|---|---|---|---|---|---|---|
| Hot | .06 | .1008<br>←N | 0 | **.00263**<br>←C | .000632<br>←H | .000152<br>←H | **.0000248**<br>←N |
| Normal | **.56** | **.1568**<br>←N | .00627<br>←N | .000659<br>←C | **.000738**<br>←H | **.000207**<br>←N | .0000165<br>←N |
| Cold | .03 | .0504<br>←N | **.0329**<br>←N | 0 | .000158<br>←H | .0000664<br>←N | 0 |

Table 13: The Viterbi trellis (values rounded to 3 significant digits). Under each value, the backpointer: the state of the previous day that gave the maximum. Shaded: the best path, recovered by backtracking from the largest value of day 7.

- Some cells are 0 because the output is impossible in that state: no hot chocolate on a hot day, no ice cream on a cold day.
- On day 7 the best value is Hot ($.0000248 > .0000165$), so the path ends in Hot. Its backpointer is Normal (day 6), whose backpointer is Normal (day 5), then Hot (day 4), then Cold (day 3), then Normal, Normal.

**Key result — Result**

**Most likely weather**: normal, normal, cold, hot, normal, normal, hot.

**Watch out — rounding on the slides**

The slides compute with rounded intermediate values, so some cells differ slightly from the exact ones (e.g. day 4 Hot: $.007 \cdot .4 = .0027$ instead of $.0329 \cdot .2 \cdot .4 = .00263$). One cell is a slip: day 2 Normal is written "$.224 \cdot .7 = .1589$", but $.224 \cdot .7 = .1568$. None of this changes the final path.

**Intuition — numbers get tiny**

Every step multiplies by numbers smaller than 1, so after a few dozen words the values underflow the floating-point range. Real implementations add *logarithms* of the probabilities instead of multiplying them: log is increasing, so the max (and the best path) does not change.

### 7.4.4 From weather to POS tags

[Slides · Syntax: sentence, HMM: 17–20]

The weather problem is POS tagging in disguise:

| | **Weather** | **POS tagging** |
|---|---|---|
| hidden states | cold, normal, hot | tags: DT, NN, VB, ... |
| observations | snacks in the diary | words of the sentence |
| transitions | $P$(hot \| cold) | $P$(NN \| DT): a noun after a determiner |
| emissions | $P$(soda \| hot) | $P$(flies \| VBZ) |
| answer | the weather of the week | the tag of each word |

Transition and emission probabilities are estimated by *counting* in a hand-tagged corpus (e.g. the Brown corpus): $P$(NN | DT) = (number of times DT is followed by NN) / (number of DTs). The Python code of the course implements Viterbi on the weather example and prints the guessed sequence (`day=1 normal`, `day=2 normal`, `day=3 cold`, ..., `day=7 hot`).

**HMM lab.** Experiment with the HMM Python code; extend it with other parts of speech (see the Brown corpus tags, https://en.wikipedia.org/wiki/Brown_Corpus#Part-of-speech_tags_used), words and transitions, and use it to tag "*time flies like an arrow*" (Section 7.2); invent other short ambiguous sentences and extend the code so that it tags them properly.

### 7.4.5 Maximum Entropy Markov Models

[Slides · Syntax: sentence, HMM: 19]

An HMM can reach very high accuracy in POS tagging, but it needs architectural additions to deal with **unknown words**, suffixes and so on: an HMM can only use $P(\text{word} \mid \text{tag})$, so a word never seen in training has no emission probability at all.

**Maximum Entropy Markov Models** (MEMM) combine the HMM idea (a sequence of hidden states, decoded with Viterbi) with **logistic regression**, a probabilistic classifier trained with supervised machine learning. Instead of $P(\text{word} \mid \text{tag})$, an MEMM directly models $P(\text{tag} \mid \text{word, previous tag, features})$, where the features can be anything: suffix (*-ing*, *-ly*), capitalisation, presence of digits or hyphens. This handles unknown words naturally. An MEMM is trained on a POS-annotated text of the language; this is how the Stanford POS tagger works (https://nlp.stanford.edu/software/tagger.shtml).

### 7.4.6 Exercises

[Not in the slides: added material]

**Exercise 1 — a shorter diary**

With the same model, find the most likely weather for a two-day diary: *hot chocolate, ice cream.*

**Solution**

Day 1 (hot chocolate): $v_1(\text{C}) = .1 \cdot .7 = .07$, $v_1(\text{N}) = .8 \cdot .1 = .08$, $v_1(\text{H}) = .1 \cdot 0 = 0$.
Day 2 (ice cream):

$$
\begin{aligned}
v_2(\text{C}) &= \max(\dots) \cdot 0 = 0 \\
v_2(\text{N}) &= \max(.07 \cdot .1,\ .08 \cdot .4,\ 0) \cdot .2 = .032 \cdot .2 = .0064 && \triangleleft \text{ from N} \\
v_2(\text{H}) &= \max(.07 \cdot .2,\ .08 \cdot .3,\ 0) \cdot .4 = .024 \cdot .4 = .0096 && \triangleleft \text{ from N}
\end{aligned}
$$

Best final state: Hot (.0096), backpointer Normal. Answer: **normal, hot**.
*Lesson*: on day 1 Cold explains hot chocolate better (.7 vs .1), but the start probabilities favour Normal so much (.8 vs .1) that Normal wins. The best path is a compromise between transitions and emissions.

**Cheat sheet — syntax at the sentence level**

- Sentence breaking: period = end, unless the previous token is an abbreviation, but yes if the next token is capitalised: about 95% correct.
- POS tagging: tag each word using definition *and* context; Penn Treebank tagset (36 tags: NN, NNS, VB, VBZ, JJ, DT, IN, ...).
- Generative grammar (Chomsky 1957). Chomsky hierarchy: regular $\subset$ context-free $\subset$ context-sensitive $\subset$ recursively enumerable (finite automaton, pushdown automaton, linear-bounded TM, TM).
- CFG = lexicon + rules with one nonterminal on the left; parse tree; plain

CFGs overgenerate (no agreement). DCGs add arguments to nonterminals (Prolog).
- HMM: hidden states, observed outputs; $\pi$, transitions $a$, emissions $b$. Viterbi: $v_t(s) = \max_{s'} v_{t-1}(s')\, a(s' \to s)\, b_s(o_t)$ + backpointers.
- MEMM = HMM + logistic regression; uses arbitrary features, handles unknown words.

# 8 Distributional Semantics

[Slides · Distributional semantics: 1–27; Semantics (long version): 1–6]

With this section we move up from syntax to **semantics**: *what does it mean?* (Section 3.1). The first and today most successful approach does not define meanings at all: it derives them from *how words are used* in large corpora. This is the idea behind word embeddings and, ultimately, behind LLMs. The slides are the first 13 slides of the *Distributional Semantic Models* tutorial by Stefan Evert, Alessandro Lenci, Marco Baroni and Gabriella Lapesa (`http://wordspace.collocations.de`).

This section covers:

1. the distributional hypothesis (Section 8.1);
2. the co-occurrence matrix and a thought experiment that shows its power (Section 8.2);
3. the geometric view: words as vectors, similarity as angle (Section 8.3).

## 8.1 The Distributional Hypothesis

[Slides · Distributional semantics: 3–16; Semantics (long version): 3–5]

### 8.1.1 Meaning and distribution

[Slides · Distributional semantics: 4–7]

Four famous quotations, each followed by what it means for us:

- "*Die Bedeutung eines Wortes liegt in seinem Gebrauch*" ("the meaning of a word is its use", Ludwig Wittgenstein).
  ⇒ **meaning = use = distribution in language**.
- "*You shall know a word by the company it keeps!*" (J. R. Firth, 1957).
  ⇒ **distribution = collocations** = habitual word combinations (Section 4.3).
- **Distributional hypothesis**: difference of meaning correlates with difference of distribution (Zellig Harris, 1954).
  ⇒ we can measure a **semantic distance** between words.

- "*What people know when they say that they know a word is not how to recite its dictionary definition – they know how to use it [. . . ] in everyday discourse*" (George Miller, 1986).

**Definition — Distributional hypothesis**

Words that occur in similar contexts tend to have similar meanings. Conversely, the more the contexts of two words differ, the more their meanings differ.

### 8.1.2 What is the meaning of "bardiwac"?

[Slides · Distributional semantics: 8–16]

You have never seen the word *bardiwac*. Read these sentences, one at a time:

- He handed her her glass of bardiwac.
- Beef dishes are made to complement the bardiwacs.
- Nigel staggered to his feet, face flushed from too much bardiwac.
- Malbec, one of the lesser-known bardiwac grapes, responds well to Australia's sunshine.
- I dined off bread and cheese and this excellent bardiwac.
- The drinks were delicious: blood-red bardiwac as well as light, sweet Rhenish.

**Intuition — what you just did**

After a few sentences you know that **bardiwac is a heavy red alcoholic beverage made from grapes**, i.e. a red wine (the word is invented; it stands for *claret*). Nobody gave you a definition: you inferred it from the *company the word keeps*: *glass of*, *too much*, *grapes*, *drinks*, *blood-red*, *beef dishes*. That is the distributional hypothesis in action, and it is how we learn most of our vocabulary.

The examples are hand-picked and edited, of course. But in a corpus like the British National Corpus (BNC) there is at least as much relevant information. The slides show a *word sketch* of the real word (BNC frequency 230): grouped by grammatical relation, its most typical companions are verbs like *uncork*, *gulp*, *sip*, *drink*, *swig*; coordinated nouns like *burgundy*, *Sancerre*, *scampi*, *ruby*; containers like *bottle*, *glass*, *jug*, *goblet*; modifiers like *full-bodied*, *blood-red*, *ready-to-drink*. A computer can collect these statistics automatically.

## 8.2 The Co-occurrence Matrix

[Slides · Distributional semantics: 17–21]

### 8.2.1 A thought experiment: deciphering hieroglyphs

[Slides · Distributional semantics: 17–20]

Imagine an unknown language. We know the meaning of some nouns (in brackets)

but not of *dog*, and we do not know any of the verbs. For each noun we count how often it appears as the *object* of each verb in a large corpus:

| | | **get** | **sij** | **ius** | **hir** | **iit** | **kil** |
|---|---|---|---|---|---|---|---|
| (knife) | naif | 51 | 20 | 84 | 0 | 3 | 0 |
| (cat) | ket | 52 | 58 | 4 | 4 | 6 | 26 |
| ??? | dog | 115 | 83 | 10 | 42 | 33 | 17 |
| (boat) | beut | 59 | 39 | 23 | 4 | 0 | 0 |
| (cup) | kap | 98 | 14 | 6 | 2 | 1 | 0 |
| (pig) | pigij | 12 | 17 | 3 | 2 | 9 | 27 |
| (banana) | nana | 11 | 2 | 2 | 0 | 18 | 0 |

Which known word is *dog* most similar to? Comparing rows (the similarity measure is explained in Section 8.3):

$$\text{sim}(\text{dog}, \text{naif}) = 0.770, \qquad \text{sim}(\text{dog}, \text{pigij}) = 0.939, \qquad \text{sim}(\text{dog}, \text{ket}) = 0.961.$$

*Dog* behaves most like *cat*, then *pig*, and much less like *knife*: it is probably an animal, a pet. We concluded this without understanding a single verb.

### 8.2.2 English as seen by the computer

[Slides · Distributional semantics: 21]

The "unknown language" was English with scrambled spelling: the verbs are **get, see, use, hear, eat, kill**, and the counts are real verb–object co-occurrence counts from the British National Corpus. For a computer, every language looks like hieroglyphs: it only has the counts.

**Definition — Co-occurrence matrix and distributional semantic model**

A **co-occurrence matrix** $M$ has one row per **target word** and one column per **context** (here: verbs taking the word as object; in general: nearby words, syntactic relations, documents, ...). The cell $M_{w,c}$ counts how often $w$ occurs with context $c$. The row $\mathbf{x}_w$ is the **distributional vector** of $w$. A **distributional semantic model** (DSM) represents the meaning of each word by such a vector.

**Watch out — how the similarity values were computed**

The slides do not say which similarity gives 0.770, 0.939 and 0.961. They are the **cosine similarity** (Section 8.3) of the rows after replacing each count $f$ by $\log(1+f)$. On the raw counts the cosine gives different values:

| similarity to *dog* | knife | cat | boat | cup | pig | banana |
|---|---|---|---|---|---|---|
| cosine, raw counts | 0.552 | **0.923** | 0.915 | 0.832 | 0.676 | 0.629 |
| cosine, $\log(1+f)$ | 0.770 | **0.961** | 0.828 | 0.850 | 0.939 | 0.785 |

With raw counts *boat* is almost as close to *dog* as *cat*, and *pig* is far: the large counts of the very frequent verb *get* dominate everything. The logarithm dampens large counts, so the rarer but more telling contexts (*hear*, *eat*, *kill*) matter too,

and the two animals come out on top. Real DSMs always apply such a weighting (log, TF-IDF-like association scores) before comparing vectors.

## 8.3 Geometric Interpretation

[Slides · Distributional semantics: 22–27; Semantics (long version): 6]

### 8.3.1 Words as points

[Slides · Distributional semantics: 22–24]

The row vector $\mathbf{x}_{\text{dog}} = (115, 83, 10, 42, 33, 17)$ describes how *dog* is used in the corpus. It can be seen as the coordinates of a **point** in an $n$-dimensional Euclidean space ($n = 6$ here, one dimension per context). We can draw two dimensions, *get* and *use*: $\mathbf{x}_{\text{dog}} = (115, 10)$, $\mathbf{x}_{\text{cat}} = (52, 4)$, $\mathbf{x}_{\text{knife}} = (51, 84)$, $\mathbf{x}_{\text{boat}} = (59, 23)$.

Two dimensions of the English V–Obj DSM

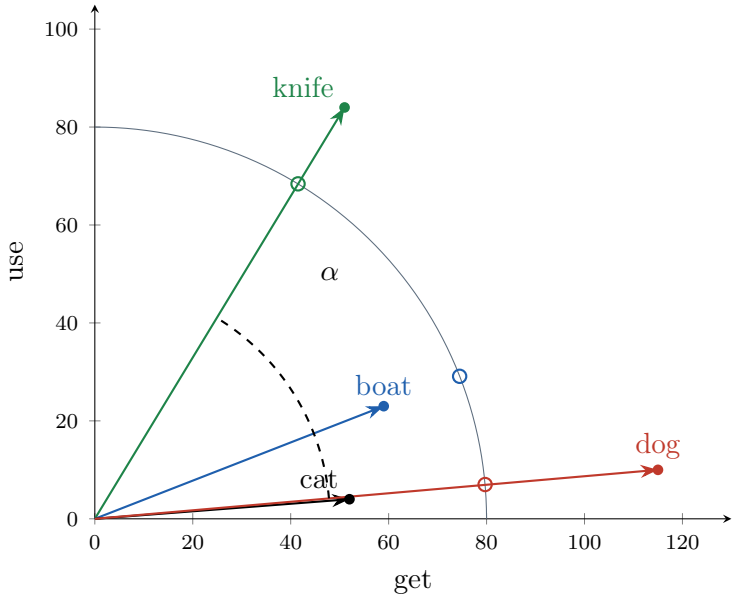



Figure 10: Words as vectors in the (get, use) plane. Circles on the arc: the same vectors normalised to the same length (only the direction is kept). *Dog* and *cat* point in almost the same direction even though they are far apart as points.

A first idea: **similarity = spatial proximity**, measured with the Euclidean distance. But the location of a point depends on the **frequency** of the noun: *dog* is about 2.7 times more frequent than *cat*, so all its counts are larger and its point lies much farther from the origin.

**Worked example — Euclidean distance is misleading**

$$d(\text{dog}, \text{cat}) = \sqrt{(115-52)^2 + (10-4)^2} = \sqrt{3969+36} \approx 63.3$$
$$d(\text{dog}, \text{boat}) = \sqrt{(115-59)^2 + (10-23)^2} = \sqrt{3136+169} \approx 57.5$$

By Euclidean distance, *boat* is closer to *dog* than *cat* is: a consequence of frequency, not of meaning.

### 8.3.2 Direction, not location

[Slides · Distributional semantics: 25–27]

A vector can also be seen as an **arrow from the origin**. Its *length* mostly reflects how frequent the word is; its *direction* reflects *the proportions* in which the word occurs with the different contexts, i.e. how it is used. So the direction is more important than the location. Two equivalent solutions:

- use the **angle** $\alpha$ between two vectors as a distance measure;
- or **normalise** the length $\|\mathbf{x}\|$ of every arrow to the same value, and then compare points (the circles on the arc in the figure).

**Definition — Cosine similarity**

The **cosine similarity** of two vectors is the cosine of the angle between them:

$$\cos\alpha = \frac{\mathbf{x}\cdot\mathbf{y}}{\|\mathbf{x}\|\,\|\mathbf{y}\|} = \frac{\sum_i x_i y_i}{\sqrt{\sum_i x_i^2}\,\sqrt{\sum_i y_i^2}}.$$

It is 1 for vectors pointing in the same direction, 0 for orthogonal vectors (with non-negative counts it is never negative), and it does not change if a vector is multiplied by a positive number: it ignores frequency.

**Worked example — dog vs. cat and dog vs. knife in two dimensions**

$$\cos(\text{dog}, \text{cat}) = \frac{115\cdot 52 + 10\cdot 4}{\sqrt{115^2+10^2}\,\sqrt{52^2+4^2}} = \frac{6020}{115.43\cdot 52.15} \approx 1.000 \quad (\alpha \approx 0.6^\circ)$$
$$\cos(\text{dog}, \text{knife}) = \frac{115\cdot 51 + 10\cdot 84}{115.43\cdot 98.27} = \frac{6705}{11343} \approx 0.591 \quad (\alpha \approx 53.8^\circ)$$

Now *cat* is (in these two dimensions) practically identical to *dog*, and *knife* is far. The slides report $\alpha = 54.3^\circ$ for knife–dog; computing it from the counts gives $53.8^\circ$, the small difference does not matter.

**Why this matters for NLP**

Every modern word representation, from count-based DSMs to **word embeddings** (*word2vec*, GloVe) to the internal vectors of BERT and GPT, follows this recipe: a word is a vector learnt from its contexts, and similar words have vectors with a small angle between them. Word2vec learns short dense vectors (a few hundred dimensions) directly, instead of counting; implementations in Python and C, with examples, are linked in the course's AulaWeb module.

**Cheat sheet — distributional semantics**

- Wittgenstein: meaning = use; Firth: "you shall know a word by the company it keeps"; Harris: difference of meaning correlates with difference of distribution.
- *bardiwac*: the meaning of an unknown word can be inferred from its contexts alone.
- Co-occurrence matrix: rows = target words, columns = contexts, cells = counts; each row is the word's vector. Weight the counts (e.g. $\log(1+f)$) before comparing.
- Euclidean distance is dominated by frequency; use the **angle**, i.e. **cosine similarity** $\mathbf{x}\cdot\mathbf{y}/(\|\mathbf{x}\|\|\mathbf{y}\|)$, or normalise the vectors.
- This is the basis of word embeddings (word2vec) and of the representations in LLMs.

# 9 Frame and Model-Theoretic Semantics

[Slides · Other semantics: 1–44; Semantics (long version): 7–10]

Distributional semantics (Section 8) tells us that *sold* is similar to *bought*, but not *who* sold *what* to *whom*, nor whether a sentence is *true*. This section presents two other views of meaning, following slides 36–56 of Percy Liang's tutorial *Natural Language Understanding: Foundations and State-of-the-Art* (Simons Institute, 2017, `https://simons.berkeley.edu/talks/percy-liang-01-27-2017-1`). The tutorial covers, in order, properties of language, distributional semantics, frame semantics, model-theoretic semantics, interactive learning and reflections; we use the parts on frame and model-theoretic semantics.

This section covers:

1. **frame semantics** and semantic role labelling (Section 9.1);
2. a comparison of the three types of semantics on one sentence (Section 9.2);
3. **model-theoretic semantics**: executable semantic parsing and compositionality (Section 9.3).

## 9.1 Frame Semantics

[Slides · Other semantics: 2–18; Semantics (long version): 10]

### 9.1.1 Word meaning revisited

[Slides · Other semantics: 2–6]

What is the meaning of *sold*?

- **Distributional semantics**: all the contexts in which *sold* occurs (*... was sold by...*, *... sold me that piece of...*). It can find similar words and contexts and generalise (e.g. with dimensionality reduction), but the representation is **monolithic**: a word vector has no internal structure, nothing in it says "there is a seller and a buyer".
- **Frame semantics**: the meaning is given by a **frame**, a stereotypical situation with **roles** (slots) to be filled.

**Definition — Frame**

A **frame** is a structured representation of a stereotypical situation, made of named **roles** (frame elements). A word *evokes* the frame, and the sentence fills its roles.

**Worked example — a commercial transaction**

*Cynthia sold the bike for $200.*

**Commercial transaction**

| | |
|---|---|
| SELLER | Cynthia |
| BUYER | ? |
| GOODS | the bike |
| PRICE | $200 |

The BUYER role stays empty: the sentence does not say who bought the bike, but the frame tells us that *someone* did.

### 9.1.2 Two properties of frames

[Slides · Other semantics: 7–11]

Following Fillmore (1977) and Langacker (1987):

- **Prototypical**: a frame does not need to handle all the cases. The frame of *widow* is "a woman marries one man, and the man dies". What if a woman had three husbands, two of whom died? The frame does not say, and it does not need to: it describes the typical situation.
- **Profiling**: words evoking the same frame can *highlight* different aspects of it.
  - *sell* is seller-centric, *buy* is buyer-centric: *Cynthia sold the bike (to Bob)* vs. *Bob bought the bike (from Cynthia)*;
  - *rob* highlights the person, *steal* the goods: *Cynthia robbed Bob (of the bike)* vs. *Cynthia stole the bike (from Bob)*.

The profiled role becomes the subject or object; the others are optional (in brackets).

### 9.1.3 From syntax to semantics

[Slides · Other semantics: 12–15]

The same situation, with SELLER Cynthia, BUYER Bob, GOODS the bike, PRICE $200, can be expressed by many **syntactic alternations**, with different verbs and different arguments in different positions:

- Cynthia sold the bike to Bob for $200.
- The bike sold for $200.
- Bob bought the bike from Cynthia.
- The bike was bought by Bob.
- The bike was bought for $200.
- The bike was bought for $200 by Bob.

The subject is the seller in the first sentence, the goods in the second, the buyer in the third. **Goal**: map *syntactic positions* to *semantic roles*.

### 9.1.4 Semantic role labelling

[Slides · Other semantics: 16–18]

**Definition — Semantic role labelling (SRL)**

Given a sentence, label each relevant phrase with its semantic role in the frame evoked by the predicate:

| | | | | | |
|---|---|---|---|---|---|
| **Input:** | Cynthia | sold | the bike | to Bob | for $200 |
| **Output:** | SELLER | PREDICATE | GOODS | BUYER | PRICE |

Subtasks:

1. **frame identification**: find the PREDICATE and the frame it evokes;
2. **argument identification**: find the phrases filling SELLER, GOODS, etc.

**Intuition — SRL needs syntax**

*Bob* is the BUYER in "Cynthia sold the bike *to Bob*" and in "*Bob* bought the bike", but in different syntactic positions. SRL systems therefore use the output of POS tagging and parsing (Section 7): syntax is the input of semantics.

**Tools**: **FrameNet** (`https://framenet.icsi.berkeley.edu/`) is a lexical database of English frames, with their roles and annotated example sentences; see for example the frame *Commerce_buy*.

## 9.2 Three Types of Semantics

[Slides · Other semantics: 19–22]

One sentence, three views:

***Every** non-blue block is next to **some** blue block.*

- **Distributional semantics**: *block* is like *brick*, *some* is like *every*. True as far as usage goes (they appear in similar contexts), and fatal for meaning: swapping *every* and *some* changes what the sentence says completely.
- **Frame semantics**: *is next to* has two arguments, *block* and *block*. It identifies the structure, but not the truth.
- **Model-theoretic semantics**: tells the difference between these two situations ("models"):




world A: the sentence is **true** world B: the sentence is **false**

Figure 11: Two worlds; blocks 1–2 and 3–4 are next to each other.

**Worked example — evaluating the sentence in each world**

In logic the sentence is

$$\forall x \left(\neg blue(x) \rightarrow \exists y \left(blue(y) \wedge nextTo(x, y)\right)\right).$$

The non-blue blocks are 1 and 3 in world A, 1, 2 and 3 in world B.

- World A: block 1 is next to 2 (blue), block 3 is next to 4 (blue): every non-blue block has a blue neighbour. **True**.
- World B: block 1 is next to 2, which is red, and to nothing else. **False** (one counterexample is enough for $\forall$).

**Definition — Model-theoretic semantics**

The meaning of a sentence is given by its **truth conditions**: the set of situations (*models* of the world) in which it is true. Understanding a sentence means being able to decide, given a model, whether it is true; more generally, to map the sentence to a formal representation (a logical form, a query, a program) that can be *evaluated* against the world.

| **Type** | **Represents a meaning as…** | **Good at / misses** |
|---|---|---|
| Distributional | a vector of contexts (Section 8) | similarity, learnt automatically / no structure, no truth |
| Frame | a frame with roles (Section 9.1) | who did what to whom / no logic (*every*, *not*) |
| Model-theoretic | truth conditions, a logical form or program | precise reasoning, answers / hard to obtain from free text |

Table 14: The three types of semantics compared.

## 9.3 Model-Theoretic Semantics in Practice: Semantic Parsing

[Slides · Other semantics: 23–44; Semantics (long version): 10]

### 9.3.1 Executable semantic parsing

[Slides · Other semantics: 23–34]

**Definition — Executable semantic parsing**

**Semantic parsing** maps a natural-language sentence to a **program** (logical form) in a formal language. The program is then **executed** in a context (a database, a calendar, a robot's world) to produce a behaviour: an answer, an action.

$$[\text{context}] + [\text{sentence}] \xrightarrow{\text{semantic parsing}} [\text{program}] \xrightarrow{\text{execute}} [\text{behaviour}]$$

**Worked example — a question to a database**

*What is the largest city in Europe by population?* The program is built piece by piece:

- *Cities*: the set of all cities;
- $\mathit{ContainedBy}(\mathit{Europe})$: the set of things located in Europe;
- $\mathit{Cities} \cap \mathit{ContainedBy}(\mathit{Europe})$: cities in Europe;
- the one with the largest population:
$$\arg\max\big(\mathit{Cities} \cap \mathit{ContainedBy}(\mathit{Europe}),\ \mathit{Population}\big).$$

Executed on a geographical database it returns **Istanbul**.

**Worked example — a command to a calendar**

*Remind me to buy milk after my last meeting on Monday.*

$$\mathit{Add}\big(\mathit{Buy}(\mathit{Milk}),\ \arg\max(\mathit{Meetings} \cap \mathit{HasDate}(\text{2016-07-18}),\ \mathit{EndTime})\big)$$

"My last meeting on Monday" becomes the meeting of that date with the largest end time; executing the program adds the reminder.

Note the context: "Monday" became a precise date (2016-07-18) only because the system knows today's date; "my meetings" are the ones in *this* user's calendar.

**Why this matters for NLP**

This is exactly what ChatBDI does (Section 2.5): it turns a sentence into a term such as `tire_pressure(front, _)` that an agent can execute. It is also how voice assistants and "text-to-SQL" systems work, and how LLMs call tools (they generate a function call, which is then executed).

### 9.3.2 A brief history of semantic parsing

[Slides · Other semantics: 35]

| | |
|---|---|
| GeoQuery | Zelle & Mooney 1996 |
| Inductive logic programming | Tang & Mooney 2001 |
| CCG | Zettlemoyer & Collins 2005 |
| String kernels | Kate & Mooney 2006 |
| Synchronous grammars; relaxed CCG | Wong & Mooney 2007; Zettlemoyer & Collins 2007 |
| Learning from the world | Clarke et al. 2010 |
| Higher-order unification; learning from answers | Kwiatkowski et al. 2011; Liang et al. 2011 |
| Language + vision | Matuszek et al. 2012 |
| Large-scale KBs; regular expressions; instruction following | 2013 (Berant et al., Kwiatkowski et al.; Kushman et al.; Artzi & Zettlemoyer) |
| Reduction to paraphrasing | Berant & Liang 2014 |
| Compositionality on tables; dataset from logical forms | Pasupat & Liang 2015; Wang et al. 2015 |

The trend: from small hand-annotated datasets (GeoQuery: questions about US geography, each paired with its logical form) to learning from weaker supervision (only the answers, or the effect in the world) and to large knowledge bases.

### 9.3.3 Compositional semantics

[Slides · Other semantics: 36–41]

How do we get from the sentence to the program? The principle of **compositionality**, formalised by **Richard Montague**: the meaning of a phrase is a function of the meanings of its parts and of the way they are combined syntactically.

**Worked example — cities in Europe**

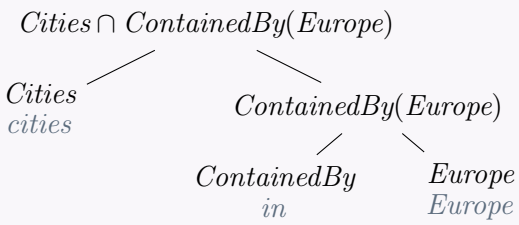



Figure 12: Bottom-up composition: each word gets a meaning from the lexicon (*cities* ↦ *Cities*, *in* ↦ *ContainedBy*, *Europe* ↦ *Europe*), and each node of the tree combines the meanings of its children (function application, then intersection).

The tree is the parse tree of Section 7.3, decorated with meanings: once again, syntax is the scaffolding of semantics.

### 9.3.4 Language variation

[Slides · Other semantics: 42–44]

The same meaning,

$$Cities \cap ContainedBy(Europe),$$

can be expressed in many ways:

- cities in Europe
- European cities
- cities that are in Europe
- cities located in Europe
- cities on the European continent

Can we write composition rules for all of these by hand? The slides answer: **no!** *European* is a single word that must produce *ContainedBy*(*Europe*); *located in* and *on the . . . continent* are other ways of saying *in*; and natural language has endlessly many such variants. This is why modern semantic parsers *learn* the mapping from data (and today LLMs do it from examples in the prompt), keeping the logical form as the target.

**Tools**: for model-theoretic semantics the slides report "... not found...": there is no standard off-the-shelf tool comparable to FrameNet.

**Cheat sheet — frame and model-theoretic semantics**

- Distributional vectors are monolithic. **Frame semantics**: a word evokes a frame (stereotypical situation) with roles (SELLER, BUYER, GOODS, PRICE). Frames are **prototypical** (*widow*) and words **profile** roles (*sell*/*buy*, *rob*/*steal*).
- Many syntactic alternations, one frame: **semantic role labelling** maps syntactic positions to roles (frame identification + argument identification). Tool: FrameNet.
- "Every non-blue block is next to some blue block": distributional (block ≈ brick), frame (*next to*(block, block)), model-theoretic (true or false in a given world).
- **Model-theoretic semantics**: meaning = truth conditions. Executable semantic parsing: sentence → program → execution (database, calendar).
- **Compositionality** (Montague): meaning of the whole from the meanings of the parts along the parse tree. Language variation makes hand-written rules impossible: the mapping is learnt.

# 10 Lexical Semantics, WSD and NER

[Slides · Semantics (long version): 11–43]

**Watch out — source of this section**

This section comes from the *long* version of the semantics slides, which was not shown in class (the lessons used the shortened decks of Sections 8 and 9). It is included because it completes the semantics part of the course map (Section 3.3): lexical semantics, ontologies, word sense disambiguation and named entity recognition. Check with the lecturer how much of it is exam material.

Distributional semantics learns meaning from usage; frame and model-theoretic semantics build structured meanings of sentences. **Lexical semantics** studies the meanings of *words* and the relations among them, and organises them into resources such as WordNet. Two classic tasks use these resources: deciding which meaning of an ambiguous word is meant (WSD), and finding the names of people, places and organisations in a text (NER).

This section covers:

1. the lexicon and the relations among word meanings (Section 10.1);
2. thesauri and WordNet (Section 10.2);
3. word sense disambiguation and the Lesk algorithm (Section 10.3);
4. named entity recognition (Section 10.4).

## 10.1 The Lexicon and Lexical Relations

[Slides · Semantics (long version): 11–21]

### 10.1.1 Lexicon and lexical units

[Slides · Semantics (long version): 11–13]

Linguistic theories generally regard a human language as made of two parts:

- a **lexicon**: essentially a catalogue of the language's words (its *wordstock*). It is also thought to include bound morphemes, which cannot stand alone as words (most affixes, Section 4.1), and in some analyses compound words and certain idioms and collocations;
- a **grammar**: a system of rules that combine those words into meaningful sentences (Section 7.3).

Dictionaries try to list the lexicon in alphabetical order, usually without the bound morphemes.

**Definition — Lexical semantics**

The units of analysis of **lexical semantics** are **lexical units**: not only words but also sub-words (affixes) and larger units (compound words, phrases). Lexical units make up the lexicon. Lexical semantics studies how the meaning of lexical units correlates with the structure of the language (syntax), and how meanings relate to each other through **lexical relations**: hyponymy, hypernymy, synonymy,

antonymy, homonymy, ...

### 10.1.2 Synonymy

[Slides · Semantics (long version): 14]

**Definition — Synonymy**

**Synonymy** is similarity of meaning. Leibniz: two expressions are synonymous if substituting one for the other *never* changes the truth value of a sentence in which the substitution is made.

Such *global* synonymy is rare (a language with perfect synonyms would be redundant). A more useful notion is **synonymy relative to a context**: two expressions are synonymous in a linguistic context $C$ if substituting one for the other in $C$ does not alter the truth value. E.g. *board* and *plank* are synonyms in "he nailed a __ to the wall" but not in "the __ of directors". A consequence of defining synonymy by substitution: words of different syntactic categories cannot be synonyms (*quick* and *quickly* cannot replace each other).

### 10.1.3 Antonymy

[Slides · Semantics (long version): 15]

The antonym of a word $x$ is sometimes *not-x*, but not always: *rich* and *poor* are antonyms, but *not rich* does not imply *poor* (many people consider themselves neither). Antonymy is a lexical relation between *word forms*, not a semantic relation between concepts: *rise*/*fall* and *ascend*/*descend* are antonym pairs, but *rise*/*descend* sounds odd, even though the concepts are the same.

### 10.1.4 Hyponymy and hypernymy

[Slides · Semantics (long version): 16–17]

**Definition — Hyponymy and hypernymy**

**Hyponymy** is a semantic relation between word meanings: {maple} is a **hyponym** of {tree}. The inverse relation is **hypernymy**: {tree} is a **hypernym** of {maple}. Also called subordination/superordination, subset/superset, or the **IS-A** relation. Test: a native speaker must accept sentences of the form "An $x$ is a (kind of) $y$". For verbs the relation is called **troponymy** (*to lisp* is a manner of *to talk*).

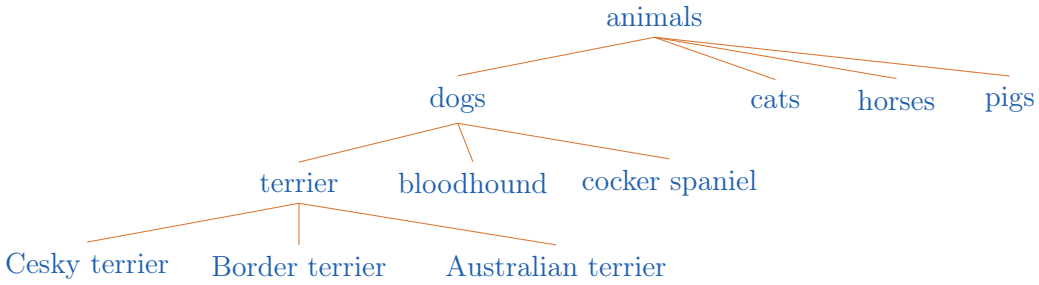



Figure 13: A hyponymy (IS-A) hierarchy: a Border terrier is a terrier, a terrier is a dog, a dog is an animal. Hyponymy is transitive.

### 10.1.5 Meronymy and holonymy

[Slides · Semantics (long version): 18–20]

**Definition — Meronymy and holonymy**

A concept $C_1$ is a **meronym** of a concept $C_2$ in a language $L$ if native speakers of $L$ accept sentences like "A $C_2$ has a $C_1$ (as a part)" or "A $C_1$ is a part of a $C_2$". The inverse relation is **holonymy**. It is the **HAS-AS-PART** relation, organised in a part hierarchy (e.g. *car* has *engine*, *door*, *wheel*, *tyres*; *engine* has *cylinder*, *exhaust valve*, ...).

Part-of is asymmetric and, *with caution*, transitive. Transitivity fails when different kinds of part-whole relation are chained:

- A musician has an arm. (component of a body)
- An orchestra has a musician. (member of a group)
- but: ? An orchestra has an arm.

### 10.1.6 Homonymy

[Slides · Semantics (long version): 21]

**Definition — Homonyms, homophones, homographs**

- **Homonyms**: the same word form with multiple unrelated meanings.
- **Homophones**: words that *sound* alike but are written differently.
- **Homographs**: words with the same spelling, possibly different pronunciation, and different meanings.

| Homonyms | Homophones | Homographs |
|---|---|---|
| the *spruce* tree / to *spruce* up | *addition* for maths / *edition* of a book | *desert* = abandon / area of land |
| *suit* yourself / wore a *suit* | I want *to* go / I like it *too* / one plus one is *two* | *bass* = fish / instrument |
| weigh on the *scale* / *scale* the wall | *capitol* building / state *capital* | *close* = nearby / to shut |
| the price is *fair* / go to the *fair* | pick a *flower* / bake with *flour* | *bow* = to bend down / ribbon |

**Why this matters for NLP**

Homographs and homonyms are a problem for every task that looks at written forms: a search for *bass* returns both fish and guitars (low precision, Section 4.4), and a distributional vector for *bass* mixes the contexts of all its meanings. Choosing the right meaning in context is word sense disambiguation (Section 10.3).

## 10.2 Thesauri and WordNet

[Slides · Semantics (long version): 22–30]

### 10.2.1 Structured lexicons and thesauri

[Slides · Semantics (long version): 22–23]

A **thesaurus** (structured lexicon) is an alternative to the alphabetical dictionary: a list of words grouped by *meaning*, with a **hierarchical** organisation, like the taxonomies of the natural sciences (children are "types of" the father and inherit its properties). For ordinary words the hierarchy is given by hyponymy and synonymy (Section 10.1).

- It shows the structure of lexical knowledge, and is much used for technical terminology.
- It can be enriched with other relations: antonyms, part/member relations (*has-as-part*, *has-as-member*), not just *is-a-type-of*.
- It can be explored in any direction (up, down, across), and obvious **distance metrics** (e.g. the number of links between two words) measure their similarity.

### 10.2.2 WordNet

[Slides · Semantics (long version): 24–30]

**WordNet** is a large lexical database of English, started in **1985** at Princeton University by a group of psychologists and linguists (led by George Miller, the author of the quotation in Section 8.1). Its theoretical basis is psycholinguistics: what are the properties of the *mental lexicon*? Global organisation: **nouns** are organised as topical hierarchies, **verbs** by entailment relations.

**Definition — Synset**

In WordNet the basic unit is the **synset** (synonym set). A word meaning is represented simply by listing the word forms that can express it. Synsets are unambiguous designators: the two main senses of *board* are

{board, plank, . . . } (a piece of lumber) vs. {board, committee, . . . } (a group of people assembled for some purpose).

- Members of a synset are rarely true synonyms: WordNet does not try to capture subtle distinctions (connotation, collocation) among them.
- Synsets are often enough to *differentiate* senses. If no suitable synonym exists, a short **gloss** is used, e.g. {board, (a person's meals, provided regularly for money)}. WordNet gives a gloss for every meaning, and often an example sentence.

**Worked example — the noun senses of *board* in WordNet (excerpt)**

- board (a committee having supervisory powers) "*the board has seven members*";
- board, plank (a stout length of sawn timber; made in a wide variety of sizes and used for many purposes);
- board (a flat piece of material designed for a special purpose) "*he nailed boards across the windows*";
- board, table (food or meals in general) "*she sets a fine table*";
- display panel, display board, board (a vertical surface on which information can be displayed to public view);
- dining table, board (a table at which meals are served);
- control panel, instrument panel, control board, board, panel (electrical device consisting of a flat insulated surface that contains switches and dials).

For the first sense WordNet also lists its relations: its members (*director*, *board member*: meronyms), its hypernym (*committee*), its sister terms.

**Size.** WordNet 3.0:

| | **Words** | **Synsets** | **Word–sense pairs** | **Monos. words** | **Polys. words** | **Polys. senses** |
|---|---|---|---|---|---|---|
| Noun | 117 798 | 82 115 | 146 312 | 101 863 | 15 935 | 44 449 |
| Verb | 11 529 | 13 767 | 25 047 | 6 277 | 5 252 | 18 770 |
| Adjective | 21 479 | 18 156 | 30 002 | 16 503 | 4 976 | 14 399 |
| Adverb | 4 481 | 3 621 | 5 580 | 3 748 | 733 | 1 832 |
| Total | 155 287 | 117 659 | 206 941 | 128 391 | 26 896 | 79 450 |

Most words have a single sense (128 391 of 155 287), but the polysemous ones are the frequent ones: verbs in particular have on average 18 770/5 252 $\approx$ 3.6 senses each.

| POS | Lexical relations in WordNet |
|---|---|
| Nouns | synonym ∼ antonym (opposite of); hypernym (is a kind of) ∼ hyponym (for example); coordinate (sister) terms: share the same hypernym; holonym (is part of) ∼ meronym (has as part) |
| Verbs | synonym ∼ antonym; hypernym ∼ troponym (*lisp* – *talk*); entailment (*snore* – *sleep*: if you snore you sleep); coordinate terms |
| Adjectives, adverbs | the above, plus related nouns, verb participles, derivational information |

WordNet is available in NLTK (Section 6.1) and online ("WordNet Search"); for Italian and many other languages, multilingual resources such as BabelNet extend it.

## 10.3 Word Sense Disambiguation

[Slides · Semantics (long version): 31–39]

**Definition — Word sense disambiguation (WSD)**

**Word sense disambiguation** is the task of identifying which sense (meaning) of a word is used in a sentence, when the word has several meanings. It is an open problem of NLP and ontology engineering.

Consider the senses of *bass* (WordNet lists eight noun senses: the lowest part of the musical range, the lowest part in polyphonic music, an adult male singer with the lowest voice, the lean flesh of a saltwater fish, any of various North American freshwater fish, the member with the lowest range of a family of instruments, ...) and the two sentences

- I went fishing for some sea bass.
- The bass line of the song is too weak.

To a human it is obvious that the first sentence uses *bass* (fish) and the second *bass* (sound/instrument). Replicating this ability with an algorithm is difficult, as shown even by the implicit equivocation between "bass (sound)" and "bass (musical instrument)": the senses themselves are not always clearly separated.

### 10.3.1 The Lesk algorithm

[Slides · Semantics (long version): 35–37]

The **Lesk algorithm** (Michael Lesk, 1986, "Automatic sense disambiguation using machine readable dictionaries: how to tell a pine cone from an ice cream cone") is the classical WSD algorithm. It assumes that words in a given "neighbourhood" (a section of text) tend to share a common topic.

**Definition — Simplified Lesk**

For every sense of the word to disambiguate, count the words that occur both in the **neighbourhood** of the word and in the **dictionary definition** (gloss) of that sense. Choose the sense with the largest count.

**Worked example — pine cone**

In the original version the glosses of *two* ambiguous neighbouring words are compared with each other. Context: "pine cone". Dictionary definitions:

| | |
|---|---|
| **pine** 1 | kinds of evergreen tree with needle-shaped leaves |
| **pine** 2 | waste away through sorrow or illness |
| **cone** 1 | solid body which narrows to a point |
| **cone** 2 | something of this shape whether solid or hollow |
| **cone** 3 | fruit of certain evergreen trees |

The best intersection is pine 1 ∩ cone 3 = {evergreen, tree(s)}, with 2 words: *pine* is the tree and *cone* is its fruit. All other pairs share no content word.

**Watch out — tree vs. trees**

The overlap of 2 counts *tree* and *trees* as the same word. A literal string comparison would find only *evergreen*. Lesk needs the normalisation of Section 6.3 (and stop-word removal, Section 6.2, otherwise *of* and *which* would count as overlaps too).

**Worked example — simplified Lesk on "I went fishing for some sea bass"**

After removing stop words and stemming, the context is {go, fish, sea}. Glosses (with the synset members) of three senses of *bass*:

- *sea bass, bass*: the lean flesh of a saltwater fish of the family Serranidae → overlap {sea, fish} = 2;
- *freshwater bass, bass*: any of various North American freshwater fish with lean flesh → overlap {fish} = 1;
- *bass*: the lowest part of the musical range → overlap 0.

Simplified Lesk chooses the sea bass. Note that it found *sea* only thanks to the synset member *sea bass*: *saltwater* and *sea* do not match as strings.

### 10.3.2 Extensions and limitations

[Slides · Semantics (long version): 38–39]

- **Adapted Lesk** (Banerjee and Pedersen, 2002): uses WordNet, comparing also the glosses of related synsets (hypernyms, hyponyms, ...), which gives more words to overlap.
- **Limitations**: Lesk is very sensitive to the exact wording of definitions: the absence of a single word can change the result radically. And it only looks at

overlaps among the glosses of the senses considered, which are short and do not provide enough vocabulary for fine-grained sense distinctions.

- Many later variants use other resources (thesauri, synonym dictionaries, morphological and syntactic models), and today distributional/contextual representations (Section 8) are the standard approach.

## 10.4 Named Entity Recognition

[Slides · Semantics (long version): 40–43]

**Definition — Named entity recognition (NER)**

**Named entity recognition** locates and classifies the **named entities** of a text into predefined categories: names of persons, organisations, locations, expressions of times, quantities, monetary values, percentages, etc.

Most NER systems take an unannotated block of text and produce an annotated one:

Jim bought 300 shares of Acme Corp. in 2006.

↓

$[\text{Jim}]_{\text{PERSON}}$ bought 300 shares of $[\text{Acme Corp.}]_{\text{ORGANIZATION}}$ in $[\text{2006}]_{\text{TIME}}$.

Online demos: Stanford NER (`http://nlp.stanford.edu:8080/ner/process`), spaCy NER; for Italian (not only NER): `http://www.italianlp.it/`, `http://parli.di.unito.it/link_it.html`.

**Approaches.**

- **Grammar-based** (hand-crafted rules by linguists): typically better **precision**, but lower **recall** and months of work by experienced computational linguists.
- **Statistical** (machine learning): need a large amount of manually annotated training data.
- **Semi-supervised** approaches avoid part of the annotation effort.

**Intuition — why NER is not a dictionary lookup**

A list of names is not enough: *Washington* can be a person, a city, a state or a government ("Washington announced…"); *Apple* can be a fruit or a company; new names appear every day. NER needs context, like POS tagging and WSD: in fact it is often solved with the same sequence models (HMMs, MEMMs and their successors, Section 7.4), with tags such as B-PER, I-PER, O marking the beginning and inside of each entity.

**Cheat sheet — lexical semantics, WSD, NER**

- Language = lexicon + grammar. Lexical relations: **synonymy** (substitution without changing truth, usually only in context), **antonymy** (between word forms), **hyponymy/hypernymy** (IS-A; troponymy for verbs), **meronymy/holonymy** (HAS-AS-PART; transitive only with caution), homonyms/homophones/homographs.
- Thesauri organise words by meaning in hierarchies. **WordNet**: synsets + glosses + relations; nouns in IS-A hierarchies, verbs by entailment.
- **WSD**: choose the sense in context. **Lesk**: the sense whose gloss overlaps most with the context (or with the glosses of neighbouring words); sensitive to wording, needs normalisation.
- **NER**: find and classify names, times, quantities; rules (precision) vs. statistical (needs annotated data) vs. semi-supervised.